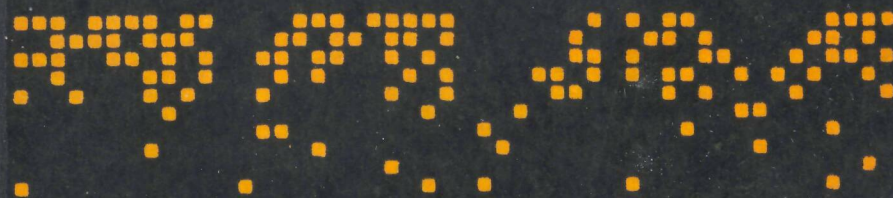




Л. К. Гребенников, Л. А. Летник

**ПРОГРАММИРОВАНИЕ
МИКРОПРОЦЕССОРНЫХ СИСТЕМ
НА ЯЗЫКЕ ПЛ/М**



Л. Н. Гребенников, Л. А. Летник

**ПРОГРАММИРОВАНИЕ
МИКРОПРОЦЕССОРНЫХ СИСТЕМ
НА ЯЗЫКЕ ПЛ/М**



МОСКВА
„ФИНАНСЫ И СТАТИСТИКА“
1986

Рецензент: С. П. ПРОХОРОВ

ПРЕДИСЛОВИЕ

Эта книга рассчитана на специалистов, занимающихся разработкой микропроцессорных систем и знакомых с их архитектурой и с основами программирования. Ее основная задача — помочь программистам быстрее освоить язык ПЛ/М, несколько отличающийся от языков, применяемых при работе с универсальными ЭВМ. В настоящее время ощущается большая потребность в привлечении профессиональных программистов к разработке программного обеспечения микропроцессорных систем, особенно систем, работающих в реальном времени. Освоение языка ПЛ/М позволит программистам работать над созданием подобных систем как группой (благодаря модульной структуре языка), так и самостоятельно.

В то же время данное пособие может оказаться полезным и для инженерно-технических работников, связанных с разработкой микропроцессорных систем различного уровня сложности, так как невозможно стать высококвалифицированным специалистом в этой области без серьезного изучения вопросов программирования.

Разработку программ для микропроцессорных систем, как известно, можно вести с помощью машинных команд, языка ассемблера и языков высокого уровня. Языки высокого уровня сокращают сроки разработки, дают возможность получать наглядные, легко читаемые тексты программ. В то же время объем загрузочных модулей, получаемых после трансляции программ, составленных на этих языках, значительно превышает объем модулей, созданных после трансляции с языка ассемблера.

Язык высокого уровня ПЛ/М, предназначенный для программирования микропроцессорных систем, созданных на базе микропроцессора серии КР580, позволяет строить загрузочные модули, которые по объему лишь

Гребенников Л. К., Летник Л. А.
Г79 Программирование микропроцессорных систем на языке ПЛ/М. — М: Финансы и статистика, 1986. 160 с.: ил.

В книге излагаются основы программирования на ПЛ/М — языке высокого уровня, предназначенного для программирования микропроцессорных систем, построенных на базе отечественного микропроцессора серии КР580. Особое внимание уделяется практическим приемам программирования, приводится большое количество примеров, позволяющих детально рассмотреть конструкции языка ПЛ/М.

Рассчитана на программистов и инженерно-технических работников, занимающихся вопросами проектирования микропроцессорных систем, а также на аспирантов и студентов, изучающих микропроцессоры и их программирование.

Г 2405000000—098
010(01)—86 —113—86

ББК 32.973

© Издательство «Финансы и статистика», 1986

незначительно превышают модули, полученные после трансляции с языка ассемблера. Главная цель, которую преследовали разработчики языка ПЛ/М, — предоставить в распоряжение инженерно-технических работников средство для быстрого создания системного и прикладного программного обеспечения микропроцессорных систем.

Как всякий алгоритмический язык, ПЛ/М является частью системы программирования, другая ее неотъемлемая часть — транслятор с этого языка. Применяются как резидентные системы трансляции, базирующиеся на микроЭВМ, например СМ-1800, так и кросс-системы, использующие в качестве инструментальных машины серии ЕС ЭВМ и СМ-4. Резидентные и кросс-трансляторы не являются конкурирующими, они дополняют друг друга.

Книга состоит из четырех глав. Первая посвящена вопросам, касающимся устройства микропроцессорных систем. Во второй главе, где приводится описание языка ПЛ/М, рассмотрено большинство конструкций языка, необходимых для программирования на этом языке и составления простейших программ. Третья глава касается практических приложений языка. Приводимые здесь примеры дают представление о создании системного и прикладного программного обеспечения. В четвертой главе описывается обработка программ с помощью как резидентных, так и кросс-средств.

Для удобства рассмотрения примеров в конце книги приводится указатель применения операторов, встроенных функций и процедур языка ПЛ/М, позволяющий быстро отыскать примеры использования отдельных средств языка.

Главы 1, 3 и 4 написаны Л. А. Летником, глава 2 — Л. К. Гребенниковым.

Авторы считают своим долгом выразить искреннюю признательность В. Н. Лебедеву и С. П. Прохорову за ценные замечания по первому варианту рукописи.



ГЛАВА 1

МИКРОПРОЦЕССОРНЫЕ СИСТЕМЫ. ИХ УСТРОЙСТВО И ПРОГРАММИРОВАНИЕ

1.1. МИКРОПРОЦЕССОРЫ

Успехи в области микроминиатюризации электронных схем привели к появлению в начале 70-х годов приборов, получивших название *микропроцессоров* (МП). Эти приборы пришли на смену логическим схемам, которые использовались для построения различных радиотехнических и вычислительных устройств. За очень короткое время МП стали элементарной базой радиоэлектронной аппаратуры. Это можно объяснить следующим фактором: МП, работающие под управлением программы, хранящейся в запоминающем устройстве, могут выполнять те же функции, что и устройства на логических схемах, но при этом изменение их функций осуществляется путем изменения программы, а это не требует переделок аппаратуры. Иными словами, на смену жесткой логике пришла программируемая.

Кроме гибкости применения, МП обладают и другими достоинствами — такими, как низкая стоимость, малые габариты, вес и потребляемая мощность, высокая точность, характерная для цифровых методов обработки, и т. д. Эти факторы привели к бурному проникновению МП в различные сферы человеческой деятельности.

Проектирование устройств, содержащих микропроцессоры, — довольно сложная задача, требующая от инженера-разработчика глубоких знаний не только в области логического проектирования и схемных решений, но и в области программирования. Точно так же программисты, участвующие в разработке микропроцессорных устройств, должны иметь представление об осо-

бенностях применения МП и тех отличиях, которые существуют между микропроцессорными устройствами и вычислительными машинами, с которыми им приходилось иметь дело раньше. Внутренняя логическая организация МП в общих чертах повторяет обычные ЭВМ, но выполнение МП в виде большой интегральной схемы с ограниченным числом выводов накладывает определенные ограничения на использование устройств, содержащих МП.

Простейшая структурная схема МП представлена на рис. 1.1. Арифметическо-логическое устройство (АЛУ) предназначено для выполнения таких действий, как сложение, вычитание, сдвиги, обнуление, загрузка, обмен и логические операции. Набор арифметических операций ограничен; это объясняется тем, что первоначально МП предназначались для построения управляющих устройств. Такие операции, как умножение, деление и другие, должны выполняться программно.

Регистры общего назначения (РОН) позволяют временно хранить промежуточные результаты, так как обращение к ним осуществляется значительно быстрее, чем к ячейкам памяти. В состав РОН входят: регистры временного хранения операндов или результатов выполнения операций, иногда называемые аккумуляторами, рабочие регистры, регистры счетчика команд и указателя стека, регистры косвенной адресации и т. д.

Схема управления, или устройство управления (УУ), выдает управляющие и синхронизирующие сигналы для всех блоков МП и внешних устройств.

Все блоки МП связаны между собой внутренними связями — магистралями, получившими название шин

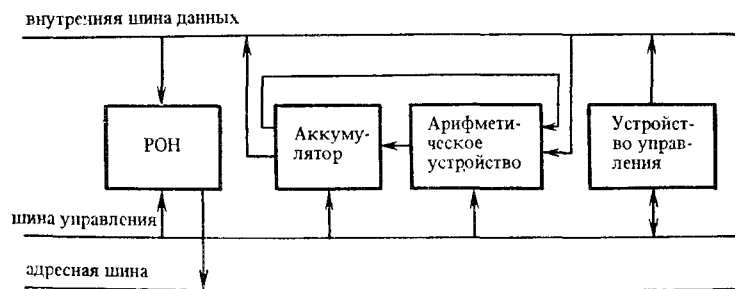


Рис. 1.1. Общая структурная схема МП:
РОН — регистры общего назначения

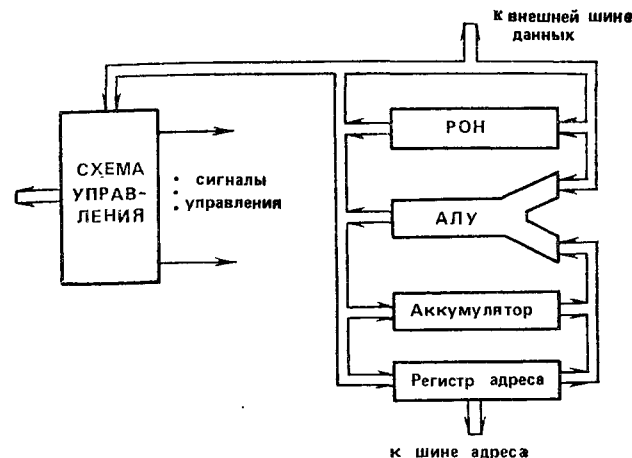


Рис. 1.2. Шинная организация МП:
АЛУ — арифметическо-логическое устройство

(рис. 1.2). Связь МП с внешними устройствами осуществляется через внешние шины данных, адреса и управления. Под *шинами МП* понимают группу линий передачи сигналов, выполняющих одинаковые функции, такие, как передача адреса, данных и сигналов управления.

Разрядность регистров МП соответствует разрядности шины данных, но поскольку разрядность шины адреса равняется 16 разрядам, то РОН можно попарно объединять и их содержимое передавать в регистр адреса.

Этот регистр связан непосредственно с шиной адреса, что позволяет формировать адреса во всех попарно объединяемых регистрах. В МП с 16-разрядной организацией шины данных адресная шина содержит 20 и более разрядов.

Для удобства построения микропроцессорных устройств промышленностью выпускаются наборы больших интегральных схем (БИС), легко сопрягаемых с данным МП, получивших название микропроцессорных комплектов. Для МП широкого применения число БИС комплекта может составлять несколько десятков штук.

1.2. МИКРОПРОЦЕССОР КР580

Для построения вычислительных систем, контроллеров и микроЭВМ, осуществляющих обработку цифровой информации, наиболее широко применяется МП серии КР580. Достаточно высокое быстродействие, 8-разрядный формат слова, низкая потребляемая мощность, разветвленная система команд обеспечивают возможность широкого использования этого МП в различных областях народного хозяйства. Кроме самого МП в комплект входит целый ряд БИС, позволяющих с минимальными затратами строить микропроцессорные устройства различной конфигурации.

Структурная схема МП КР580 состоит из регистров общего назначения и логических схем их выборки, арифметическо-логического устройства, регистра и дешифратора команд, устройства управления и буферов шин адреса и данных. Все блоки связаны между собой 8-разрядной внутренней шиной данных (рис. 1.3).

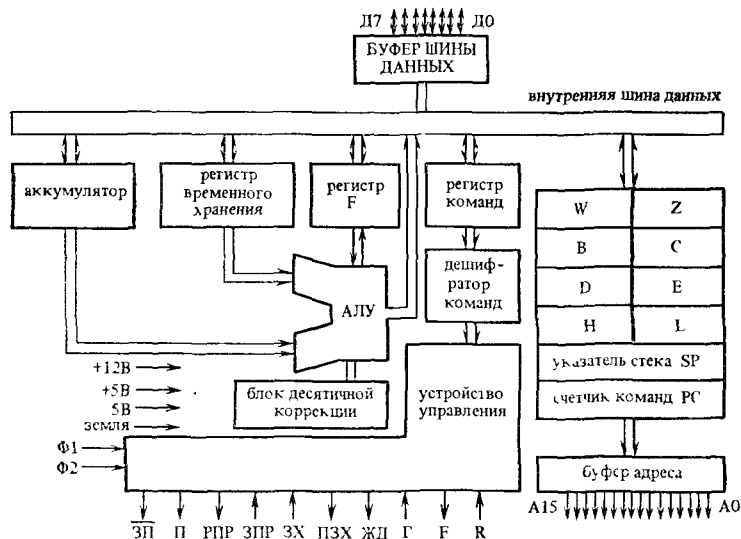


Рис. 1.3. Структурная схема МП КР580:

ЗП — запись; П — прием; РПР — разрешение прерывания; ЗПР — запрещение прерывания; ЗХ — захват; ПЗХ — подтверждение захвата; ЖД — ожидание; Г — готовность; F — синхронизация; R — сброс; Ф1, Ф2 — тактовые последовательности

РОН можно использовать для временного хранения информации и выполнения некоторых простых операций. При необходимости регистры В, С, D, E, H, L могут быть попарно объединены в три 16-разрядные пары ВС, DE и HL, причем пара HL может употребляться в качестве регистра адреса внешнего запоминающего устройства.

МП работает под управлением программы, хранящейся в запоминающем устройстве (ЗУ), а входящие в ее состав команды выполняются в общем случае по порядку, для чего их необходимо считывать из ЗУ последовательно, друг за другом. Для этих целей служит счетчик команд, содержимое которого после каждого цикла выборки автоматически увеличивается на 1.

Поясним назначение регистра указателя стека. Так как МП могут применяться для целей управления, то их работа в любой момент времени может быть приостановлена (режим прерывания) для получения и обработки информации от управляемых объектов. В связи с этим возникает необходимость в запоминании содержимого регистров, которое характеризуется так называемым словом состояния процессора — PSW. С его помощью после окончания прерывания можно продолжить выполнение прерванной программы. В МП КР580 для хранения PSW предназначена некоторая часть внешней памяти, получившая название *стек*. Запись в стек осуществляется по адресу, хранящемуся в указателе стека.

АЛУ управляется сигналами, поступающими из устройства управления (УУ) и настраивающими его на выполнение определенных операций над 8-разрядными операндами. В состав АЛУ входят: аккумулятор, или регистр А, регистр временного хранения данных, регистр признаков, или регистр флажков F, и десятичный корректор. Само АЛУ представляет собой комбинационную схему, выполняющую операции сложения, вычитания, логические операции И, ИЛИ, ИСКЛЮЧАЮЩЕЕ ИЛИ, НЕ и сдвиги.

При выполнении операций один операнд всегда находится в регистре А, а другой — в регистре временного хранения данных, причем в регистр временного хранения можно записать содержимое любого РОН или коды, хранящиеся в любой ячейке ЗУ, адресация которых осуществляется регистровой парой HL. Загрузка происходит по внутренней шине данных, соединяющей

все блоки МП. Кроме того, регистр временного хранения данных может выдавать информацию в регистр признаков F или на внутреннюю шину.

Регистр признаков F позволяет следить за результатами выполняемых операций, на основании которых осуществляется программный переход. В МП КР580 имеются следующие признаки, которые осуществляют установку или сброс триггеров регистра F:

C — признак переполнения, устанавливается в 1 при наличии переноса из старшего разряда;

S — признак знака, который совпадает со старшим разрядом результата в АЛУ;

Z — признак нуля, устанавливаемый в 1 при нулевом результате;

P — признак четности или паритета, определяющий число единиц в полученном результате: чётно $P=1$ и нечётно $P=0$;

AC — признак бита переноса, возникающий при переносе из младшего полубайта в старший.

Поясним назначение последнего признака. МП может выполнять операцию сложения над числами, представленными не только в двоичном коде, но и в двоично-десятичном формате. При этом, как известно, каждая десятичная цифра представляется четырьмя двоичными разрядами, в результате чего при сложении возможно образование запрещенных тетрад, значение которых превышает 9. В этом случае необходима корректировка. Перенос из младшей тетрады байта в старшую и прибавление 6 к младшей осуществляет блок десятичной корректировки, а признак AC при этом устанавливается в 1.

Устройство управления включает в себя 8-разрядный регистр команд и дешифратор. Регистр команд принимает команды с шины данных через буфер шины данных и внутреннюю шину МП. Кроме сигналов управления с дешифратора на входы УУ могут поступать четыре внешних управляющих сигнала: ГОТОВ (READY), ЗАПРОС ПРЕРЫВАНИЯ (INT), СБРОС (RESET) и ЗАХВАТ (HOLD), а также два синхросигнала Ф1 и Ф2. С выхода УУ выдаются шесть сигналов: РАЗРЕШЕНИЕ ПРЕРЫВАНИЯ (INTE), ПОДТВЕРЖДЕНИЕ ЗАХВАТА (INTA), ПРИЕМ С ШИНЫ ДАННЫХ (DBIN), СИНХРОНИЗАЦИЯ (F), ЗАПИСЬ (WR) и ОЖИДАНИЕ (WAIT).

Буфер шины данных предназначен для разделения

внутренней и внешней шин данных и двунаправленной передачи информации как в МП, так и из него. Для выполнения этих задач используется регистр временного хранения данных и выходная схема с тремя устойчивыми состояниями. Если информация поступает в МП, то внутренняя шина данных подсоединяется к буферному регистру. Если информация выводится из МП, то из буферного регистра, загруженного по внутренней шине данных, сигналы поступают на внешнюю шину данных. Как в первом, так и во втором случаях на шину должны выдаваться электрические сигналы, соответствующие логическому 0 и 1.

Возможен и третий режим работы, при котором в МП выполняются внутренние операции, не связанные с процедурами обмена с внешними устройствами. В этом случае внешняя шина данных должна отключаться от МП или переходить в состояние с высоким выходным сопротивлением «ни ноль, ни единица» (отсюда название подобных схем — схемы с тремя устойчивыми состояниями). У таких схем имеется управляющий вход, при подаче на который нуля осуществляется передача информации, а при подаче единицы схема переходит в состояние с высоким выходным сопротивлением.

Для связи МП с внешними устройствами используются шины. У МП КР580 имеется три шины (рис. 1.4): 16-разрядная шина адреса $A_{15}—A_0$, 8-разрядная двунаправленная шина данных и шина управления $U_{11}—U_0$, образуемая путем временного уплотнения шины данных.

По шинам адреса из МП передаются коды адреса, которые поступают на внешние устройства, готовые воспринять данный адрес.

К линиям адресной шины могут быть подсоединены входы нескольких внешних устройств, что требует определенных энергетических затрат. Так как к выходам МП КР580 можно подключать только один вход интегральных схем серии К155, то при увеличении нагрузки к линиям шины адреса должны подключаться магистральные усилители, получившие название *буферов*. Буфер будет отключать МП от внешней шины адреса, когда на него начнет поступать информация с других устройств, например при организации обмена между аналого-цифровыми (АЦП) или цифроаналоговыми преобразователями (ЦАП) с МП.

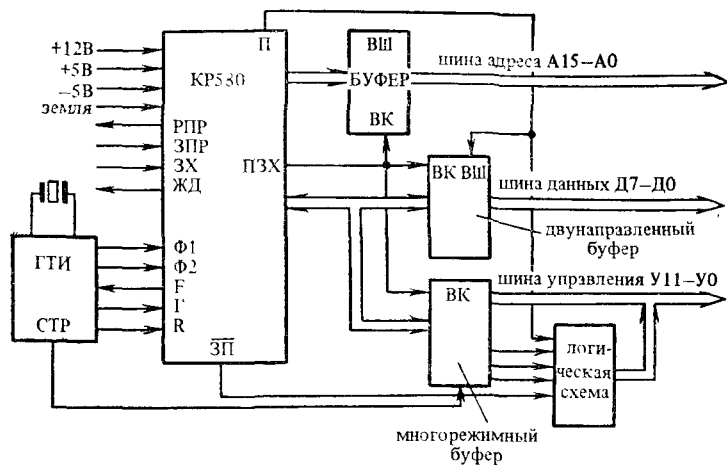


Рис. 1.4. Шинная организация МП КР580:
ГТИ — генератор тактовых импульсов; СТР — строб;
ВШ — выбор шины; ВК — выбор кристалла

Такое же назначение и шины данных, с той лишь разницей, что она позволяет передавать информацию в двух направлениях: от МП и к МП. Такое усложнение объясняется тем, что корпус, в который заключен МП, имеет ограниченное число выводов, и одни и те же выводы используются как входные и как выходные. Отсюда следует, что для шины данных требуется не простое буферное устройство, а двунаправленное. Направление передачи информации указывают сигнал $\overline{DVIN}$ с выхода МП (ПРИЕМ С ШИНЫ ДАННЫХ), сигнал $\overline{BK}$, осуществляющий выбор устройства или БИС, и один из сигналов PSW.

Сложность организации шины управления состоит в том, что малое число выводов из корпуса МП не позволяет иметь третью шину. В МП КР580 возможны мультиплексирование шины данных, т. е. временное ее уплотнение, и выдача на эту шину в определенные интервалы времени PSW. С шины данных код PSW принимается в регистр состояния, а из него в специальное внешнее логическое устройство. Таким образом, шина управления подключается не к выходам самого МП, а к некоторому логическому устройству, формирующему сигналы, подаваемые на шину управления.

Шина управления может иметь до двенадцати линий передачи, обеспечивающих работу микропроцессорной системы, которая включает не только МП, но и целый ряд вспомогательных устройств и БИС. Управляющие сигналы системы формируются из разрядов PSW и из выходных управляющих сигналов МП, при этом образуются, в частности, сигналы ЗАПИСЬ В ЗУ, СЧИТЫВАНИЕ ИЗ ЗУ, ЗАПИСЬ В УСТРОЙСТВО ВЫВОДА и СЧИТЫВАНИЕ ИЗ УСТРОЙСТВА ВВОДА.

1.3. МИКРОПРОЦЕССОРНЫЕ СИСТЕМЫ

Рассмотрим структурную схему микропроцессорной системы, представленную на рис. 1.5. В ее состав входят: генератор тактовых импульсов, микропроцессор КР580, шины, запоминающие устройства, а также различные устройства ввода и вывода информации.

Генератор тактовых импульсов, вырабатывающий две импульсные последовательности $\Phi 1$ и $\Phi 2$ и два управляющих сигнала ГОТОВ (Г) и СБРОС (R), имеет в своем составе кварцевый резонатор, обеспечивающий высокую стабильность получаемых сигналов.

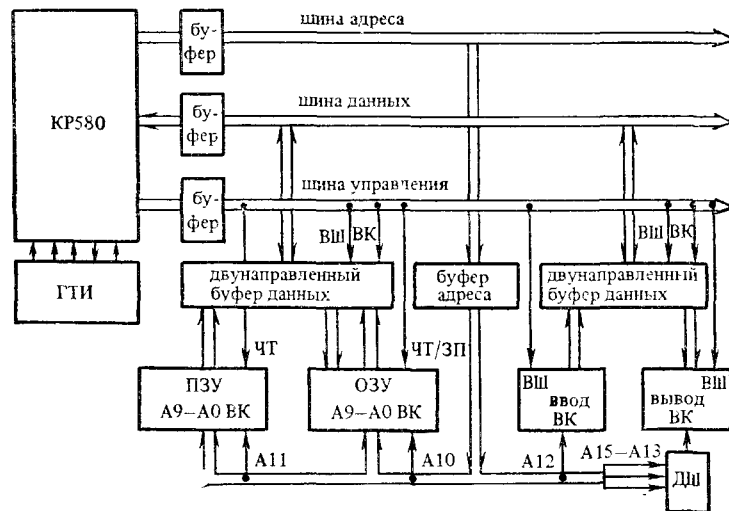


Рис. 1.5. Структурная схема МП системы:
ЧТ/ЗП — чтение или запись; ДШ — дешифратор

Запоминающие устройства микропроцессорной системы включают, как правило, оперативные ЗУ (ОЗУ) и постоянные ЗУ (ПЗУ). При рассмотрении ЗУ будем считать, например, что емкость ОЗУ и ПЗУ составляет 1024 однобайтных слова, или 1 Кбайт. Допустимая емкость памяти микропроцессорной системы, построенной на базе МП КР580, определяется разрядностью адресной шины и составляет $2^{16} \approx 65$ Кбайт. В рассматриваемом нами случае используется только часть адресного пространства. Выделим для ОЗУ область памяти с 0 по 1023 ячейку, или $(0000-03FF)_{16}$, а для ПЗУ — с 1024 по 2047 ячейку $(0400-04FF)_{16}$. Для записи адресов и данных употребляется и такая форма представления — 03FFH.

Пусть ОЗУ будет выполнено, например, на БИС типа КР565РУ2 емкостью по 1024 одноразрядных слова каждое, для чего понадобится восемь БИС, а ПЗУ будет выполняться на элементах К573РФ1 емкостью 1024 однобайтных слова. Для того чтобы обратиться к ЗУ, необходимо использовать не только сигналы с шины управления, которые выбирают и включают буферные устройства, но требуется еще и определять запоминающее устройство — ОЗУ или ПЗУ, так как они размещены в разных БИС. Для этих целей предназначен дешифратор, на одном выходе которого образуется сигнал, разрешающий чтение информации из ОЗУ, а на другом — сигнал, позволяющий использовать ПЗУ. Сигналы с выхода дешифратора подаются на специальные входы БИС ЗУ, называемые *выбором кристалла* (ВК), или *выбором микросхемы* (ВМ). Несколько сложнее управлять работой ОЗУ, так как оперативная память позволяет применять как режим записи информации, так и считывания: в зависимости от сигнала с шины управления — СЧИТЫВАНИЕ ИЗ ЗУ или ЗАПИСЬ В ЗУ.

Для обращения к устройствам ввода-вывода микропроцессорной системы в некоторых случаях можно применять сигналы, поступающие по шине адреса. Например, устройство ввода информации — клавиатура или вывода — дисплей могут иметь такие же адреса, как и любая ячейка ЗУ: 0800H (клавиатура) и 1000H (дисплей). При этом выбор внешнего устройства осуществляется по сигналу с дешифратора, у которого должно быть не два, а четыре выхода,

1.4. ПРОГРАММИРОВАНИЕ МИКРОПРОЦЕССОРНЫХ СИСТЕМ

Для рассмотрения вопросов программирования удобно использовать программную модель МП (рис. 1.6), входящего в состав микропроцессорной системы. Такая модель состоит из регистров, которые программно доступны пользователю, и внешних устройств. Знание программной модели и системы команд облегчает пользователю составление прикладных программ. Из рис. 1.6 следует, что в состав модели входят внутренние регистры В, С, D, E, H, L, ячейки памяти М, находящиеся в адресном пространстве МП и адресуемые парой регистров HL, аккумулятор и регистр признаков F. Регистры В-С, D-E и H-L, объединяясь попарно, образуют 16-разрядные регистры, именуемые как BC, DE и HL. Под внешними регистрами, называемыми *портами*, понимаются внешние устройства ввода-вывода, адресуемые так же, как и память.

Адресация памяти осуществляется кодами от $(0000)_{16}$ до $(FFFF)_{16}$, среди которых могут быть и внешние устройства ввода-вывода, например $(0800)_{16}$. При адресации портов ввода-вывода в МП КР580 предусматривается программная возможность приема и выдачи информации по шине данных в устройства, адресуемые восемью младшими (от A_7 до A_0) или восемью старшими (от A_{15} до A_8) разрядами шины данных. Это позволяет иметь $2^8 = 256$ портов ввода-вывода, причем обеспечивается дублирование адресов в младших и старших байтах шины адреса, что позволяет увеличить число подсоединяемых внешних устройств.

ВНУТРЕННИЕ РЕГИСТРЫ		ВНЕШНИЕ РЕГИСТРЫ		АДРЕСА ПРЕРЫВАНИЯ	СТЕК
B	C	0000H	00H	0000H	Область ОЗУ
D	E	0001H	:	0005H	
H	L	:	:		
M	PSW	:	FFH		
A		A4CFH	00		
F		:	:		
счетчик команд PC		:	:		
указатель стека SP		FFFFH	FFH		
		Память	Порты		

Рис. 1.6. Программная модель МП КР580

Для того чтобы не спутать адресацию к ЗУ и к портам, дополнительно используются управляющие сигналы шины управления. Программное обращение к внешним устройствам может осуществляться двояко: либо с помощью команд обращения к портам IN и OUT, либо с помощью команд пересылок и обмена, в которых внешним устройствам приписываются определенные адреса. В первом случае при обмене информацией с внешними устройствами достаточно двух команд, а во втором — используются все команды МП, предназначенные для работы с памятью микропроцессорной системы.

1.4.1. Система команд МП КР580

Система команд МП КР580 обеспечивает обработку цифровой информации, логические и арифметические операции, а также позволяет оперировать с данными, представленными в двухбайтном формате. Всего у МП имеется 78 команд, но с учетом модификаций, связанных с использованием различных регистров, их число достигает 244. Эти команды можно разделить на несколько групп по их функциональному назначению.

Команды межрегистровой передачи пересылают содержимое источника (регистр S) в регистр-получатель (регистр D). При пересылках содержимое регистра-источника и состояние триггеров регистра признаков F не изменяется. Общий вид команды: MOV D, S, где в качестве D и S можно указывать любые регистры, ячейки памяти и порты ввода-вывода. Командой MOV A, B пересылается содержимое регистра в регистр A, т. е. имеет место следующее действие: $A \leftarrow \langle B \rangle$, где $\leftarrow$ указывает на передачу содержимого в A. Код этой команды (78)₁₆. Команда MOV M, A (код (77)₁₆) передает содержимое регистра A в память, причем адрес ячейки памяти берется из пары регистров HL, т. е. $M[H, L] \leftarrow \langle A \rangle$. Команда загрузки MOV B, M выполняет пересылку содержимого памяти в регистр B, т. е. $B \leftarrow M[H, L]$.

Рассмотренные команды представляются восемью двоичными разрядами, т. е. одним байтом, но существуют команды, для представления которых в памяти используются 2 или 3 байта, располагающиеся последовательно друг за другом.

Команда передачи MVI D, <B₂> пересылает в ре-

гистр A содержимое второго байта, т. е. осуществляет непосредственную загрузку. Вместо регистра D может быть использован любой регистр.

Трехбайтная команда LDA загружает в регистр A содержимое ячейки памяти, адресуемой вторым и третьим байтами $A \leftarrow M[\langle B_3, B_2 \rangle]$.

В группу межрегистровых передач включены и команды ввода IN и вывода OUT, причем второй байт этих команд является адресом порта ввода или вывода.

Команды выполнения операций МП предназначены для арифметических и логических операций, а также для операций сдвигов.

Команды арифметических операций производят сложение и вычитание, а операции умножения и деления могут быть выполнены только с помощью подпрограмм, где применяется многократное повторение операций сложения или вычитания.

Сложение содержимого регистра A с содержимым любого регистра осуществляется однобайтной командой ADD R, где R — регистр МП. Например, ADD B имеет следующий смысл $A \leftarrow \langle A \rangle + \langle B \rangle$.

Вычитание производится командой SUB R: $(A \leftarrow \langle A \rangle - \langle B \rangle)$. Если уменьшаемое меньше вычитаемого, то выставляется признак C, т. е. C принимает значение, равное 1. При последовательной обработке многобайтных чисел применяются команды сложения с переносом

$$ADC R := (A \leftarrow \langle A \rangle + \langle B \rangle + C)$$

и вычитания с заемом

$$SBB R := (A \leftarrow \langle A \rangle - \langle B \rangle - C)$$

Двухбайтные арифметические операции позволяют выполнять действия над содержимым регистра A и непосредственно вторым байтом:

$$ADI R := (A \leftarrow \langle A \rangle + \langle B_2 \rangle);$$

$$SUI R := (A \leftarrow \langle A \rangle - \langle B_2 \rangle);$$

$$ACI R := (A \leftarrow \langle A \rangle + \langle B_2 \rangle + C);$$

$$SBI R := (A \leftarrow \langle A \rangle - \langle B_2 \rangle - C);$$

Команды логических операций и сдвигов поразрядные, они выполняются независимо для всех восьми разрядов операнда: команда $ANA R = (A \leftarrow \langle A \rangle \wedge \langle B \rangle)$ — для операции конъюнкции «И», $ORA R = (A \leftarrow \langle A \rangle \vee \langle B \rangle)$ — для операции дизъюнкции «ИЛИ», $XRA R = (A \leftarrow \langle A \rangle \vee \langle B \rangle)$ — для операции «исключительное ИЛИ».

Сдвиги осуществляются влево и вправо только на один разряд, причем существует возможность перемещать старший разряд, выдвигаемый при сдвиге влево за разрядную сетку, в младший разряд кода. Такая же возможность имеется и относительно младшего разряда, который может быть помещен в старший разряд при сдвиге вправо.

Команды передачи управления делятся на безусловные и условные переходы, осуществляемые по значению признаков.

Команды с двухбайтными операндами позволяют выполнять пересылку и обмен информацией между парами регистров, а также сложение содержимого пары HL с парами BC, DE и HL. Операция $DAD\ M:=(HL \leftarrow (HL) + \langle HL \rangle)$ удваивает содержимое пары HL, что эквивалентно сдвигу влево на один разряд.

Команды управления работой МП состоят из команд останова HALT, команд управления прерыванием EI и DI и холостой операции NOP.

1.4.2. Программирование в машинных кодах

Ниже представлены десять команд, которые позволяют рассмотреть простой пример составления программы в машинных кодах.

Наименование команды	Обозначение	Код	Описание
Пересылка из регистра в регистр	MOV R ₁ , R ₂	01DDSSSS	$R_1 \leftarrow \langle R_2 \rangle$
Пересылка из регистра в память	MOV M, R	01110SSS	$M \leftarrow \langle R \rangle$
Засылка в регистр	MVI R	00DDD110	$R \leftarrow \langle B_2 \rangle$
Засылка в память	MVI M	00110110	$M \leftarrow \langle B_2 \rangle$
Инкремент регистра	INR R	00DDD100	$R \leftarrow \langle R \rangle + 1$
Загрузка регистра А	LDA	00111010	$A \leftarrow \langle B_3, B_2 \rangle$
Загрузка регистров H, L	LXI H	00100001	$H, L \leftarrow \langle B_3 \rangle, \langle B_2 \rangle$
Загрузка памяти	STA	00110010	$M \langle B_3 \rangle, \langle B_2 \rangle \leftarrow A$
Сложение содержимого регистра и памяти	ADD M	10000110	$A \leftarrow \langle A \rangle + \langle M \rangle$
Переход по переполнению	JC	11011010	$PC \leftarrow \langle B_3, B_2 \rangle$, если $C=1$

Пусть требуется сложить два числа, находящиеся в ЗУ соответственно по адресу $(0101)_{16}$ и $(0102)_{16}$, а результат разместить по адресу $(0103)_{16}$. Составим программу, считая, что она будет размещена в памяти в ячейке с номером $(0200)_{16}$. Для пересылки числа из памяти можно употребить команду MOV A, M, но для ее выполнения необходимо заранее, т. е. до выполнения операции MOV A, M, поместить в регистровую пару HL двухбайтный адрес. Воспользуемся для этого командой $LDA\ R:=(HL \leftarrow \langle B_3, B_2 \rangle)$, позволяющей непосредственно во втором и третьем байтах записать адрес ЗУ, по которому размещается первое слагаемое. Запишем эту команду в шестнадцатеричной системе счисления:

Адрес	Код операции	Адрес B ₃	Адрес B ₂
0200	3A	01	01

После выполнения этой команды первое слагаемое, или операнд, расположится в регистре A. Сложение осуществляется по команде $ADD\ M:=(A \leftarrow \langle A \rangle + \langle M \rangle)$, ее код — $(86)_{16}$. Перед сложением в регистровой паре HL указывается адрес второго операнда с помощью команды $LXI\ H:=(H, L) \leftarrow \langle B_3, B_2 \rangle$, ее код — $(21)_{16}$. Ниже приводится запись этих команд в шестнадцатеричной системе счисления.

Адрес	Код операции	Адрес B ₃	Адрес B ₂
0203	21	01	02
0206	86	—	—

Результат на переполнение проверяется командой условного перехода JC:

Адрес	Код операции	Адрес B ₃	Адрес B ₂
0207	DA	04	00

Если имеет место переполнение, то осуществляется переход к ячейке с номером $(0400)_{16}$, которая выбрана условно, и ее адрес размещается во 2-м и 3-м байтах команды JC. Если переход отсутствует, то выполнение программы продолжается. Проверка на переполнение введена потому, что мы не знаем, какие операнды хранятся в ячейках $(0101)_{16}$ и $(0102)_{16}$. Далее необходимо из регистра A переслать результат в ЗУ по адресу $(0103)_{16}$ командой пересылки MOV M, A, но для этого

требуется изменить содержимое регистровой пары HL, где хранится адрес $(0102)_{16}$. Он на 1 меньше необходимого, поэтому его можно получить путем прибавления к содержимому регистра L единицы командой $INR L: = L \leftarrow (L + 1)$:

Адрес команды	Код операции	Адрес B_1	Адрес B_2
020A	2C	—	—
020B	77	—	—

Вся программа будет иметь вид:

Адрес команды	Код операции	Адрес B_1	Адрес B_2
0200	3A	01	01
0203	21	01	02
0206	86	—	—
0207	DA	04	00
020A	2C	—	—
020B	77	—	—

Размещение программы в памяти микропроцессорной системы представлено ниже.

Адрес команды	Код	Двоичное представление кода
0200	3A	0 0 1 1 1 0 1 0
0201	01	0 0 0 0 0 0 0 1
0202	01	0 0 0 0 0 0 0 1
0203	21	0 0 1 0 0 0 0 1
0204	02	0 0 0 0 0 0 1 0
0205	01	0 0 0 0 0 0 0 1
0206	86	1 0 0 0 0 1 1 0
0207	DA	1 1 0 1 1 0 1 0
0208	00	0 0 0 0 0 0 0 0
0209	04	0 0 0 0 0 1 0 0
020A	2C	0 0 1 0 1 1 0 0
020B	77	0 1 1 1 0 1 1 1

1.4.3. Языки программирования микропроцессорных систем

Для повышения эффективности программирования, уменьшения ошибок и упрощения самого программирования используются языки программирования нескольких уровней.

Основное достоинство языка ассемблера состоит в том, что система программирования, состоящая из языка ассемблера и служебной программы — транслятора, осуществляющего перевод с языка на машинный код,

позволяет получать загрузочные модули, объем которых практически не превосходит объема исходной программы.

Программа на языке ассемблера представляет собой последовательность предложений, каждое из которых может состоять из четырех полей: метки, оператора, операндов и комментариев.

Поле метки содержит имя оператора, необходимое для обращения к этому оператору при выполнении программы. В поле оператора записывается символический мнемокод выполняемой операции. В поле операндов должна содержаться информация, необходимая для выполнения команды, определяемой в поле оператора. Поле комментариев отводится для пояснений и удобства чтения программы.

Обязательным является только поле оператора, в котором записывается выполняемая операция, а остальные поля могут отсутствовать.

В языке ассемблера МП КР580 мнемонические коды операций совпадают с мнемокодами МП, что упрощает их использование. Для написания программы используются буквы латинского алфавита, арабские цифры и специальные знаки, а в качестве программно-доступных регистров — регистры общего назначения МП.

В качестве примера программы, написанной на языке ассемблера, рассмотрим умножение двух чисел [1]. Пусть в регистре HL располагаются частичные произведения и результат, в регистре E хранится множимое, а в регистре A — множитель. Для организации программного счетчика, подсчитывающего число сложений, используем регистр B. После выполнения программы значение множителя сохраняется в регистре A.

Как видно из представленной на с. 22 программы, даже наличие подробных комментариев не делают ее легко читаемой. Разработка программ на ассемблере, требующем от программиста знания мнемокодов данного МП, весьма трудоемка. В связи с этим на практике для программирования микропроцессорных систем используют языки высокого уровня: Бейсик, Фортран, ПЛ/М и т. д. Выбор того или иного языка определяется типом решаемой задачи. Так, например, если в разрабатываемой программе большое место занимают вычисления, то используется Бейсик или Фортран, а если решаемая задача носит управленческий характер, то больше подходит язык ПЛ/М.

MULT:	LXI	H,0	обнуление регистров HL
	MVI	B,8	загрузка в регистр В числа разрядов множителя
LOOP:	DAD	H	сдвиг влево на один разряд частичного произведения
	RAL		циклический сдвиг множителя влево
	JNC	DEC	переход при отсутствии переноса; C=0
	DAD	D	если C=0, производится сложение множимого и частичного произведения
	DCR	B	уменьшение на 1 содержимого счетчика
DEC:	ACI	0	восстановление множителя в аккумуляторе
	JNZ	LOOP	если в счетчике 0, то происходит выход из программы умножения, иначе переход к ...

Язык ПЛ/М обладает широкими возможностями, позволяющими решать самые разнообразные задания [2—6]. Кроме задач управления, ПЛ/М позволяет эффективно осуществлять системное программирование для микропроцессорных систем, разрабатывать мониторы, супервизоры, программное обеспечение баз данных и т. д. Этот язык предоставляет большие возможности для построения систем на базе МП КР580, приняты попытки распространить его на МП других серий. Достоинством этого языка являются:

- реализация методов структурного программирования;
- способность отражать те технологические функции, которые выполняет микропроцессорная система;
- обеспечение программного доступа к узлам МП;
- использование подпрограмм, написанных на языке ассемблера.

Язык ПЛ/М позволяет легко реализовать методы структурного программирования за счет применения операторных скобок DO — END, процедур и литералов. Для организации разветвлений в языке употребляется конструкция IF — THEN — ELSE.

Наличие литералов, процедур и операторных скобок позволяет сделать программу легко читаемой, написанной как бы на «естественном языке». Удобству восприятия и написания программ способствует также свободное размещение операторов языка ПЛ/М, по-

зволяющее делать отступы и пропуски строк при записи исходных программ.

Преимущества, которые предоставляет язык ПЛ/М разработчикам микропроцессорных систем, состоят в следующем:

- снижаются затраты на программирование;
- значительно сокращаются сроки создания программ;
- повышается надежность разрабатываемых программ за счет применения методов структурного программирования;
- улучшается документация программ и упрощается их сопровождение.

ГЛАВА 2

ОПИСАНИЕ ЯЗЫКА ПЛ/М

2.1. АЛФАВИТ

Основой любого языка программирования является *алфавит* — набор допустимых знаков (литер), которые можно использовать для записи программ и данных. Алфавит языка ПЛ/М включает следующие знаки (см. приложение):

26 латинских прописных букв от А до Z;
арабские цифры от 0 до 9 (ноль иногда перечеркивают, чтобы отличить от буквы O);
специальные знаки:
*=./ () + —, > < : ; ' и др.

Кроме того, в некоторых версиях языка дополнительно используются прописные русские буквы, а также знак денежной единицы (⌘) и (—) — символ разбивки в словах.

Знаки, не входящие в алфавит, недопустимы; они воспринимаются как пробелы.

Специальные знаки и некоторые их комбинации имеют особый смысл в программе, что видно из табл. 2.1. В памяти машины каждый знак алфавита изображается двоичным числом в коде КОИ-7 и занимает один байт (восемь двоичных разрядов, где старший разряд в коде КОИ-7 всегда содержит нулевое значение).

Неделимые последовательности знаков алфавита образуют слова, имеющие определенный смысл в программе. Слова в тексте программы разделяются пробелами или другими специальными знаками. Внутри слов пробелы недопустимы. В зависимости от назначения и роли в программе различают ключевые слова, идентификаторы и константы.

Изо- браже- ние знака	Название	Назначение
=	Равно	Оператор присваивания, знак операции отношения
:=	Присваивание	Внутренний оператор присваивания
.	Точка	Адресная ссылка
/	Наклонная черта	Знак операции деления
*	Звездочка	Знак операции умножения
+	Плюс	Знак операции сложения
-	Минус	Знак операции вычитания
<	Меньше	Знаки операции сравнения
>	Больше	
<=	Меньше или равно	
>=	Больше или равно	
<>	Не равно	Левый и правый ограничители в выражениях и в списках элементов
(	Левая скобка	
)	Правая скобка	Ограничитель строчной константы
'	Апостроф	
:	Двоеточие	Ограничитель метки
;	Запятая	Ограничитель элемента в списке
,	Точка с запятой	Ограничитель оператора
/*	Левый ограничитель	Левый и правый ограничители комментария
*/	Правый ограничитель	Разделитель в идентификаторах и числах
⌘	Знак денежной единицы	
—	Подчеркивание, символ разбивки	Разделитель в идентификаторах
	Пробел	Разделитель слов

Смысл ключевых слов установлен заранее при разработке языка. Во всех программах на языке ПЛ/М смысл ключевых слов одинаков. Смысл идентификаторов и констант программист устанавливает сам при составлении программы. В языке определены только правила записи идентификаторов и констант.

Последовательность слов, записанная в соответствии с синтаксическими правилами языка, образуют предложения-операторы. Определенным образом записанная последовательность предложений-операторов образует программу.

Любой набор знаков, над которым можно выполнять определенные действия, в программировании называют данными. Часть данных, с которой можно производить элементарные действия, например складывать с другой частью данных, называют элементом данных, или элементарным данным.

Конкретная последовательность знаков, образующая величину, есть значение этой величины. Различают переменные и постоянные величины. *Переменная* — это величина, которая в ходе выполнения программы может принимать разные значения. *Постоянная величина*, или *константа*, — это величина, которая в ходе выполнения программы всегда имеет только одно значение, которое не может быть изменено.

Обозначением константы в программе является она сама; обозначением константы-числа служит изображение самого числа.

Переменная величина в программе обозначается идентификатором. Идентификатор в языке ПЛ/М — это последовательность букв и цифр, начинающаяся с буквы длиной не более 31 знака. Имя переменной (идентификатор) выбирают так, чтобы оно отражало смысл переменной.

Примеры идентификаторов:

X Y Z10 SUMMA ITOG AMAX BETA

Внутри идентификаторов может размещаться специальный знак $\odot$, который используется в этом случае лишь для удобства чтения. При трансляции этот знак в записи идентификатора игнорируется, поэтому идентификатор, содержащий такой знак, тождествен такому же идентификатору без знака $\odot$. Например, идентификаторы SUM $\odot$ WES и SUMWES будут восприняты транслятором как эквивалентные.

Иногда для удобства чтения программы некоторые части идентификатора выделяют включением в последовательность знаков специального знака разбивки (подчеркивания). Тогда этот знак становится составной частью идентификатора. Например, идентификаторы НАЧАЛО_БЛОКА и НАЧАЛОБЛОКА будут восприняты транслятором как два разных идентификатора.

В языке ПЛ/М используется ряд слов, которые имеют раз и навсегда установленное значение в программе. Эти слова — ключевые, они являются составной частью языка, и их нельзя использовать для обозначения имен переменных. Список ключевых (зарезервированных) слов приведен в табл. 2.2, где указаны области их использования.

Таблица 2.2

Ключевые слова	Область использования	Ключевые слова	Область использования
IF THEN ELSE	Условный оператор	GO TO BY GOTO CASE WHILE	Безусловный переход и управление циклом
DO END	Ограничители группы операторов	OR AND XOR NOT	Знаки логических операций
PROCEDURE RETURN CALL INTERRUPT	Описание и использование процедур	HALT	Останов
DECLARE BYTE ADDRESS LABEL INITIAL DATA LITERALLY BASED	Описание данных	ENABLE	Разрешение прерываний
		DISABLE	Запрет прерываний
		MOD	Остаток от деления
		PLUS	Сложение с переносом
		MINUS	Вычитание с переносом
		EOF	Конец текста программы

2.4. ЧИСЛОВЫЕ КОНСТАНТЫ

Числовые константы — это целые числа, которые могут быть записаны в любой из следующих систем счисления: десятичной, двоичной, восьмеричной и шестнадцатеричной.

цатеричной. В любой системе счисления значение числовой константы не должно превосходить $2^{16}-1$ (65535).

Десятичная константа — это десятичное число, вслед за последней цифрой которого записывается буква D. Примеры:

17D 123D 6374D 936D 350D

Двоичная константа — это последовательность двоичных нулей и единиц длиной не более 16 цифр, вслед за последней цифрой записана буква B. Примеры:

0101101B 11010100B 11110101111101B

Восьмеричная константа — последовательность восьмеричных цифр (0, 1, 2, 3, 4, 5, 6, 7) длиной не более 5, за последней цифрой записана либо буква O, либо буква Q. Примеры:

773Q 67354Q 10774Q

Шестнадцатеричная константа — последовательность шестнадцатеричных цифр (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) длиной не более 4. За последней цифрой такой константы записывается буква H. Начальной литерой шестнадцатеричного числа должна быть цифра, чтобы исключить сходство константы с идентификатором. Поэтому в необходимых случаях перед первой шестнадцатеричной цифрой (A, B, C, D, E, F) такой константы записывают нуль. Примеры:

123H 0A7BH 0E1BH 77AH

В десятичной константе разрешается не записывать букву D за младшей цифрой числа, поскольку любое число, за которым не следует одна из букв D, B, Q, O или H, считается десятичным.

В некоторых случаях между цифрами константы удобно вставлять знак $\odot$. При трансляции он в константе игнорируется. В приведенном ниже примере две двоичные константы эквивалентны.

11101011 $\odot$ 11101110B
1110101111101110B

2.5. СТРОКИ

Строкой называют константу, представляющую собой последовательность любых знаков алфавита, заключенную в апострофы. Примеры:

'AB'
'PI'
'SUMMA'
'77TB'

В составе строки можно указывать апостроф, но для этого его необходимо записать дважды. Например, строка "'T'" содержит два знака: апостроф и букву T.

Транслятор размещает в памяти каждый знак строки в одном байте в коде КОИ-7, а вся строка размещается в последовательных байтах памяти без окаймляющих апострофов, например: 'A' эквивалентно 41H, 'AG' — 4147H.

2.6. ОПИСАНИЕ ПЕРЕМЕННЫХ

Каждая переменная в программе до ее первого использования в выражении должна быть описана. Такое описание определяет имя переменной, ее характеристики (атрибуты) и дает транслятору необходимую информацию для резервирования объема памяти при размещении значений переменной.

В языке ПЛ/М допускается два типа переменных: BYTE и ADDRESS. Переменная типа BYTE занимает в памяти 8 бит и является однобайтным элементом; переменная типа ADDRESS занимает 16 бит и является двухбайтным элементом.

Тип и имя каждой переменной должны быть указаны в операторе описания, имеющем следующую форму:

DECLARE идентификатор тип;

Пример:

DECLARE X BYTE;
DECLARE Z ADDRESS;

Здесь X и Z — имена переменных, а DECLARE, BYTE и ADDRESS — ключевые слова.

При описании нескольких переменных одного и того же типа их имена можно перечислять в списке, заключенном в скобки. Вслед за скобками указывается тип переменных. Примеры описаний:

DECLARE (A,B,C,Q) BYTE;
DECLARE (X,Y,Z,M) ADDRESS;

В одном операторе DECLARE можно указать переменные разных типов, отделив их описания запятой, и апример:

DECLARE (W,T) BYTE, (SUM,BET) ADDRESS;

Здесь транслятор зарезервирует по одному байту памяти для каждой из переменных W и T и по два байта для переменных SUM и BET.

При описании переменной можно указать либо абсолютный, либо относительный адрес ее размещения в памяти. Для этого в операторе DECLARE используется атрибут AT. Оператор описания переменной в этом случае имеет следующий формат:

DECLARE имя переменной тип AT $\left\{ \begin{array}{l} \text{число} \\ \text{(выражение)} \\ \text{идентификатор} \end{array} \right\} ;$

Адрес размещения переменной указывается вслед за атрибутом AT в виде числа или выражения (абсолютный адрес) либо определяется адресом размещения ранее описанной переменной (относительный адрес).
Пример:

DECLARE (X,Y) BYTE, Z ADDRESS AT X;

Здесь описаны две переменные X и Y типа BYTE и переменная Z типа ADDRESS. При таком описании переменная Z размещается в памяти, начиная с адреса размещения переменной X. Поскольку длина Z равна двум байтам, то эта переменная накладывается на поле, где размещаются переменные X и Y. В результате к отдельным байтам этого поля из двух байт можно обращаться по именам X и Y, а ко всему полю из двух байт — по имени Z.

Если в операторе DECLARE с атрибутом AT описывается несколько переменных, то по указанному адресу размещается первая из них, а остальные располагаются в последовательных полях со сдвигом в соответствии с их длиной. Пример:

DECLARE (I,J,K,M) BYTE, (X,Y) ADDRESS AT I;

Здесь переменная X накладывается на поле размещения переменных I и J, а переменная Y — на поле переменных K и M.

2.7. АРИФМЕТИЧЕСКИЕ ВЫРАЖЕНИЯ

Выражение — это правило для вычисления значения. Выражение записывается в виде последовательности имен переменных, констант и функций, соединенных знаками операций и круглыми скобками. В арифметическом выражении используются переменные типа BYTE или ADDRESS, знаки арифметических операций и круглые скобки.

Арифметическое выражение вычисляется в определенном порядке, определяемом скобками и старшинством операций. Вначале выполняются действия в скобках. Арифметические операции имеют следующий порядок старшинства:

- 1 — умножение (знак *), деление (знак операции «/»), вычисление остатка от деления (знак операции MOD);
- 2 — сложение (знак «+» и PLUS) и вычитание (знак «-» и MINUS).

Операции одинакового приоритета в выражении выполняются слева направо. Все указанные выше операции производят действия по правилам двоичной арифметики с битными или адресными операндами.

Если при сложении или вычитании оба операнда имеют тип BYTE, операция осуществляется в пределах 8 бит, а результат имеет тип BYTE. Если один из операндов имеет тип ADDRESS, а второй — тип BYTE, то ко второму операнду будет добавлено восемь нулевых старших битов, операция будет выполнена в 16-разрядной сетке, а результат операции имеет тип ADDRESS.

В выражении можно использовать одноместную операцию вычитания (-). В этом случае выражение -A эквивалентно выражению 0-A. Например, -1 эквивалентно 0-1, что соответствует величине типа BYTE, равной 255, или OFFH.

Операции PLUS и MINUS выполняются аналогично операциям «+» и «-», но они сопровождаются установкой текущего значения признака переноса в центральном процессоре (признак CARRY).

Операции «*» и «/» осуществляют двоичное умножение и деление целых чисел без знака над операндами типа ADDRESS и BYTE; результат всегда имеет тип ADDRESS. Результат операции деления всегда округляется до целого значения, а результат деления на нуль имеет неопределенное значение.

В случае арифметического переполнения при умножении результат имеет неопределенное значение.

Значение признака переноса при выполнении операций умножения и деления не определено.

Операция MOD выполняется аналогично операции «/», но ее результатом является остаток от деления.

В отличие от обычной алгебраической записи в языке ПЛ/М выражения записываются в строку, причем знак умножения опускать нельзя, как это допускается

в алгебре. Перенос выражения на следующую строку можно сделать в любом месте, не разрывая ключевых слов, идентификаторов и констант. Знак переноса в языке ПЛ/М не применяется. Между операндами, знаками операций и скобками можно включать любое число пробелов.

Примеры выражений:

```
A+B  
A-B*15/C  
(X PLUS Y)*(A MOD B)
```

2.8. ОПЕРАТОРЫ И КОММЕНТАРИИ

Программа на языке ПЛ/М состоит из последовательности операторов. *Оператор* — это предписание машине произвести определенные действия либо при выполнении программы, либо в процессе ее трансляции.

При исполнении программы можно вычислить выражение и присвоить его значение определенной переменной, ввести данные из заданного канала ввода, вывести результаты вычислений в заданный канал вывода и др., при трансляции — предусмотреть выделение области памяти для размещения указанных переменных и поместить в нее начальные значения этих переменных, закончить трансляцию и др.

Большинство операторов ПЛ/М начинается специфичным ключевым словом, позволяющим компилятору опознать тип оператора. Оператор всегда заканчивается точкой с запятой, отделяющей этот оператор от следующего. Операторы выполняются последовательно в том порядке, в котором они записаны в программе. Однако существуют специальные операторы, которые изменяют естественный порядок выполнения операторов либо безусловно, либо в зависимости от некоторых условий. Перейти к выполнению оператора можно при условии, если перед ним записать метку или несколько меток. Метка в простейшем случае записывается как идентификатор. Метка отделяется от оператора двоеточием. Если меток несколько, то они разделяются двоеточиями. Естественный порядок выполнения операторов могут нарушать также прерывания, возникающие вследствие ошибок в программе и в некоторых специальных случаях. Программист может запретить или, наоборот, разрешить обработку прерываний.

Ключевые слова, определяющие тип оператора, отделяются одним или несколькими пробелами от последующих частей оператора. Пробелы могут находиться также между другими составными частями оператора. Там, где разрешено разместить один пробел, можно вставлять произвольное число пробелов. В программе можно использовать пустой оператор, который состоит только из точки с запятой. Этот оператор не выполняет никаких действий и употребляется только для выделения некоторых точек программы.

Для пояснения назначения программы или ее отдельных частей среди операторов можно помещать комментарии. *Комментарий* — это пояснительный текст, который можно записать всюду, где разрешен пробел. При трансляции комментариев исключается из программы, и, следовательно, наличие комментария в исходном тексте программы не отражается на объеме загрузочного модуля, полученного после трансляции. Комментарии записывают так:

```
/*ТЕКСТ*/
```

Комбинации специальных знаков «/*» и «*/» обозначают начало и конец комментария. Между знаками «/» и «*», «*» «/» пробелы недопустимы. ТЕКСТ — это произвольная последовательность литер, в том числе русских букв, не содержащая комбинаций специальных знаков «/*» и «*/».

Обычно комментарии помещают между операторами и перед программой.

Поскольку комментарии печатаются на листинге, они повышают наглядность программы и качество программной документации.

Примеры комментариев:

```
/*ПРОГРАММА СОРТИРОВКИ*/  
/*ОПЕРАТОР ВЫДЕЛЕНИЯ ЭЛЕМЕНТА*/
```

2.9. ЛИНЕЙНЫЕ ПРОГРАММЫ

2.9.1. Оператор присваивания

Линейная программа — это программа, все операторы, которой выполняются строго последовательно в том порядке, как они записаны в тексте программы. Наиболее простым и часто используемым оператором

в программах является оператор присваивания. В простейшем случае он имеет следующую форму записи:

Имя переменной = выражение;

Оператор присваивания предписывает вычислить выражение, записанное в его правой части (после знака «=»), и присвоить вычисленное значение переменной, имя которой записано в левой части. Прежнее значение переменной теряется. Отметим, что в операторе присваивания знак «=» не означает «равно», как в алгебре. Здесь этот знак читается: «присвоить значение» и определяет действие присваивания переменной значения выражения.

Примеры:

$X = A + B - 17;$
 $Z = X + 5;$
 $I = I + Z;$

В последнем примере к «старому» значению переменной I прибавляется значение переменной Z, и значение полученной суммы замещает «старое» значение переменной I, которое безвозвратно теряется.

Для фактического вычисления выражения в правой части оператора присваивания нужно, чтобы все входящие в выражение переменные имели значение, в противном случае выражение имеет неопределенное значение. Переменные могут получать значения при выполнении предыдущих операторов присваивания или другими способами, которые будут рассмотрены в дальнейшем.

Пример. Требуется вычислить значение переменной по формуле

$z = ax^2 + b^2$ при условии
 $a = 1; b = 7;$
 $x = b - 3;$

Решение:

DECLARE (A,B,X,Z) BYTE;
A = 1;
B = 7;
X = B - 3;
Z = A * X * X + B * B;

Указанный при описании тип переменной из левой части оператора присваивания (BYTE или ADDRESS) влияет на объем памяти, необходимой для размещения результата. Если тип переменной BYTE, а тип выражения из правой части оператора присваивания ADDRESS, то старший байт результата теряется. Если

результат выражения однобайтный, а переменная из левой части оператора присваивания описана с типом ADDRESS, старший байт записываемого в память результата заполняется двоичными нулями.

Иногда необходимо присвоить одинаковое значение сразу нескольким переменным. Это можно выполнить с помощью одного оператора присваивания, если в левой его части перечислить через запятые имена всех переменных.

Пример. Присвоить переменным V, T и W нулевое значение.

Решение:

$V, T, W = 0;$

В выражениях можно использовать оператор присваивания специального формата. Такой «вложенный» оператор присваивания имеет формат:

(переменная := выражение)

Он применяется как элемент выражения.

Выражение, записанное справа от знака вложенного присваивания «:=», вычисляется, и результат присваивается переменной, имя которой записано слева от знака «:=».

Пример. Следующая группа операторов

$B = C + D;$
 $E = F * G;$
 $Z = A + B - E;$

может быть записана более компактно с использованием «вложенного» оператора присваивания

$Z = A + (B := C + D) - (E := F * G);$

Эти две записи выполняют эквивалентные действия в программе.

2.9.2. Назначение начальных значений переменным

Описание переменной в операторе DECLARE обеспечивает выделение для этой переменной участка памяти (один байт для переменной типа BYTE и два соседних байта для переменной типа ADDRESS). Однако значение переменной остается неопределенным до тех пор, пока ей не будет присвоено значение.

Начальное значение переменной присваивается оператором присваивания. Начальные значения могут быть

присвоены также посредством включения в оператор описания атрибута (признака) INITIAL. Этот атрибут имеет следующую форму записи:

INITIAL (список констант)

где «список констант» — последовательность констант, разделенных запятыми.

Атрибут INITIAL в операторе DECLARE должен следовать сразу же за атрибутом типа (BYTE или ADDRESS), например:

```
DECLARE (X,Y,Z) BYTE INITIAL (1,2,3);
DECLARE (A,B) ADDRESS INITIAL (0CF9H, 'PI' );
DECLARE (V,T) BYTE INITIAL (5,6),
(L M) ADDRESS INITIAL (1356, ITOG' );
```

Объем памяти, необходимый для размещения в памяти констант из списка атрибута INITIAL, не должен превышать суммарной длины объявленных с этим атрибутом переменных.

Пример. Требуется вычислить значение переменной T по формуле $T = \frac{W^2 + V}{2} - W \cdot V$ при условии, что $W=3$, а $V=11$.

Решение:

```
DECLARE T ADDRESS,
(W,V) BYTE INITIAL (3,11);
T=(W*W+V*V)/2 - W*V;
```

Действие атрибута INITIAL состоит в том, чтобы назначить переменной начальное значение. Переменной выделяется участок памяти так же, как и при отсутствии атрибута INITIAL в операторе DECLARE. Затем в момент загрузки программы в память указанной переменной присваивается начальное значение. Это значение сохраняется в процессе выполнения программы лишь до первого присваивания переменной нового значения.

При повторном выполнении программы атрибут INITIAL уже не действует, и переменные, описанные с этим атрибутом, будут иметь значения, полученные ими последний раз при предшествующем выполнении программы. Следовательно, программист должен использовать атрибут INITIAL с осторожностью, поскольку

повторный запуск программы не восстанавливает начальных значений переменных, заданных в атрибуте INITIAL. Их можно восстановить повторной загрузкой программы в память. По этой причине не рекомендуется использовать атрибут INITIAL в системах с постоянной памятью.

2.9.3. Назначение имен константам

Числовые и строчные константы в программе не описываются в операторе DECLARE. Однако в языке ПЛ/М имеется возможность связать с константой некоторое имя (идентификатор) и использовать в дальнейшем это имя вместо константы. Такое назначение имен константам выполняется оператором DECLARE, имеющим следующий формат

DECLARE идентификатор DATA (константа);

Пример:

DECLARE X DATA (17), Y DATA ('T');

Здесь константа 17 получает имя X, которое в дальнейшем можно использовать везде в программе, где нужно записать константу 17. Действие атрибута DATA во многом схоже с INITIAL, но имеются и существенные различия. Во-первых, если в операторе DECLARE с атрибутом DATA отсутствует атрибут типа, то подразумевается BYTE. Во-вторых, идентификатор константы не может изменить своего значения во время выполнения программы. По этой причине имя константы не может быть записано в левой части оператора присваивания, вследствие чего константа сохраняет начальное значение независимо от числа повторений программы.

2.9.4. Понятие о встроенных функциях

При составлении программ часто возникает необходимость реализовать в программе некоторые типичные действия: ввести данные из внешних источников; вывести результаты вычислений; преобразовать данные одного типа в данные другого типа и т. д. Поскольку такие операции применяются в большинстве программ, алгоритмы выполнения их запрограммированы заранее и включены в состав транслятора, говорят, что они «встроены» в транслятор. Каждая встроенная программа имеет уникальное имя.

Для реализации типичных операций в программе на языке ПЛ/М нужно указать имя встроенной программы

и данные для ее работы. Чаще всего результатом работы встроенной программы является одно значение, поэтому такие «стандартные» программы называют встроенными функциями (по аналогии с функциями типа $\sin x$, $\cos x$, где также вычисляется только одно значение).

Для получения значения встроенной функции достаточно в программе записать указатель функции, т. е. ее имя и аргументы в следующем виде:

имя функции (список аргументов)

Аргументы в списке разделяются запятыми. В общем случае каждый аргумент может представлять собой выражение. У некоторых встроенных функций список аргументов отсутствует.

Данные в процессе выполнения программы вводятся с помощью встроенной функции INPUT, имеющей следующую форму записи:

INPUT (число)

В качестве аргумента этой функции записывается число в диапазоне 0—255, являющееся номером входного канала (порта). Значением функции INPUT является 8-битная величина, полученная в результате ввода данных через порт с указанным номером. Указатель функции ввода данных всегда используется в виде операнда выражения.

Пример:

```
DECLARE X BYTE;
X = INPUT (22H);
. . . . .
```

Здесь переменная X получит значение, введенное через порт с номером 34.

Вывести результаты вычислений можно с помощью указателя функции вывода OUTPUT. Его формат:

OUTPUT (число)

Здесь аргумент является номером канала (порта) вывода и должен иметь значение в пределах 0—255.

Указатель функции вывода можно использовать только в левой части оператора присваивания (но не «вложенного» оператора присваивания). Результатом выполнения функции OUTPUT является передача 8-битной величины на заданный выходной канал — порт.

Пример:

```
DECLARE Z BYTE INITIAL (5);
OUTPUT (20H) = Z * Z + 1;
. . . . .
```

Здесь в канал с номером 32 передается число 26.

Имеется группа встроенных функций для преобразования типа данных. Встроенные функции LOW и HIGH предназначены для преобразования величин типа ADDRESS в величины типа BYTE. Результатом выполнения каждой из этих функций является величина типа BYTE, а аргументом — величина типа ADDRESS. Указатели этих функций всегда служат операндами выражений и записываются в следующей форме:

LOW (выражение)
HIGH (выражение)

Функция LOW позволяет получить в результате ее выполнения младший байт аргумента, а функция HIGH — старший байт аргумента. С помощью встроенной функции DOUBLE производится преобразование аргумента типа BYTE в результат типа ADDRESS. В процессе такого преобразования слева от аргумента типа BYTE появляется байт, все биты которого содержат нули.

Пример:

```
DECLARE (X,Y) BYTE, (T,Z) ADDRESS;
Z=0AB75H;
X=LOW(Z);
Y=HIGH(Z);
T=DOUBLE(X);
. . . . .
```

В результате выполнения этих операторов переменная X будет содержать значение, равное 75H, переменная Y — 0ABH, а переменная T — 0075H.

Кроме встроенных функций, обращение к которым выполняется с помощью указателя функции, в языке ПЛ/М определены встроенные процедуры.

В языке ПЛ/М наиболее часто используются следующие встроенные функции и процедуры:

CARRY	LOW	SCR
DEC	MEMORY	SHL
DOUBLE	OUTPUT	SHR
HIGH	PARITY	SIGN
INPUT	ROL	STACKPTR
LAST	ROR	TIME
LENGTH	SCL	ZERO

2.9.5. Структура простой программы

Программа на языке ПЛ/М представляет собой последовательность операторов. Каждый оператор заканчивается точкой с запятой.

Текст программы записывается в свободной форме без привязки к каким-либо определенным колонкам бланка программирования. Между отдельными элементами программы (ключевыми словами, идентификаторами, константами, операторами) могут быть вставлены пробелы. Они не являются обязательными, но повышают наглядность программы и качество программного документа.

Внутри ключевых слов, идентификаторов и констант пробелы недопустимы.

Все используемые в программе переменные должны быть описаны в операторах DECLARE, прежде чем они будут использованы в операторах. В конце программы записывается специальное ключевое слово EOF, которое не вызывает никаких действий, а служит лишь для обозначения конца текста программы.

Пример линейной программы:

```
/* программы вычисления площади прямоугольника */  
DECLARE (A B) BYTE.S ADDRESS;  
/* ввод значения A через порт 1 */  
A=INPUT (1);  
/* ввод значения B через порт 2 */  
B=INPUT (2);  
S=A*B; /* вычисление площади S */  
/* вывод старшего байта результата через порт 5 */  
OUTPUT (S)=HIGH(S);  
/* вывод младшего байта результата через порт 5 */  
OUTPUT (S)=LOW(S);  
EOF
```

2.10. РАЗВЕТВЛЯЮЩИЕСЯ ПРОГРАММЫ

2.10.1. Метки и оператор перехода

Разветвляющаяся программа — это программа, в которой операторы могут выполняться не в том порядке, в каком они записаны в тексте исходной программы. Такая программа состоит из линейных участков и точек ветвления. Операторы, входящие в линейный участок, выполняются последовательно, как в ли-

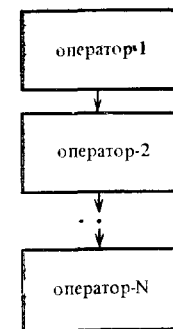
нейной программе. В точках ветвления происходит выбор того или иного линейного участка (той или иной ветви программы). На рис. 2.1 показаны возможные схемы линейной и разветвляющейся программ.

Разветвляющаяся программа на рис. 2.1, б состоит из трех линейных участков, помеченных метками M1, M2 и M3. В точке ветвления происходит выбор ветви с меткой M2 или с меткой M3. В ходе трансляции транслятор заменяет каждый оператор группой машинных команд. При загрузке программы эти машинные команды размещаются в памяти в том порядке, в каком они были записаны в исходной программе. При этом каждая метка получает значение, равное адресу первого байта группы команд, соответствующей помеченному оператору.

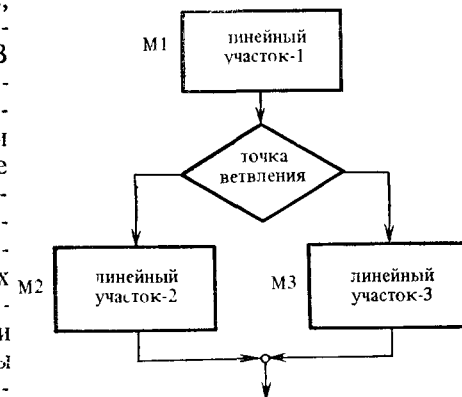
Метки используются для управления последовательностью выполнения операторов в разветвляющейся программе с помощью оператора перехода, имеющего формат:

GO TO метка;

Ключевое слово GO TO (разрешается записывать без пробела GOTO) означает «перейти на». После выполнения этого оператора следующим будет выполнять-



а



б

Рис. 2.1. Линейные (а) и разветвляющиеся (б) программы

ся оператор, метка которого указана в операторе GO TO.

Пример:

```

M1: .....
    X=A+B;
    GO TO M3;
M2 Y=A-B,
M3 Z=Z+X*X;

```

Здесь после выполнения оператора с меткой M1 осуществляется переход на выполнение оператора с меткой M3, а оператор с меткой M2 пропускается

2.10.2. Операции отношения и логические операции

Выбор той или иной ветви в точке ветвления производится в результате проверки некоторого условия, например, проверяется, больше ли значение одной переменной значения другой переменной. В зависимости от выполнения этого условия производится выбор ветви.

Проверка условий в разветвляющихся программах производится с помощью операций отношения и логических операций.

Операции отношения применяют для сравнения двух величин. Знаки операций отношения и их смысл показаны ниже.

< меньше
 > больше
 <= меньше или равно
 >= больше или равно
 <> не равно
 = равно

Каждый из двух операндов этих операций может быть типа BYTE или ADDRESS. При необходимости короткий (типа BYTE) операнд дополняется слева нулевым байтом так же, как в арифметических операциях.

При сравнении всегда предполагается, что операнды представляют собой целые двоичные числа без знака. Если заданное отношение выполняется, то результатом сравнения будет величина OFFH, в противном случае результатом будет величина 00H. Следовательно, результат сравнения всегда имеет тип BYTE, все восемь бит которого устанавливаются в 1 при выполнении условия или же в 0, если условие не выполнено.

Примеры:

Выражение	Результат
2 > 5	00000000B
7 <= 9	11111111B
A = X	OFFH, если значение переменной A равно значению переменной X, в противном случае — 00H
1 < 0	00H

Для проверки более сложных условий используются логические операции. В ПЛ/М определены четыре логические (булевы) операции, знаки операций которых и их названия приведены ниже.

NOT отрицание
 AND логическое умножение (операция И)
 OR логическое сложение (операция ИЛИ)
 XOR отрицание равнозначности (операция ИСКЛЮЧАЮЩЕЕ ИЛИ)

Операция NOT одноместная, она всегда записывается перед операндом. Операции AND, OR и XOR двухместные; они выполняют поразрядные логические операции над соответствующими друг другу битами операндов независимо от соседних битов.

Результаты логических операций над соответствующими битами операндов приведены в табл. 2.3.

Таблица 2.3

X	Y	NOT X	NOT Y	X AND Y	X OR Y	X XOR Y
0	0	1	1	0	0	0
1	0	0	1	0	1	1
0	1	1	0	0	1	1
1	1	0	0	1	1	0

В двухместных операциях AND, OR и XOR, если оба операнда имеют тип BYTE, операция выполняется над 8 битами и дает результат типа BYTE. Если хотя бы один из операндов имеет тип ADDRESS, операция выполняется над 16 битами, а результат имеет тип ADDRESS. В этом случае, если один операнд имеет тип BYTE, он предварительно расширяется до 16 бит добавлением 8 нулей в старшие разряды.

Примеры:

Операция	Результат
NOT 10101110 B	01010001 B
11110011B AND 10101010 B	10100010 B
11110011B OR 10100101 B	11110111 B
10111010B XOR 11010100 B	01101110 B

В общем случае выражение может содержать любые операции: арифметические, сравнения, логические. Все операции упорядочены отношением старшинства, определяющим порядок выполнения операций, если нет скобок. При наличии скобок сначала выполняются действия в скобках, при этом также учитывается старшинство операций. Ниже приведены операции, расположенные в порядке убывания приоритета. Операции, расположенные в одной строке, имеют одинаковый приоритет и выполняются слева направо.

* /	MOD
+ -	PLUS MINUS
< > =	< > =
	NOT
	AND
OR	XOR

Изменить обычный порядок выполнения операций можно с помощью скобок.

Примеры:

Выражение	Последовательность выполнения операций
A/B>C AND B*C<=(C+D)	$((A/B)>C) \text{ AND } ((B*C) \leq (C+D))$ I II III IV
A>B AND A>6 OR D*C>=5	$((A>B) \text{ AND } (A>6)) \text{ OR } ((D*C) \geq 5)$ I II III IV
NOT A*B>3 AND B<1 OR C>5	$((\text{NOT}((A*B)>3)) \text{ AND } (B<1)) \text{ OR } (C>5)$ I II III IV V

Если для трех приведенных выше выражений переменные имеют следующие значения:

$$A=6; B=3; D=5; C=1;$$

то результатом первого и второго выражений будет величина 0FFH («истина»), а результатом последнего — величина 00H («ложь»).

2.10.3. Условный оператор

Операции сравнения обычно используются в *условных* операторах, которые специально предназначены для записи разветвляющихся программ. Формат условного оператора:

IF выражение THEN оператор-1; ELSE оператор-2;

Здесь ключевые слова IF, THEN и ELSE означают соответственно «если», «то» и «иначе».

«Выражение» может содержать арифметические и логические операции.

За ключевыми словами THEN и ELSE могут быть записаны любые выполняемые операторы.

Условный оператор выполняется следующим образом. Вычисляется выражение, записанное за ключевым словом IF. Если правый бит результата равен 1, то выражение является истинным и выполняется «оператор-1», а «оператор-2» не выполняется.

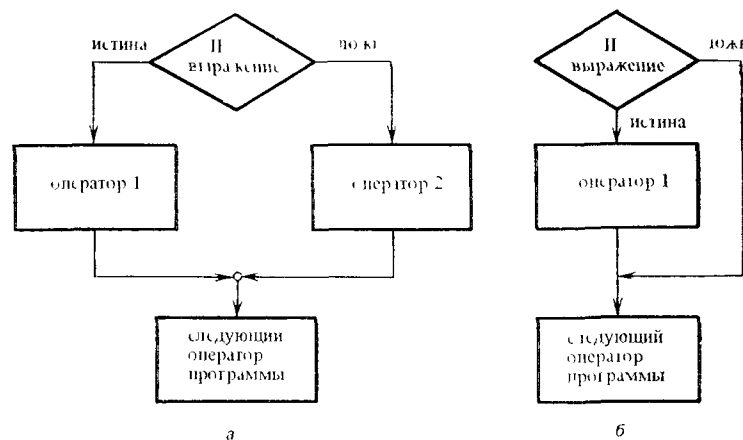


Рис. 2.2 Условный оператор а — типа IF...THEN...ELSE; б — типа IF...THEN

Если правый бит результата равен 0, то выражение считается ложным, «оператор-1» пропускается и выполняется «оператор-2». После выполнения любого из операторов («оператор-1» или «оператор-2») следующим выполняется оператор, записанный в программе за условным оператором (рис. 2.2, а). Заметим, что следующий выполняемый оператор может быть указан в самом «операторе-1» или «операторе-2» в виде оператора перехода GO TO.

Рассмотрим примеры выполнения условного оператора.

Пример 1. Требуется вычислить значение функции

$$z = \begin{cases} 0, & \text{если } x^2 + y^2 > r^2; \\ 1, & \text{если } x^2 + y^2 \leq r^2. \end{cases}$$

Решение:

```
IF X*X+Y*Y>R*R THEN Z=0; ELSE Z=1;
```

Пример 2. Если $X > Y$, то перейти к выполнению оператора с меткой M1, в противном случае — к оператору с меткой M2.

Решение:

```
IF X>Y THEN GO TO M1; ELSE GO TO M2;
```

В условном операторе конструкция «ELSE оператор-2» может отсутствовать. Такой сокращенный условный оператор (рис. 2.2, б) имеет вид:

```
IF выражение THEN оператор-1;
```

Здесь «оператор-1» будет выполняться только в том случае, когда результат выражения содержит 1 в правом бите. В противном случае условный оператор не выполняет никаких действий, и управление передается следующему оператору программы.

Пример 3. Если переменная X содержит значение «AA» или состоит из пробелов, то нужно перейти к выполнению оператора с меткой MM, в противном случае — выполнить следующий оператор.

Решение:

```
IF X='AA' OR X=' ' THEN GO TO MM;
```

2.10.4. Составной оператор

Иногда в одной ветви разветвляющейся программы нужно выполнить несколько операторов. Однако в условном операторе после ключевых слов THEN и ELSE

можно записать только один оператор. В этих случаях используют *составной* оператор, который имеет форму:

```
DO; оператор; оператор; ...; оператор; END;
```

Ключевое слово DO означает «выполнить» и указывает начало, а ключевое слово END — конец составного оператора.

Операторы «DO;» и «END;» не выполняют никаких действий. Они образуют своеобразные скобки, выделяющие группу операторов — тело составного оператора.

Группа операторов, записанная между DO и END, синтаксически рассматривается как один оператор, поэтому составные операторы можно записывать после слов THEN и ELSE.

Пример. Если $X \leq 5$, то переменной Y нужно присвоить значение 0, а переменной Z — значение 1. В противном случае переменной Y присвоить значение 1, а переменной Z — значение 2.

Решение:

```
IF X<=5 THEN DO, Y=0; Z=1; END;
ELSE DO; Y=1; Z=2; END;
```

В теле составного оператора можно записывать любые операторы, в том числе и составные. В этом случае внутренние составные операторы называют *вложенными* по отношению к охватывающим их внешним составным операторам.

Внутри составного оператора могут быть записаны и условные операторы. Здесь следует отметить, что существует только одно ограничение на использование операторов, входящих в условный оператор: «оператор-1» (оператор, записанный за ключевым словом THEN) не может сам быть условным оператором. В частности, конструкция

```
IF выражение-1 THEN IF выражение-2 THEN
оператор-3; ELSE оператор-2;
```

двусмысленна и неверна, поскольку неизвестно, к какому IF относится ELSE. Эта конструкция должна быть заменена на одну из двух следующих конструкций в зависимости от того, какой результат нужен.

1. IF выражение-1 THEN
DO;
IF выражение-2 THEN оператор-3;
END;
ELSE оператор-2;
2. IF выражение-1 THEN
DO;
IF выражение-2 THEN оператор-3;
ELSE оператор-2;
END;

Пример. Записать операторы, реализующие вычисления по следующим соотношениям:

$$\left. \begin{array}{l} Z=1; P=0; \text{ при } 1 < Y \leq 5 \\ Z=2; P=1; \text{ при } 5 < Y \leq 10 \\ Z=3; P=2; \text{ при остальных} \\ \text{значениях } Y \end{array} \right\} \text{ при } X > 10$$

$Z=0; P=5; \text{ при любых значениях } Y \text{ при } X \leq 10$

Решение:

```

IF X > 10 THEN
  DO;
    Z=3;
    P=2;
    IF Y > 1 AND Y <= 5 THEN
      DO;
        Z=1;
        P=0;
      END;
    IF Y > 5 AND Y <= 10 THEN
      DO;
        Z=2;
        P=1;
      END;
    END;
  ELSE
    DO;
      Z=0;
      P=5;
    END;
  END;

```

2.10.5. Оператор DO CASE

При программировании сложных разветвлений не всегда целесообразно использовать условный оператор, особенно когда нужно выбрать одну из множества вет-

вей. В тех случаях, когда условие выбора ветви однозначно определяется значением некоторой переменной, удобно применить оператор DO CASE. Его формат:

```

DO CASE выражение;
  оператор-1;
  оператор-2;
  .....
  оператор-n;
END;

```

При выполнении этого оператора сначала вычисляется значение выражения, записанного за ключевым словом CASE. Результат должен быть в пределах от 0 до $n-1$. Этот результат используется для выбора одного из n операторов группы. Выбранный оператор выполняется, после чего управление передается за пределы оператора DO CASE. Выполнение первого оператора группы (оператор-1) соответствует значению выражения, равного нулю. Второй оператор (оператор-2) будет осуществляться, если значение выражения равно 1 и т. д. При каждом выполнении DO CASE будет выбран только один оператор из группы. Если текущее значение выражения превышает количество операторов в группе, действие оператора DO CASE равносильно пустому оператору. Рассмотрим два примера.

Пример 1:

```

DO CASE N;
  X=X+5;
  ;
  Y=Z+X;
  ;
  V=W+7;
END;

```

Здесь значение переменной N должно быть в пределах 0—4. Если в момент выполнения оператора DO CASE значение переменной N равно 1 или 3, то выполняются пустые операторы, записанные в группе операторов на 2- и 4-м месте. При значениях переменной N, равных 0, 2, 4, выполняются операторы из группы, изменяющие значения переменных X (при $N=0$), Y (при $N=2$) и V (при $N=4$). В составе группы DO CASE могут использоваться любые операторы, в том числе условные и составные, что позволяет реализовать сколь угодно сложные разветвления.

Пример 2. Пусть через порт с номером 5 вводится значение переменной X. Переменная X связана с управляющим воздействием Y следующими соотношениями:

$$Y = \begin{cases} X^2, & \text{при } X < 10 \\ 90 + X, & \text{при } 10 \leq X \leq 30 \\ 150 - X, & \text{при } 30 < X < 150 \\ 0, & \text{при } X \geq 150 \end{cases}$$

Вычисленное значение Y должно выводиться через порт с номером 35.

Решение:

```
DECLARE (X,Y) BITF;
X=INPUT(5);
IF X < 10 THEN Y=X*X;
IF X >= 10 AND X <= 30 THEN Y=90+X;
IF X > 30 AND X < 150 THEN Y=150-X;
IF X > 150 THEN Y=0;
OUTPUT(35)=Y;
EOF
```

Можно предложить другой вариант программы с использованием оператора DO CASE:

```
DECLARE (X,Y) BYTE.K BYTE;
X=INPUT(5);
IF X < 10 THEN DO; K=0; GO TO M1; END;
IF X >= 10 AND X <= 30 THEN DO; K=1; GO TO M1; END;
IF X > 30 AND X < 150 THEN DO; K=2; GO TO M1; END;
IF X >= 150 THEN DO; Y=0; GO TO M2; END;
M1: DO CASE K;
    Y=X*X;
    Y=90+X;
    Y=150-X;
END;
M2: OUTPUT(35)=Y;
EOF
```

2.11. ЦИКЛИЧЕСКИЕ ПРОГРАММЫ

2.11.1. Оператор DO WHILE

Циклической называют программу, содержащую один или несколько многократно повторяющихся участков. Повторяющийся участок программы называют циклом. Цикл содержит операторы, составляющие тело цикла, и операторы, управляющие повторениями тела цикла, — заголовок цикла (рис. 2.3, а).

Цикл всегда можно построить, используя условный оператор IF и оператор перехода GO TO. На рис. 2.3, б

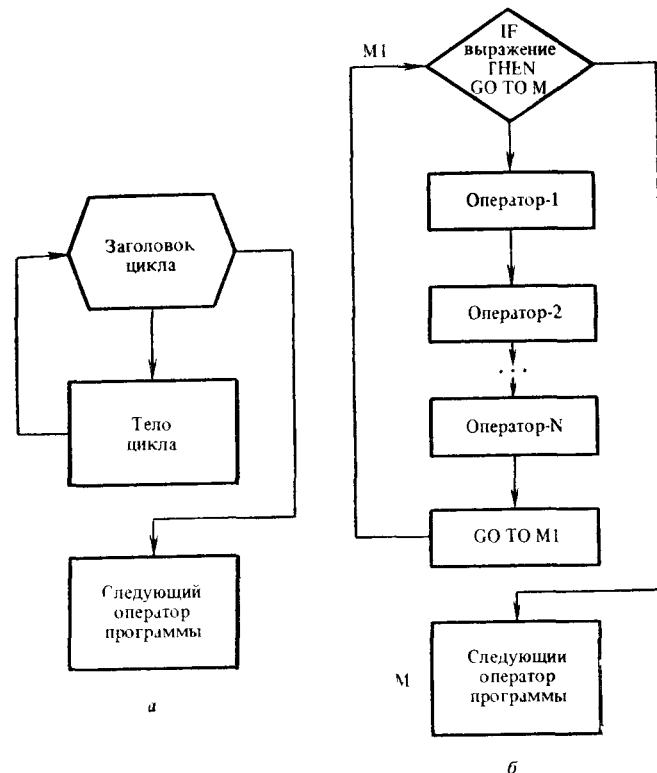


Рис. 2.3. Схема цикла: а — общая; б — с операторами IF и GO TO

показан цикл, где оператор IF выполняет роль заголовка цикла, а операторы с номерами 1, 2, ..., n составляют тело цикла. Поскольку циклы программируются очень часто, в языке ПЛ/М имеются специальные операторы цикла, которые упрощают программирование циклических процессов и делают циклическую программу более наглядной. К тому же в программах не рекомендуется использовать оператор перехода GO TO во избежание трудно обнаруживаемых ошибок.

В языке ПЛ/М имеются две формы оператора цикла. Простейшая из них имеет следующий формат:

```
DO WHILE выражение;
  оператор-1;
  оператор-2;
  .....
  оператор-n;
END;
```

Здесь оператор

DO WHILE выражение;

является заголовком цикла. Ключевое слово WHILE можно перевести как «пока» или «до тех пор, пока». Группа операторов, записанных между заголовком цикла и END, составляет тело цикла.

Перед каждым выполнением тела цикла вычисляется значение выражения в заголовке цикла. Если правый (младший) бит результата равен 1 (т. е. выражение «истинно»), то выполняются операторы из тела цикла и затем снова производится вычисление выражения в заголовке цикла. Если правый бит результата равен 0 (т. е. выражение «ложно»), то операторы тела цикла не выполняются, а происходит переход к оператору, записанному за END, т. е. происходит выход из цикла. Иными словами, выполнение тела цикла повторяется до тех пор, пока выражение в заголовке цикла «истинно».

Отметим, что в случае, когда перед первым выполнением цикла выражение в заголовке цикла «ложно», тело цикла не выполняется.

Операторы, составляющие тело цикла, могут быть помечены. Однако запрещается передавать управление внутрь тела цикла, минуя заголовок. Выход из цикла возможен при помощи оператора перехода GO TO.

Пример:

```
S=0;
Y=50;
DO WHILE Y >= 10;
  Y=Y-5;
  S=S+Y;
END;
```

Здесь тело цикла, состоящее из двух операторов присваивания, будет выполняться 9 раз. При выходе из цикла значение переменной Y будет равно 5, а значение суммы S — 225.

Обычно оператор цикла DO WHILE используют в тех случаях, когда число повторений цикла заранее неизвестно и момент окончания цикла определяется выполнением некоторого логического условия.

2.11.2. Итеративный цикл

При повторениях цикла часто требуется изменять по определенному правилу значение некоторой переменной, которая влияет на результат выполнения операторов в теле цикла и может использоваться для проверки условия прекращения повторений цикла. Такую переменную называют управляющей, или параметром цикла.

Итеративный цикл, или цикл с параметром, обеспечивает повторение тела цикла определенное, заранее известное число раз. Простейшая форма записи итеративного цикла:

```
DO переменная=выражение-1 TO выражение-2;
  оператор-1;
  оператор-2;
  .....
  оператор-n;
END;
```

Начало этой конструкции от ключевого слова DO до первой точки с запятой представляет собой заголовок цикла, который изменяет по определенному правилу значение переменной, имя которой записано за словом DO, и управляет числом повторений тела цикла. Тело цикла составляют операторы, записанные между заголовком цикла и ключевым словом END.

«Выражение-1» и «выражение-2» — обычные выражения. Перед первым выполнением цикла эти выражения вычисляются. Затем параметру цикла (переменной, записанной за DO) присваивается значение «выражения-1». После этого проверяется значение параметра цикла. Если оно меньше или равно значению «выражения-2», то выполняются операторы тела цикла. В конце выполнения тела цикла значение параметра цикла увеличивается на 1, и вновь производится проверка значения параметра цикла. Тело цикла будет повторно выполняться до тех пор, пока значение параметра цикла меньше или равно значению «выражения-2». Если при очередной проверке окажется, что значение параметра цикла больше значения «выражения-2», то тело цикла выполняться не будет и управление передается оператору, записанному в программе за ключевым словом END (выход из цикла).

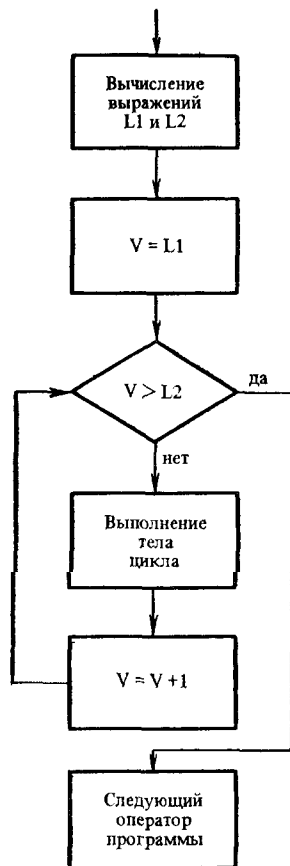


Рис. 2.4. Схема итеративного цикла

На рис. 2,4 схематически поясняется алгоритм выполнения итеративного цикла, записанного в форме:

```

DO V=L1 TO L2;
  ТЕЛО ЦИКЛА;
END;
  
```

Здесь V — параметр цикла;
L1 — выражение-1;
L2 — выражение-2.

Пример. Требуется получить сумму четных чисел натурального ряда от 1 до 20.

Фрагмент программы может быть записан в следующем виде:

```

S=0;
DO I=1 TO 20;
  K=I MOD 2;
  IF K=0 THEN S=S+I;
END;
  
```

Здесь операторы тела цикла будут выполняться ровно 20 раз. При этом каждый раз будет вычисляться значение переменной K как остаток от деления значения переменной I на 2. Если остаток равен нулю (K=0), то текущее значение переменной I накапливается в сумме S, в противном случае суммирование не производится. При выходе из цикла переменная I (параметр цикла) имеет значение 21, а переменная S содержит сумму четных чисел от 2 до 20, т. е. S=110.

В более общей форме записи итеративного цикла значение параметра можно изменять на любую заданную величину, а не только на единицу, как было показано выше. Общая форма записи итеративного цикла имеет вид:

```

DO переменная=выражение-1 TO выражение-2 BY
  выражение-3;
  оператор-1;
  оператор-2;
  .....
  оператор-n;
END;
  
```

Здесь перед выполнением цикла вычисляются «выражение-1», «выражение-2» и «выражение-3». При каждом повторении цикла параметр цикла увеличивается на величину, равную значению «выражение-3». Таким образом, значение «выражение-1» определяет начальное значение параметра цикла, значение «выражение-2» — предельное значение параметра цикла, а значение «выражение-3» определяет величину приращения (шаг изменения) параметра цикла.

Пример. Записанный в предыдущем примере фрагмент программы можно представить в следующем виде:

```

S=0;
DO I=0 TO 20 BY 2;
  S=S+I;
END;
  
```

Параметр цикла можно использовать в любых выра-

жениях в теле цикла в качестве операнда, но его нельзя записывать в левой части оператора присваивания.

Типы значений выражений в заголовке цикла (выражение-1, выражение-2, выражение-3) могут отличаться от типа значения параметра цикла. В этом случае производится приведение типов значений этих выражений к типу параметра цикла.

2.11.3. Вложенные циклы

В совокупности операторов, составляющих тело цикла, любой из операторов может быть, в свою очередь, оператором цикла. Это позволяет строить циклы, содержащие внутренние циклы. Такие внутренние циклы называют *вложенными*.

Пример:

```
S=0;
DO I=1 TO 10;
  K=I+3;
  DO J=1 TO 15;
    S=S+K*J;
  END;
END;
```

Здесь при каждом фиксированном значении управляющей переменной внешнего цикла I внутренний цикл с управляющей переменной J будет выполняться 15 раз, все тело внутреннего цикла — 150 раз.

2.11.4. Массивы

Массив — это упорядоченная совокупность однотипных элементов. С каждым элементом массива связано целое число — индекс, который определяет положение элемента в массиве. Практически понятие индекса элемента массива тождественно понятию индекса элемента вектора.

Использованию элементов массива в программе должно предшествовать описание массива. В простейшем случае описание массива выполняется оператором DECLARE следующего вида:

DECLARE имя-массива (размер) тип;

где «имя-массива» — идентификатор;

«размер» — число элементов массива;

«тип» — тип элементов массива (BYTE или ADDRESS).

Например, оператор

DECLARE Z(50) BYTE;

описывает идентификатор Z как имя массива, содержащего 50 элементов типа BYTE.

В языке ПЛ/М могут быть определены только одномерные массивы.

В одном операторе DECLARE можно указать несколько массивов, в том числе разных типов. Например, в следующем операторе

DECLARE (A,B,C) (200) ADDRESS, (X,Y,Z) (100) BYTE;

описаны шесть массивов, из которых три массива с именами A, B и C содержат каждый по 200 элементов типа ADDRESS, а массивы с именами X, Y и Z содержат по 100 элементов типа BYTE.

Для использования в программе отдельного элемента массива нужно указать имя массива и порядковый номер элемента в массиве.

Например, оператор

DECLARE V(30) BYTE;

фактически описывает 30 однотипных (тип BYTE) переменных V(0), V(1), ..., V(29).

Чтобы получить сумму первых трех элементов этого массива, а результат присвоить последнему элементу, нужно записать

$V(29) = V(0) + V(1) + V(2);$

Числа в скобках, указывающие конкретный элемент массива, называют *индексами*, а элементы массива — *переменными с индексами*, или *индексированными переменными*, чтобы отличить их от простых или неиндексированных переменных.

Переменные с индексами допустимы во всех конструкциях языка, в которых можно использовать простые переменные, с единственным исключением — переменная с индексом не может быть параметром итеративного цикла. Индекс элемента массива не обязательно должен быть числовой константой. В общем случае в качестве индекса можно использовать любое допустимое в языке выражение. Например, следующий фрагмент программы выполняет суммирование отдельно четных и нечетных элементов массива X.

```

DECLARE (S1,S2,I) BYTE;
DECLARE X(10) BYTE;
...
S1,S2=0;
DO I=0 TO 9 BY 2;
  S1=S1+X(I);
  S2=S2+X(I+1);
END;
EOF

```

Начальные значения элементам массива могут быть присвоены при описании массива с использованием либо атрибута INITIAL, либо атрибута DATA. В первом случае оператор описания массива имеет вид:

```

DECLARE имя-массива (размерность) тип INITIAL (список
                                     констант);

```

Константы в списке разделяются запятыми, например:

```

DECLARE U(10) BYTE INITIAL (1,2,3,4,5,6,7,8,9,10);

```

В общем случае количество констант в списке может не соответствовать количеству элементов массива. Здесь имеется только одно ограничение — длина списка начальных значений не должна превышать объявленную длину массива. Константы из списка начальных значений размещаются в памяти в последовательных полях, начиная с первого байта поля памяти, выделенного для массива. Если длина поля памяти для массива больше, чем необходимо для размещения констант, то последним элементам массива не будет присвоено начальных значений, например:

```

DECLARE Z(3) BYTE INITIAL (20,'A');

```

Здесь элементу Z(0) будет присвоено начальное значение 20, элемент Z(1) будет содержать в качестве начального значения литеру A, а элементу Z(2) не будет присвоено начальное значение.

При задании начальных значений с помощью атрибута DATA оператор описания массива имеет вид:

```

DECLARE имя-массива [тип] DATA (список констант);

```

В этом операторе отсутствует размер массива, он определяется количеством элементов из списка констант. Кроме того, атрибут типа может отсутствовать, тогда по умолчанию принимается тип BYTE, например:

```

DECLARE M DATA('МАССИВ СТРОК');

```

Здесь неявно объявлен массив M типа BYTE из 12 строчных констант, каждому элементу массива присвоено начальное значение — одна литера из строчной константы. Например, элемент M(4)

будет содержать код литеры И, а элемент M(6) — код литеры «пробел».

Особенности использования начальных значений элементов массива, присвоенных атрибутом INITIAL и DATA, точно такие же, как и для простых переменных (см. разд. 2.9.2).

Удобным средством при задании начальных значений элементам массива является описание массива с неявной длиной. Если при описании массива задаются начальные значения всем его элементам, то программист имеет возможность задать его длину неявно, указав вместо количества элементов знак * (звездочка). В этом случае длина массива будет определяться длиной списка констант, например:

```

DECLARE X(*) BYTE INITIAL(1,2,3,'ABC');
DECLARE Y(*) ADDRESS INITIAL(200,37,'TEXT');

```

Здесь описаны массивы X и Y. Первый состоит из 6 элементов, второй (массив Y) состоит из 4 элементов. Появление в программе оператора описания массива Y эквивалентно наличию в программе следующих операторов:

```

DECLARE Y(4) ADDRESS;
Y(0)=200;
Y(1)=37;
Y(2)='TE';
Y(3)='XT';

```

2.11.5. Встроенные функции для массивов

В языке ПЛ/М определены две встроенные функции LENGTH и LAST, использование которых может быть полезным при работе с массивами. Первая из них предназначена для определения длины аргумента, ее указатель имеет вид:

LENGTH (имя)

где LENGTH — ключевое слово (переводится как «длина»);
имя — идентификатор массива или простой переменной.

Для определения индекса последнего элемента массива используется встроенная функция LAST, указатель которой имеет вид

LAST (имя)

где LAST — ключевое слово (переводится как «последний»).

Указатели каждой из этих функций могут использоваться в качестве элемента любого выражения. Тип вычисленного значения обеих функций ADDRESS.

Для любого идентификатора V справедливо соотношение

$$\text{LENGTH}(V) = 1 + \text{LAST}(V),$$

если определена функция LAST.

Функция LENGTH существует для любых переменных независимо от того, как они описаны, а функция LAST не определена для переменных, длина которых по описанию равна 0.

Пример. В приведенной ниже программе функция LAST используется для того, чтобы присвоить всем элементам массива X с четными номерами значение 0, а всем остальным — значение 1.

```
DECLARE X(50) BYTE;
DECLARE I BYTE;
DO I=0 TO LAST(X) BY 2;
  X(I)=0;
  X(I+1)=1;
END;
EOF
```

2.11.6. Структуры

Для представления логически связанных данных (в общем случае разных типов) в языке ПЛ/М используются *структуры*. Примером такого данного, компоненты которого логически связаны, является данное ANKETA, показанное на рис. 2.5, а в виде дерева. На этом рисунке под каждой компонентой приведены в скобках их возможные значения.

Прежде чем использовать структуру в программе, ее необходимо описать. Описание задает для каждого элемента структуры имя и атрибуты.

В простейшем случае описание структуры имеет вид:

```
DECLARE имя-структуры STRUCTURE (имя-поля [размерность] тип,
имя-поля [размерность] тип, ...);
```

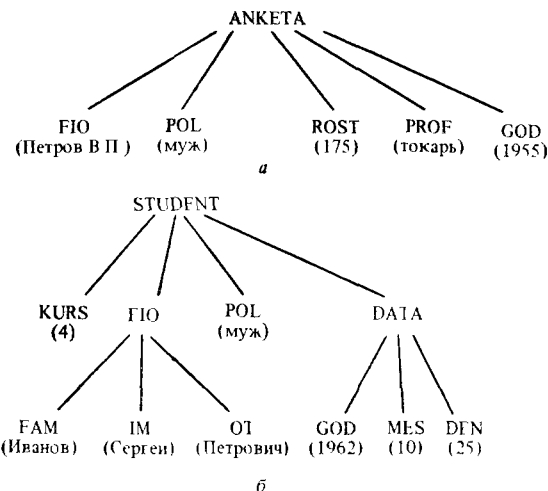


Рис. 2.5. Примеры структур: а — ANKETA, б — STUDENT

Пример. Для структуры рис. 2.5, а описание может быть таким:

```
DECLARE ANKETA STRUCTURE (FIO(20) BYTE, POL(3) BYTE,
ROST BYTE, PROF(6) BYTE, GOD ADDRESS);
```

Здесь объявлена одна структура, в состав которой входит пять полей: FIO — массив из 20 байт; POL — массив из 3 байт; ROST — простая переменная типа BYTE; PROF — массив из 6 байт и GOD — простая переменная типа ADDRESS.

В программе могут использоваться массивы структур. Тогда в операторе описания после имени структуры записывается атрибут размера. Например, описание

```
DECLARE SPISOK(10) STRUCTURE (NAME(60) BYTE,
AVTOR(20) BYTE, GODIZD ADDRESS;
```

определяет массив из 10 структур.

Компонентами структуры могут быть простые переменные, массивы и структуры низших уровней (подструктуры), причем в отличие от массивов в структуру могут входить элементы разных типов. Например, в состав структуры STUDENT, изображенной на рис. 2.5, б

в виде дерева, входят две подструктуры FIO и DATA. Описание такой структуры имеет вид:

```
DECLARE STUDENT  STRUCTURE (KURS BYTE,
FIO              STRUCTURE (FAM(20) BYTE, IM(10) BYTE,
                           OT(15) BYTE),
DATA            STRUCTURE (POL(3) BYTE,
                           (GOD ADDRESS, MES BYTE,
                           DEN BYTE));
```

Длину текста оператора описания структуры можно существенно сократить, если вновь описываемая структура подобна ранее описанной. У подобных структур отличаются лишь имена и атрибут размера старшей структуры, а имена полей, их количество, размеры и тип элементов одинаковы. Описание подобной структуры производится оператором следующего вида:

```
DECLARE имя-структуры [размерность] BY
        имя-структуры-образца;
```

Здесь атрибут BY предписывает скопировать из структуры-образца имена полей, их количество, размер и тип элементов. Однако атрибут размера самой структуры-образца не копируется, например:

```
DECLARE TOVAR(0) STRUCTURE (NAME(10) BYTE,
                           NOMNOMER ADDRESS, CENA BYTE, KOL ADDRESS);
DECLARE (OBUV(200), SHAPKA(300)) BY TOVAR;
```

Здесь в первом операторе DECLARE описывается массив структур TOVAR с размерностью, равной 0. Тем самым для такой структуры память в программе не резервируется, а структура TOVAR служит образцом для описания массивов структур OBUV и SHAPKA.

Во втором операторе DECLARE по образцу структуры TOVAR создаются массив OBUV из 200 структур и массив SHAPKA из 300 структур. Каждая из этих 500 структур будет копией структуры TOVAR.

Аналогичным образом можно использовать атрибут BY при описании подструктур. Однако здесь образцом подструктурой может быть подструктура, ранее описанная в той же структуре, внешняя по отношению к описываемой подструктуре, например:

```
DECLARE STUDENT  STRUCTURE (KURS BYTE,
FIO              STRUCTURE (FAM(20) BYTE, IM(10) BYTE,
                           OT(15) BYTE, POL(3) BYTE,
DATA            STRUCTURE (GOD ADDRESS, MES BYTE,
                           DEN BYTE),
TEACH            BY FIO);
```

Здесь поле TEACH описывается по образцу поля FIO, представленному ранее как подструктура.

В отличие от простых переменных и массивов при описании структур им нельзя присваивать начальных значений с помощью атрибута INITIAL.

Для того чтобы обратиться к определенному элементу структуры, необходимо указать полное составное имя этого элемента. Составное имя образуется из простых имен структуры, подструктур и имени самого элемента, разделенных точками. Например, чтобы однозначно обратиться к полю MES, в структуре STUDENT необходимо записать полное составное имя этого поля в следующем виде

STUDENT.DATA.MES

В языке ПЛ/М определены только одномерные массивы. Структуры являются удобным средством представления многомерных массивов.

Пример описания двумерного массива:

```
DECLARE STROKA(24) STRUCTURE STOLBEC(80) BYTE;
```

Здесь описан массив STROKA, состоящий из 24 структур, каждая из которых представляет собой массив STOLBEC из 80 элементов типа BYTE.

Для того чтобы обратиться к элементу массива STOLBEC с номером 20, находящемуся в 5-й структуре STROKA, нужно записать

STROKA(5). STOLBEC(20);

Рассмотрим пример работы с двумерным массивом. Пусть необходимо в матрице, состоящей из 10 строк и 15 столбцов, найти среднее арифметическое значение элементов, содержащих ненулевые значения, и подсчитать количество элементов с нулевым значением. Фрагмент программы может быть таким:

```
DECLARE A(10) STRUCTURE B(15) BYTE,
        (I,J,M,K) BYTE, S ADDRESS;
        .....
        S=0; K,M=0;
DO I=0 TO 9;
  DO J=0 TO 14;
    IF A(I).B(J)=0 THEN K=K+1;
    ELSE DO;
      S=S+A(I).B(J);
      M=M+1;
    END;
  END;
END;
S=S/M;
```

EOF

2.12.1. Назначение процедур

Программа на языке ПЛ/М может включать процедуры. В виде процедуры оформляется часть программы, которая должна использоваться многократно в разных частях программы при различных значениях переменных.

Применение процедур составляет основу модульного программирования, где алгоритмически независимые части программы оформляются в виде процедур — модулей. Модульное программирование позволяет:

экономить память, поскольку многократно повторяющийся участок программы (процедура) записывается в памяти только один раз;

экономить время на разработку и отладку программ, потому что программа может «собираться» из отдельных ранее разработанных и отлаженных частей — процедур;

появляется возможность параллельной работы нескольких программистов при разработке одной программы,

так как каждый из них создает относительно независимую часть программы — процедуру.

Использование процедуры предполагает наличие в программе описания процедуры, где оформляется алгоритм обработки данных, и обращения к процедуре, которое возбуждает однократное выполнение этого алгоритма.

Процедура в программе описывается один раз, обращение к ней может быть многократным из разных точек программы с различными значениями переменных. Описание процедуры в программе должно предшествовать обращению к ней.

При последовательном выполнении операторов (рис. 2.6) часть программы, составляю-

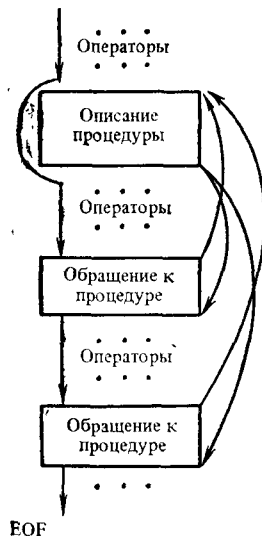


Рис. 2.6. Схема выполнения процедуры в программе

щая описание процедуры, пропускается. Инициирование выполнения процедуры происходит только при ее вызове. При этом управление передается в точку входа в процедуру. Далее выполняются операторы процедуры, определяющие алгоритм обработки данных, после чего осуществляется возврат управления из процедуры к оператору программы, записанному вслед за обращением к процедуре.

2.12.2. Описание процедуры

Описание процедуры представляет собой программно оформленный алгоритм обработки данных, который в последующем может выполняться многократно с употреблением разных входных данных.

Описание процедуры должно предшествовать ее использованию. Это означает, что описание процедуры должно встретиться в тексте программы раньше, чем обращение к этой процедуре.

Описание процедуры состоит из заголовка и тела процедуры. Заголовок содержит три части: имя процедуры, список параметров и тип результата. Все эти элементы записываются в следующей форме:

заголовок процедуры	{	имя	PROCEDURE(список параметров) тип;
тело процедуры	{	оператор-1;	
		оператор-2;	
			
		оператор-n;	
		END	имя;

Имя процедуры — это идентификатор, который в пределах одной программы должен быть уникальным. В дальнейшем процедура в программе вызывается указанием этого имени.

Ключевое слово PROCEDURE (переводится как «процедура») дает информацию транслятору для оформления процедуры как обособленного участка программы.

Список параметров представляет собой последовательность идентификаторов, разделенных запятыми. Эти идентификаторы называются формальными параметрами, поскольку при объявлении процедуры они не содержат реальных значений. Реальные значения параметры получают в момент обращения к процедуре. (В языке ПЛ/М процедуры относятся к категории «вызываемых

по значению»). Каждый из этих формальных параметров должен появиться среди операторов описания в теле процедуры, где задается его тип и размер.

Список параметров может отсутствовать, если процедуре не передается никаких значений.

Процедура может иметь тип `BYTE` или `ADDRESS`, если после выполнения тела процедуры в точку вызова передается результат. Такую процедуру называют *функцией*. Если выполнение процедуры не связано с вычислением определенного результата, то описание типа опускается. Такую процедуру называют *подпрограммой*. Тип процедуры-функции определяет точность получаемого результата. Функцию можно использовать как операнд выражения, при этом, если необходимо, происходит соответствующее преобразование типа по обычным правилам.

Тело процедуры представляет собой последовательность операторов, среди которых могут быть любые операторы, в том числе внутренние описания процедур и обращения к ним.

Операторы тела процедуры описывают заданный алгоритм обработки данных. Последним выполняемым оператором тела процедуры должен быть оператор `RETURN`. Этот оператор имеет одну из двух форм записи:

```
RETURN;
RETURN выражение;
```

Первая форма используется в подпрограммах, вторая — в функциях. В последнем случае в точку вызова передается значение выражения в операторе `RETURN`.

Примеры простых описаний процедур:

```
A: PROCEDURE(X,Y) ADDRESS;
   DECLARE(X,Y) ADDRESS;
   RETURN X+Y;
END A;
```

Здесь процедура-функция содержит в заголовке все составные части, поскольку она передает в точку вызова результат сложения двух величин.

```
B: PROCEDURE(T);
   DECLARE T BYTE;
   IF T > 5 THEN OUTPUT(S)=0;
               ELSE OUTPUT(S)=1
   RETURN;
END B;
```

В заголовке данной процедуры-подпрограммы отсутствует тип, поскольку процедура не передает результата вычислений в точку вызова.

```
DECLARE Z ADDRESS;
C:  PROCEDURE (PARAM) ADDRESS;
    DECLARE PARAM ADDRESS;
    Z=Z+PARAM;
END C;
```

Здесь отсутствует оператор `RETURN` в теле процедуры. Такая конструкция допустима, поскольку процедура не передает результата в точку вызова, а оператор `RETURN` неявно подразумевает в точке `END` в теле любой процедуры.

Процедуры не могут быть рекурсивными. Это означает, что процедура не должна содержать в теле обращений к самой себе. Кроме того, в теле процедуры не должно быть цепочки обращений к процедурам, приводящим к рекурсивному обращению.

2.12.3. Обращение к процедуре

Процедуру можно вызвать (т. е. обратиться для ее выполнения) только после того, как в тексте программы записано описание этой процедуры. Существуют две формы вызова процедуры: с помощью оператора `CALL` и с помощью указателя функции.

Если процедура не передает результат вычислений в точку вызова, то обращение к процедуре выполняется оператором вида:

`CALL имя-процедуры (список аргументов);`

Примеры:

```
CALL B(8);
CALL C(S+5);
```

При обращении к процедуре `B` передается один аргумент — число 8, при обращении к процедуре `C` — вычисленное значение выражения `S+5`.

Если процедура передает результат вычислений в точку вызова, то обращение к ней выполняется указателем функции вида

имя-процедуры (список аргументов)

который используется как один из операторов выражения, например:

$$P = W + A(T + 5, V, 3);$$

Здесь обращение к процедуре-функции А производится с двумя аргументами Т+5 и V*3. Вычисленное значение функции А складывается со значением переменной W и присваивается переменной Р.

В этих формах обращения к процедуре элементы списка аргументов иногда называют «фактическими параметрами», чтобы их отличить от формальных параметров описания процедуры.

Аргументами в обращении к процедуре могут быть любые объекты программы, имеющие скалярные значения: переменные, константы, любые допустимые в языке выражения.

В момент обращения к процедуре происходит вычисление выражений, записанных в списке аргументов. Значения аргументов присваиваются соответствующим формальным параметрам в описании процедуры, и с этими значениями процедура выполняется. Если описание процедуры не имеет списка формальных параметров, то в обращении отсутствует и список аргументов.

Если типы аргумента и параметра не совпадают, то при обращении к процедуре автоматически производится их преобразование к типу параметра. Это означает, что аргумент типа BYTE, когда его значение присваивается формальному параметру типа ADDRESS, будет расширен добавлением нулей в восьми старших разрядах. Если значение аргумента типа ADDRESS присваивается параметру типа BYTE, то значение аргумента будет усечено отбрасыванием 8 старших битов. Если тип полученного значения в операторе RETURN не соответствует типу процедуры, то возвращаемое значение приводится к типу процедуры.

Рассмотрим два примера использования процедур в программе.

Пример 1. Пусть необходимо получить среднее значение минимальных элементов в двух массивах. Алгоритм определения минимального элемента в массиве удобно оформить в виде процедуры. Представим вариант текста программы с процедурой.

Здесь употребляется процедура-функция, поэтому в заголовке процедуры указан тип возвращаемого значения — ADDRESS. Для обращения к процедуре используется указатель функции с одним аргументом — именем массива.

```

DECLARE A(10) ADDRESS INITIAL (17,18,15,165,33,64,
                                173,3,89,8);
DECLARE B(10) ADDRESS INITIAL (12,14,74,7,9,5,
                                189,73,91,111);
/* значения элементов массива А и В могут быть произвольными */
DECLARE S ADDRESS;
/* S будет хранить значение полусуммы минимальных элементов */
/* ниже следует описание процедуры MIN */
MIN: PROCEDURE(D) ADDRESS;
      DECLARE D(10) ADDRESS;
      DECLARE M ADDRESS, I BYTE;
/* M-переменная для хранения минимального элемента массива */
/* I-параметр цикла в теле процедуры */
      M=D(0);
      DO I=1 TO 9;
        IF M > D(I) THEN M=D(I);
      END;
      RETURN M;
END MIN; /* конец описания процедуры */
      S=(MIN(A)+MIN(B))/2;
EOF

```

Пример 2. Массив с произвольными значениями элементов необходимо упорядочить по возрастанию значений элементов. Алгоритм упорядочения по методу перестановок нужно оформить в виде процедуры. Текст программы приведен ниже.

```

DECLARE X(10) ADDRESS INITIAL (33,88,66,1123,72,
                                8,175,421,9,53);
/* значения элементов массива и их взаимное размещение могут
   быть произвольными */
/* ниже следует описание процедуры ранжировки */
RANG.PROCEDURE(N);
/* N-длина исходного массива */
      DECLARE(N,I,PRZ) BYTE, RAB ADDRESS;
/* PRZ-признак неупорядоченности, RAB-рабочая переменная */
      PRZ=1;
/* при значении признака, равном единице, ранжировка не
   закончена */
      DO WHILE PRZ;
        PRZ=0; /* начало просмотра */
        DO I=0 TO N-2;
          IF X(I) > X(I+1) THEN
            DO; RAB=X(I);
              PRZ=1; /* отметка факта перестановки */
              X(I)=X(I+1);
              X(I+1)=RAB;
            /* перестановка двух соседних элементов по возрастанию значений */
          END;
        END; /* просмотр завершен */
      END; /* конец цикла с WHILE */
/* ранжировка завершена */
END RANG; /* конец процедуры RANG */
CALL RANG(10); /* обращение к процедуре RANG */
EOF

```

Здесь процедура не передает в точку вызова результата вычислений, поэтому в заголовке процедуры отсутствует атрибут типа, а обращение к процедуре выполняется оператором CALL.

При программировании процессов, выполняющихся в реальном масштабе времени, необходимы средства управления интервалами времени. Для этой цели в языке ПЛ/М имеется встроенная процедура TIME. С ее помощью организуется задержка выполнения операторов программы на интервал времени, определяемый значением аргумента.

Общий вид обращения к процедуре TIME:

CALL TIME (выражение);

Аргумент этой процедуры имеет тип BYTE, а его значение определяет интервал времени, кратный 100 мкс.

Пример:

```
DECLARE (X,Y) BYTE
OUTPUT (5)=X;
CALL TIME (50);
OUTPUT (5)=Y
.....
```

Здесь вывод значения переменной Y будет задержан на 5 мс относительно момента вывода значения переменной X.

Максимальный интервал задержки, обеспечиваемый одним обращением к процедуре TIME, равен 25,5 мс. Если требуется задержка более чем на 25,5 мс, то необходимо повторить обращение к процедуре TIME. Для получения задержки в 0,5 с можно использовать следующий фрагмент программы:

```
DO I=1 TO 20;
  CALL TIME (250);
END;
```

2.13. БАЗИРОВАННЫЕ ПЕРЕМЕННЫЕ И АДРЕСНЫЕ ССЫЛКИ

2.13.1. Базированные переменные

При загрузке программы в память за именем каждой переменной закрепляется конкретное поле памяти длиной либо в один байт, если переменная имеет тип BYTE, либо в два байта, если переменная имеет тип ADDRESS. В любой момент выполнения программы и

в любом ее месте использование имени переменной означает обращение к содержимому конкретного поля памяти.

Часто бывает удобнее не закреплять за именем переменной определенного поля, а динамически, в процессе выполнения программы, назначать разные поля в зависимости от значения другой переменной — так называемой базы. База — это простая переменная, имеющая тип ADDRESS.

Переменную, адрес которой определяется значением другой переменной, называют *базированной переменной*.

В процессе выполнения программы, изменяя значение базы, можно в разные моменты связывать с именем одной базированной переменной разные поля памяти. Таким образом, базированной переменной не выделяется определенное поле памяти, а ее адрес каждый раз вычисляется через базу. Следовательно, перед описанием базированной переменной всегда описывается ее база.

Описание базированной переменной производится оператором DECLARE:

```
DECLARE имя-базированной-переменной BASED имя-базы
атрибуты-базированной-переменной;
```

Ключевое слово BASED должно записываться сразу за именем базированной переменной. Вслед за BASED должно записываться имя базы — предварительно описанной переменной типа ADDRESS. Только после имени базы можно записывать другие атрибуты (размер массива, тип).

Базированные переменные не должны иметь атрибута INITIAL, поскольку за ними не закрепляется определенного места в памяти. Кроме того, база не может быть сама базлируемой переменной и компонентом массива.

Примеры описаний базированных переменных:

```
DECLARE (V,X,Y,Z) ADDRESS;
DECLARE A BASED X BYTE;
DECLARE (B BASED Y) (100) ADDRESS;
DECLARE (C BASED Z, D BASED V) ADDRESS;
```

Здесь первый оператор DECLARE описывает четыре переменных V, X, Y и Z типа ADDRESS, которые в последующем будут использоваться как базы. Второй оператор описывает базированную переменную A типа BYTE. Это описание предполагает, что

переменная А будет найдена по адресу, определяемому текущим значением переменной X.

Третий оператор DECLARE описывает имя В как имя базированного массива, состоящего из 100 элементов типа ADDRESS. Начальный адрес размещения этого массива определяется значением базы Y. В момент трансляции массиву В не будет выделено место в памяти. Атрибут размера (100) обеспечивает просто необходимые значения для встроенных функций LENGTH и LAST и указывает, что идентификатор В является именем массива, а не именем простой переменной. Базированные переменные могут иметь индексы, если они являются элементами массива.

Четвертый оператор DECLARE описывает сразу две базированные переменные типа ADDRESS с соответствующими базами Z и V.

Поле структуры может иметь базированное имя. В этом случае его базой должно быть ранее описанное поле той же структуры, например:

```
DECLARE BAZA STRUCTURE (PTR ADDRESS,  
    SPISOK BASED PTR STRUCTURE (AVTOR(20)  
    BYTF, NAML(30) BYTF, GODIZD ADDRESS));
```

Здесь представлена базированная подструктура SPISOK, базой которой является ранее описанное поле PTR структуры BAZA типа ADDRESS.

2.13.2. Адресные ссылки

Для практического использования базированных переменных необходимо иметь средства определения адресов объектов программы. Таким средством является адресная ссылка. Она записывается в одной из следующих форм:

- переменная
- константа
- (константа)
- (список констант)

Значением адресной ссылки является величина типа ADDRESS, равная адресу аргумента (объекта, записанного за точкой). Например, запись «А» дает значение адреса переменной «А». Запись «.37» определяет адрес размещения константы 37 в памяти. Запись «.TEXT» дает величину типа ADDRESS, указывающую на первый символ строки символов TEXT в коде КОИ-7. Списку констант, разделенных запятыми и заключенных в скобки, может предшествовать точка, например:

```
.(OBN, 'ALFA', OA9H)
```

Значением адресной ссылки в этом случае является адрес первого байта первой константы из списка констант.

Значением адресной ссылки на базированную переменную является значение базы, например:

```
DECLARE (BAZA, T) ADDRESS;  
DECLARE ARGUM BASED BAZA BYTE;  
BAZA=125H;  
T=ARGUM;
```

Здесь переменная Т получит значение 125H, потому что переменная ARGUM базирована на переменную BAZA, которая имеет значение 125H.

Адресные ссылки можно использовать во всех выражениях ПЛ/М наряду с другими переменными и константами. Однако наиболее целесообразно употреблять адресные ссылки и базированные переменные в процедурах. С помощью этих средств в процедурах можно оперировать не самими объектами (аргументами), а их адресами. Это обстоятельство позволяет разрабатывать и использовать процедуры, не зависящие от размеров объектов, т. е. создавать более универсальные процедуры, пригодные для работы с массивами, имеющими разное количество элементов.

В разд. 2.12.3 был приведен пример программы подсчета полусуммы минимальных элементов двух массивов. Алгоритм определения минимального элемента в массиве оформлен в виде процедуры. Ее возможности весьма ограничены, в частности, она может обрабатывать только массивы, состоящие из 10 элементов. Кроме того, в качестве параметра процедуры используется имя массива. Следовательно, для работы процедуры требуется в памяти резервировать некоторый объем для размещения элементов массива-параметра, что весьма неэкономично.

С помощью базированных переменных эту процедуру можно сделать более универсальной, пригодной для обработки массива любого размера без использования дополнительного объема памяти. Процедуры, позволяющие оперировать с массивами произвольного размера без их конкретных имен, становятся универсальными и могут применяться в различных блоках программ всеми разработчиками. Пример программы с такой процедурой приведен ниже.

```

DECLARE A(7) ADDRESS INITIAL(17,18,15,165,33,64,3),
        S ADDRESS;
DECLARE B(15) ADDRESS INITIAL(13,17,74,7,9,5,189,
        73,91,115,97,63,41,28,36);
MEN1:PROCEDURE (POINTER,N) ADDRESS;
    DECLARE POINTER ADDRESS;
    D BASED POINTER ADDRESS;
    DECLARE M ADDRESS,(N,I) BYTE;
    M=D(0);
    DO I=1 TO N;
        IF M>D(I) THEN M=D(I);
    FND,
    RETURN M;
END MEN1;
S=(MEN1(.A,LENGTH(A))+MEN1(.B,LENGTH(B)))/2;
EOF

```

В процедуре MEN1 в качестве параметров используются адресная ссылка на первый элемент массива и длина массива, поэтому при обращении к процедуре аргумент-массив не пересылается в тело процедуры: туда передается только его адрес и массив обрабатывается на месте его размещения. При этом не требуется дополнительного объема памяти для размещения элементов массива-аргумента. Процедура MEN1 пригодна для определения минимального элемента в массиве любого размера.

2.14. БЛОЧНАЯ СТРУКТУРА ПРОГРАММЫ

Язык ПЛ/М является языком с блочной структурой. Это означает, что определенные части программы, а именно блоки, можно программировать таким образом, чтобы исключить нежелательные взаимосвязи между другими блоками и остальной частью программы. Такая возможность обеспечивается благодаря существующему в языке принципу локализации описанных в блоке объектов, что делает эти объекты независимыми от текстуального окружения в программе.

Блочная структура программы позволяет вести раздельную (и почти независимую) разработку частей одной программы разными программистами и использовать в разрабатываемой программе готовые, ранее составленные и отлаженные части программы (блоки, процедуры).

2.14.1. Объекты и область их существования

Объектом принято называть следующие элементы программы: имена скалярных переменных и констант; имена массивов и структур; имена процедур; макроимена; идентификаторы меток.

Для каждого объекта в программе существует некоторая часть программы, где этот объект доступен. Такая часть программы называется *областью существования объекта*. Вне этой области объект использоваться не может (он не существует).

Для определения понятия «область существования» уточним понятие «блок». Блоком является составной оператор, процедура, вся программа. Исходя из этого определения следует, что составной оператор DO — END может содержать внутри себя операторы описания DECLARE и другие вложенные в него составные операторы, содержащие, в свою очередь, операторы описания.

Конструкции DO — CASE, DO — WHILE и итеративный цикл DO не являются блоками, следовательно, не могут содержать операторов описания.

Блок может целиком входить внутрь другого блока, тогда он называется *внутренним* по отношению к охватывающему его *внешнему* блоку. Блоки могут быть не-

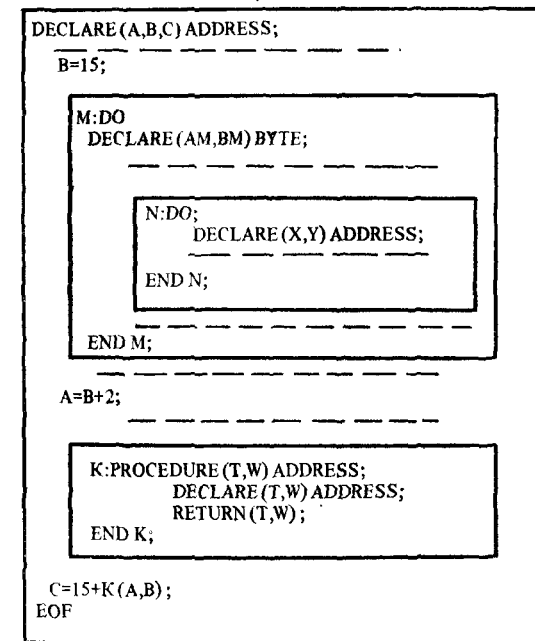


Рис. 2.7. Блочная структура программы

зависимыми, т. е. по отношению друг к другу не быть внешними и внутренними.

На рис. 2.7 представлена программа с блочной структурой, где блок М является внешним по отношению к внутреннему блоку N. В то же время блоки М и К независимы относительно друг друга, хотя оба являются внутренними по отношению к программе в целом. Максимальный блок — это вся программа.

Принцип локализации объектов заключается в том, что все объекты, описанные в некотором блоке, считаются локализованными в этом блоке и недоступны (не существуют) вне этого блока. Например, следующая программа имеет два блока:

```

DECLARE(A,B) BYTE;
  A=3;
  DO;
    DECLARE C BYTE;
    C=A-17;
  END;
  B=A+10;
EOF

```

Здесь группа DO — END образует внутренний блок.

Вся программа в целом также является блоком. Областью определения переменных А и В является вся программа, поскольку они описаны во внешнем блоке, а областью определения переменной С — только группа DO — END, так как переменная С описана внутри этого блока. Это значит, что переменные А и В могут использоваться во всей программе, в то время как переменная С — только в блоке DO — END. Ссылка на С в программе за пределами группы DO — END будет ошибкой, поскольку за пределами своей области определения переменная С просто не существует.

В приведенном примере во внутреннем блоке использовалось имя переменной С, отличное от имен переменных А и В во внешнем блоке. Однако блочная структура позволяет употреблять одинаковые имена объектов во внешнем и внутреннем блоках, поэтому при определении области существования объектов необходимо соблюдать следующее правило: областью существования имени объекта является наименьший охватывающий блок, где описано это имя, за исключением тех вложенных в него блоков, в которых описано такое же имя.

Рассмотрим пример программы, представленный на рис. 2.8. Здесь переменная I описана в строке 13, содер-

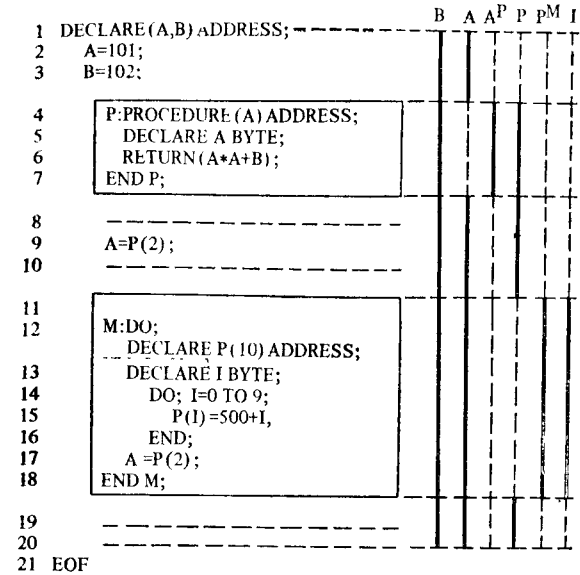


Рис. 2.8. Области существования объектов (показаны вертикальными линиями)

жащейся в блоке М. Областью существования переменной I является блок М.

Переменная В описана в строке 1, следовательно, областью ее существования является вся программа (строки 1—20). Переменная А описана в строке 1 там же, где и переменная В. Но в процедуре Р в строке 5 описана другая переменная А, областью существования которой является вся процедура Р (точнее, строки 5—7). Следовательно, переменная А, описанная в строке 1, существует везде в программе, кроме строк 5—7 процедуры Р. Таким образом, в этой программе существуют две разные переменные с одним именем А и одна переменная с именем В.

Описание процедуры Р начинается в строке 4, а внешним блоком, содержащим это описание, является вся программа. Областью существования процедуры Р будет вся программа, за исключением блока М, где идентификатор Р описан уже как имя массива. Наименьшим охватывающим блоком по отношению к описа-

нию массива Р является блок М, областью существования массива Р — строки 12—18, а областью существования процедуры Р — строки 4—11 и 19—20. Таким образом, в программе имеются два разных объекта с одним и тем же именем, но с разными (непересекающимися) областями существования и разным смыслом.

Рассмотрим два одинаковых по начертанию оператора присваивания в строках 9 и 17. В строке 9 глобальной (объявленной в самом внешнем блок-программе) переменной А присваивается значение указателем функции Р с аргументом, равным 2, значение переменной А равно 106 ($2 \cdot 2 + B$). В строке 17 той же переменной А присваивается значение третьего элемента массива Р; следовательно, переменная А получит новое значение, равное 502.

Область существования имени процедуры устанавливается по общим правилам определения областей существования объектов программы. Так, если процедура описана во внутреннем блоке, то областью существования ее имени является минимальный блок, охватывающий описание этой процедуры. Это не всегда удобно, особенно в тех случаях, когда возникает необходимость раздельной отладки частей программы.

Для процедур (как функций, так и подпрограмм) можно явным образом указать область существования использованием в их описаниях атрибутов PUBLIC и EXTERNAL. Эти атрибуты записываются в заголовке процедуры вслед за атрибутом типа BYTE или ADDRESS для функций или за списком параметров — для подпрограмм.

Описание процедуры с атрибутом PUBLIC называется *определяющим* описанием, а описание той же процедуры с атрибутом EXTERNAL — *использующим*. Определяющее описание с атрибутом PUBLIC ничем не отличается от обычного описания процедуры. В использующем описании (с атрибутом EXTERNAL) тело процедуры содержит только операторы описания формальных параметров.

Одна и та же процедура в программе может иметь только одно определяющее описание и любое число использующих описаний. В определяющем и во всех использующих описаниях одной и той же процедуры должны совпадать имена процедуры, количество параметров, порядок их следования в списке, а также тип параметров.

Применение атрибута EXTERNAL возможно только в том случае, если эта процедура имеет определяющее описание. Областью существования процедуры, описанной с атрибутами PUBLIC и EXTERNAL, является часть программы, заключенной между первым описанием процедуры (с атрибутом PUBLIC или EXTERNAL) и концом программы.

Пример:

```

DECLARE (X,Y,Z,D) BYTE, (T,W,V) ADDRESS;
-----
BL1: DO;
/* использующее описание функции SUM */
SUM: PROCEDURE (A,B) ADDRESS EXTERNAL;
      DECLARE (A,B) BYTE;
      END SUM;
-----
W=SUM (X,Y); /* обращение к функции SUM */
-----
BL2: DO;
/* определяющее описание функции SUM */
SUM: PROCEDURE (A,B) ADDRESS PUBLIC;
      DECLARE (A,B) BYTE;
      RETURN (A+B);
      END SUM;
-----
V=SUM (Z,D); /* обращение к функции SUM */
-----
END BL2;
-----
END BL1;
-----
BL3: DO
-----
/* использующее описание функции SUM */
SUM: PROCEDURE (A,B) ADDRESS EXTERNAL;
      DECLARE (A,B) BYTE;
      END SUM;
-----
T=SUM (X,W); /* обращение к функции SUM */
-----
END BL3;
EOF

```

Здесь процедура-функция SUM описана во внутреннем блоке BL2. Без использования атрибутов PUBLIC и EXTERNAL областью существования имени этой функции был бы участок блока BL2 между точкой описания этой функции и концом блока BL2.

Применение определяющего и использующих описаний расширяет область существования имени этой функции.

Алгоритмы наиболее часто используемых процессов обработки данных оформляются в виде процедур, причем в целях экономии памяти эти процедуры программируются на языке ассемблера. Библиотека процедур на языке ассемблера обычно содержит процедуры вычисления значений математических функций ($\sin x$, $\cos x$ и другие с обычной или удвоенной точностью), а также процедуры, обеспечивающие сервисные функции: ввод данных с консоли, вывод результатов на дисплей, загрузка программы в память и др. К этим процедурам возможно обращение из программ, написанных на языке ПЛ/М. Например, в системах, созданных на базе микроЭВМ CM-1800, для этого необходимо в программе на языке ПЛ/М указать использующее описание библиотечной процедуры специального формата:

```
имя-процедуры EXTERNAL адрес-точки-входа
                [(параметры-входа)] [результат];
```

где адрес-точки-входа — абсолютный адрес точки входа в процедуру;

параметры-входа — мнемонические имена регистров, где размещаются входные данные для работы процедуры: A, B, C, D, E, H, L, BC, DE, HL. Если используется несколько регистров, то они перечисляются в списке через запятые, а весь список заключается в скобки;

результат — часть описания, которая используется в том случае, если процедура является функцией. В описании функции указывается тип результата (BYTE или ADDRESS) и мнемоническое обозначение регистра для размещения результата.

Примеры описаний библиотечных процедур:

```
TTI: EXTERNAL 52H BYTE A;
LOAD-EXTERNAL 6DH(A,BC) ADDRESS HL;
```

В первой процедуре-функции TTI, предназначенной для ввода символа с консоли, результатом является введенный символ, который размещается в регистре A. Адрес входа в процедуру — 52H.

Вторая процедура-функция предназначена для загрузки программы в память начиная с адреса, находящегося в регистре HL. Загрузка происходит с устройства, номер которого находится в регистре A, а указатель имени располагается в регистре BC.

Имена встроенных процедур не являются ключевыми словами. Они считаются описанными в самом внешнем блоке программы. Правила определения области действия этих имен такие же, как и для остальных пе-

ременных, следовательно, внутри любого внутреннего блока программы имена встроенных функций и процедур могут быть вновь описаны с другим смыслом. В связи с этим имена встроенных процедур называют определенными именами в отличие от ключевых слов.

2.14.2. Описание и область существования меток

Появление в тексте программы неопisanного идентификатора неявным образом определяет этот идентификатор как метку. С этой точки программы начинается область существования метки.

Оператор описания метки может иметь одну из двух форм:

```
DECLARE идентификатор LABEL;
DECLARE (список идентификаторов) LABEL;
```

В первой форме идентификатор описывается как метка (LABEL — метка), а во второй — несколько имен описываются метками. В списке идентификаторы разделяются запятыми.

Рассмотрим три примера использования меток.

Пример 1. Следующий фрагмент программы

```
Z=X+1;
IF Z > Y THEN GO TO M;
```

эквивалентен такому фрагменту

```
Z=X+1;
DECLARE M LABEL;
IF Z > Y THEN GO TO M;
```

Здесь показано, что неявное описание метки M эквивалентно размещению оператора описания метки M перед оператором, в котором первый раз используется эта метка.

Пример 2.

```
DECLARE (M1,M2,M3) LABEL;
```

Здесь имена M1, M2 и M3 объявляются метками.

Пример 3. Предположим, что в программе используется оператор

```
MM : Z=Y+2;
```

Этот оператор эквивалентен следующему фрагменту программы:

```
DECLARE MM LABEL;
MM:Z=Y+2;
```

В этом фрагменте предполагаемое описание метки записано непосредственно перед помеченным оператором.

Область существования метки, как и других объектов программы, определяется по правилам, изложенным выше. Обычно нет необходимости в явном описании меток. Оно требуется только в том случае, если неявное описание приводит к ошибкам в программе, имеющей вложенные блоки.

Рассмотрим пример, где во избежание ошибок необходимо явное описание метки. Пусть в следующем фрагменте программы необходимо из внутреннего блока

```
      X=X+1;
KK: DO;
      DECLARE M LABEL;
      GO TO M;
END KK;
      DECLARE M LABEL
      M: Z=X*2+Y;
      .....
EOF
```

KK передать управление оператору, размещенному во внешнем блоке. В соответствии с правилами неявного описания меток этот фрагмент программы эквивалентен следующему:

```
      X=X+1;
KK: DO;
      GO TO M;
END KK;
      M:Z=X*2+Y;
      EOF
```

Первое (по тексту программы) использование метки М определяет область ее существования — внутренний блок KK. Следовательно, адрес перехода в операторе GO TO неопределен, поскольку второе использование (в операторе присваивания M:Z=X*2+Y;) находится за пределами области существования первой метки М.

Для того чтобы иметь возможность передать управление во внешний блок, необходимо метку М явно описать во внешнем блоке. Тогда фрагмент программы будет иметь следующий вид:

```
DECLARE M LABEL;
      X=X+1;
      .....
KK: DO;
      GO TO M; ..
END KK;

      M:Z=X*2+Y;
      EOF
```

Теперь фрагмент программы записан правильно, поскольку областью существования метки М является весь внешний блок, а неявное описание метки во внутреннем блоке «подавлено» явным ее описанием во внешнем блоке. Теперь во всей программе существует только одна метка М.

В общем случае перед любым выполняемым оператором программы можно записать несколько меток. Например:

```
M:P:МЕТКА:T:W=V-1;
```

Друг от друга они отделяются двоеточием. Метка может быть также числом:

```
40:X=X+1;
```

Этот оператор в объектном коде должен быть загружен в основную память начиная с адреса 40. Оператору может предшествовать только одна числовая метка.

Если наряду с числовой меткой в одном операторе используются символьные метки, то числовая метка должна быть первой слева:

```
128:M1:M3:X=X*2-Z;
```

В связи с наличием числовых меток оператор перехода GO TO может иметь одну из трех форм записи:

```
GO TO имя метки;
GO TO имя переменной;
GO TO число;
```

В первом случае именем метки является идентификатор, используемый в качестве метки оператора. Во втором случае имя переменной указывает переменную, содержащую предварительно вычисленный адрес. В третьем случае число представляет собой абсолютный адрес памяти, и управление передается прямо по этому адресу.

Две последние формы записи оператора GO TO следует использовать крайне осторожно, поскольку наличие

исполняемых команд по адресу, указанному в операторе GO TO, не гарантировано. В этом операторе вообще нецелесообразно использовать абсолютный адрес, так как здесь всегда можно указать символическое имя перехода.

2.14.3. Особенности использования блоков

Необходимые описания рекомендуется размещать в начале блока перед исполняемыми операторами. Это дает возможность легко устанавливать область существования объектов, поскольку она совпадает со всем блоком в целом, а правильное определение области существования позволяет избежать большого числа программистских ошибок, связанных с неправильным определением «самого внутреннего охватывающего блока».

Переход к выполнению любого блока должен всегда осуществляться с начала блока, а завершение выполнения блока — через его конец (в процедуре — через оператор RETURN).

Безусловный переход к оператору GO TO в середину блока или тела процедуры может привести к непредсказуемым результатам. Переход к выполнению процедуры должен осуществляться только путем обращения к процедуре (либо оператором CALL, либо указателем функции). Выход из тела процедуры по оператору GO TO также может привести к непредсказуемым результатам, поскольку при этом игнорируется обычный механизм обработки выхода оператором RETURN. В связи с этим недопустимо указывать в теле процедуры оператор GO TO, передающий управление за пределы процедуры, если передача происходит не на самый внешний блок программы. Такие безусловные переходы оправданы необходимостью выхода при обнаружении существенных ошибок.

Излишнее использование операторов безусловного перехода всегда усложняет отладку программы и затрудняет ее модификацию.

Чтение текста программы и процесс отладки значительно облегчаются, если придерживаться следующих рекомендаций по размещению отдельных частей программы на листинге. Блоки (процедуры, группы DO — END) следует отделять от предшествующего и последующего текста пустой строкой. Текст каждого блока

должен смещаться на некоторое число позиций от левой границы размещения основной программы. При этом выдерживается совмещение начальной и завершающей меток блока по вертикали. При соблюдении такого порядка размещения блоков в программе ясно просматривается структура программы и глубина вложенности блоков.

Блочная структура языка обеспечивает возможность создания полностью независимых программных модулей. Процедуры можно сделать независимыми от остальной части программы (за исключением числа и типа параметров). Появляется возможность переносить процедуры из одной программы в другую. Кроме того, можно накапливать процедуры в библиотеках и создавать новые программы из готовых частей-блоков и процедур.

2.15. МАКРООПРЕДЕЛЕНИЯ

В программе, написанной на языке ПЛ/М, можно связать строку произвольных знаков с одним именем и в дальнейшем записывать его вместо строки в тексте программы всюду, где встречается эта строка. Такое описание имени для строки производится оператором DECLARE следующего вида:

DECLARE имя LITERALLY 'строка';

где имя — обычный идентификатор;

строка — произвольный текст длиной не более 255 литер.

После описания в тексте программы можно использовать «имя» всюду, где встречается «строка».

В процессе трансляции программы всюду, где встречается имя, описанное с атрибутом LITERALLY, вставляется строка, записанная после LITERALLY. В качестве такой строки можно использовать любой текст, в том числе и фрагмент программы, если этот фрагмент часто повторяется в программе.

Использование макроопределений позволяет существенно сократить исходный текст программы и тем самым снизить количество ошибок перфорации. Следует заметить, что использование макроопределений не уменьшает объема программы, полученной после трансляции.

Пример. Требуется подсчитать среднее арифметическое элементов массива, значения которых находятся в заданных пределах.

Решение.

```

DECLARE LIT LITERALLY 'LITERALLY';
DECLARE DCL LIT 'DECLARE';
DCL IN LIT 'INITIAL';
DCL X(15) BYTE IN (17,6,75,9,24,39,42,98,
                  6,63,124,3,31,8,175);
DCL Z1 BYTE IN (10), Z2 BYTE IN (40);
DCL S ADDRESS, (I,K) BYTE;
      S=0;      K=0;
DO I=0 TO 14;
  IF X(I) >= Z1 AND X(I) <= Z2 THEN
    DO; S=S+X(I);
      K=K+1;
    END;
  END;
END;
      S=S/K;
EOF

```

Здесь первый оператор DECLARE описывает имя LIT как заменитель строки LITERALLY. Во втором операторе DECLARE это имя уже используется для того, чтобы строке DECLARE назначить имя DCL. В последующих операторах оба этих имени употребляются для сокращения операторов описания переменных, а для текста INITIAL выбрано имя IN.

2.16. ВСТРОЕННЫЕ ФУНКЦИИ СДВИГА КОДОВ

В языке ПЛ/М имеется шесть встроенных функций, с помощью которых можно выполнять сдвиги двоичных значений аргументов на заданное число разрядов. Обращение к этим функциям производится указателем функции следующего формата:

имя-функции (аргумент-1, аргумент-2)

где имя-функции — одно из следующих предопределенных имен: ROL, ROR, SCL, SCR, SHL, SHR;
 аргумент-1 — выражение, двоичное значение которого подвергается сдвигу;
 аргумент-2 — выражение типа BYTE, значение которого задает количество сдвигов.

Второй аргумент этих функций не может иметь нулевого значения.

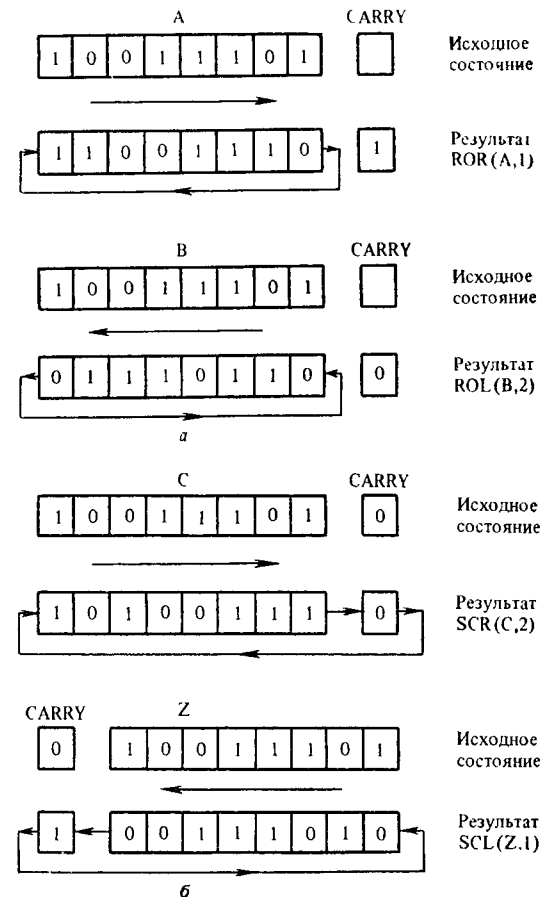
Встроенные функции с именами ROL и ROR производят циклический сдвиг кода первого аргумента типа BYTE. Функция ROL сдвигает код первого аргумента влево, а функция ROR — вправо. При этих сдвигах разряды, выходящие за пределы байта, появляются снова с другой стороны байта. При выполнении этих функций

производится установка признака переноса CARRY в соответствии со значением последнего сдвинутого бита, как показано на рис. 2,9,а.

Встроенные функции SCL и SCR выполняют циклический сдвиг кода первого аргумента с участием признака переноса CARRY.

Первый аргумент для этих функций может иметь тип BYTE либо тип ADDRESS. Тип результата определяется типом первого аргумента.

Первый аргумент сдвигается влево (SCL) или вправо (SCR) на число разрядов, заданное значением второ-



Продолжение

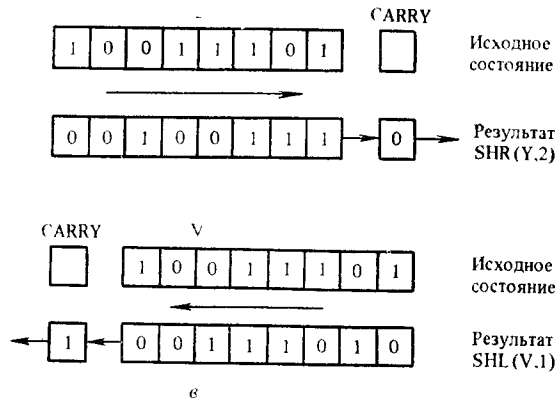


Рис. 2.9. Схема выполнения функций: а — ROR и ROL; б — SCR и SCL; в — SHR и SHL

го аргумента, так же как и при выполнении функций ROL и ROR. Однако в сдвигах в качестве дополнительного разряда первого аргумента (9-го, если первый аргумент имеет тип BYTE, или 17-го при первом аргументе типа ADDRESS) используется бит из признака переноса CARRY. Очередной выдвинутый из аргумента разряд поступает в CARRY, а старое значение CARRY вдвигается с другого конца первого аргумента, как показано на рис. 2.9, б.

Функции логического сдвига SHL и SHR выполняются аналогично соответствующим функциям SCL и SCR. Здесь первый аргумент может иметь тип либо BYTE, либо ADDRESS; тип результата определяется типом первого аргумента. При сдвигах влево (SHL) или вправо (SHR) выдвинутые разряды поступают в признак CARRY, а выдвинутые из CARRY разряды теряются. Освободившиеся разряды первого аргумента заполняются двоичными нулями, как показано на рис. 2.9, в.

2.17. ВСТРОЕННЫЕ ФУНКЦИИ ДЕСЯТИЧНОЙ АРИФМЕТИКИ

В языке ПЛ/М имеется средство, с помощью которого можно значительно упростить арифметические операции с данными в десятичной системе счисления. Для

использования этих средств необходимо, чтобы десятичные данные (переменные и константы) имели тип BYTE. Их можно представить состоящими из двух полей по 4 бита в каждом поле. Поле содержит одну десятичную цифру от 0 до 9 в двоичном представлении. Такую величину типа BYTE называют двоично-десятичной парой.

Представление числа в виде строки десятичных цифр, в которой каждая цифра является четырехразрядным двоичным числом, называют двоично-десятичной формой десятичного числа. В двоично-десятичной паре младшая цифра двухзначного десятичного числа занимает 4 младших разряда байта, а старшая цифра — 4 старших разряда байта. Например, десятичные числа

73 85 90 17

запишутся в виде следующих шестнадцатеричных констант

73H 85H 90H 17H

и будут в памяти машины представлять двоично-десятичные пары.

Двоично-десятичные пары можно складывать с помощью операций + и PLUS, а результат сложения преобразовывать снова в двоично-десятичную пару с помощью встроенной функции DEC. Указатель этой функции записывается в одном из следующих форматов:

DEC (D1+D2)
DEC (D1 PLUS D2)

где D1 и D2 — двоично-десятичные пары, т. е. переменные типа BYTE без индексов, константы или выражения, результаты которых представляются двоично-десятичной парой, например вложенным обращением к функции DEC.

Результатом выполнения функции DEC является двоично-десятичная пара суммы аргументов D1 и D2 с учетом возможного переноса из младшего разряда суммы в старший. Если полученная сумма превышает 99H, то признак переноса CARRY устанавливается в 1.

Примеры:

Указатель функции	Результат суммирования	Признак переноса
DEC (25H+37H)	62H	0
DEC (73H+84H)	57H	1
DEC(DEC(25H+15H)+37H)	77H	0
DEC(DEC(38H+13H)+DEC(18H+45H))	14H	1

Операция PLUS используется вместо операции «+» в тех случаях, когда нужно сложить десятичные числа, содержащие более двух десятичных цифр. В этих случаях необходимо сначала сложить с помощью функции DEC с операцией «+» два младших разряда этих чисел, а остальные пары последовательно складывать с использованием функции DEC с операцией PLUS, для того чтобы учесть возможные переносы от младших пар к старшим. Например, чтобы сложить число 1234 с числом 5678, нужно сначала сложить пару младших цифр

```
DEC(34H+78H)
```

а затем пару старших цифр

```
DEC(12H PLUS 56H)
```

Такая последовательность выполнения функции DEC обеспечит учет переноса из суммы младших цифр в сумму старших. Этот пример можно решить следующим фрагментом программы:

```
DECLARE (M,S) BYTE;
M=DEC(34H+78H);
S=DEC(12H PLUS 56H);
.....
```

Здесь младшие цифры результата (12H) будут помещены в переменную M в виде двоично-десятичной пары, а старшие разряды результата 69H — в переменную S.

Для того чтобы не исказить результат сложения двух многоразрядных десятичных чисел за счет случайного изменения значений признака переноса CARRY, нужно между двумя обращениями к функции DEC употреблять только скалярные операторы присваивания, которые не изменяют признака CARRY.

Пример сложения двух шестиразрядных десятичных чисел. Одно число составлено из переменных X1, X2, X3, причем в переменной X3 находятся две младшие цифры первого числа, в переменной X1 — две старшие цифры этого числа. Переменные Y1, Y2 и Y3 хранят двоично-десятичные пары второго числа. Результат сложения размещается в переменных Z1, Z2 и Z3. Фрагмент программы сложения этих чисел может иметь вид:

```
DECLARE (X1,X2,X3,Y1,Y2,Y3) BYTE;
DECLARE (Z1,Z2,Z3) BYTE;
Z3=DEC(X3+Y3);
Z2=DEC(X2 PLUS Y2);
Z1=DEC(X1 PLUS Y1);
```

Для сложения двух десятичных чисел любой разрядности, каждое из которых представлено вектором двоично-десятичных пар, можно предложить следующий фрагмент программы:

```
DECLARE (I,PER) BYTE;
DECLARE (X,Y,Z) (10) BYTE;
DECLARE (T,V,W) BYTE;
.....
PER=0;
DO I=0 TO LAST(X);
T=X(I);
V=Y(I);
W=DEC(T+PER);
PER=CARRY;
W=DEC(W+V);
PER=(PER OR CARRY) AND 1;
Z(I)=W;
END;
```

Здесь предполагается, что младшие двоично-десятичные пары складываемых чисел размещаются в элементах векторов X и Y с индексом 0.

В языке ПЛ/М нет средств для непосредственного выполнения других арифметических операций с десятичными данными, кроме сложения. Однако эти операции можно выполнять, используя дополнительные коды десятичных чисел и операции сдвига.

Вычитание можно заменить сложением с дополнительным кодом вычитаемого. Например, для двоично-десятичной пары X величина 99H — X представляет собой дополнительный код X.

Десятичное умножение (деление) можно выполнять путем повторения сложения (вычитания) со сдвигом влево (вправо) на четыре разряда после каждого сложения (вычитания). Арифметические операции производятся с помощью неупакованных десятичных чисел, где каждая цифра занимает один байт.

2.18. СРЕДСТВА ОБРАБОТКИ ПРЕРЫВАНИЙ

Прерывание — это временная приостановка выполняющейся в данный момент программы или участка программы с запоминанием состояния процессора в момент прерывания и переход к выполнению другой программы (или участка программы). После окончания выполнения программы, вызванной прерыванием, происхо-

дит восстановление состояния процессора и переход к выполнению прерванной (приостановленной) программы с той ее точки, где она была приостановлена. Таким образом, прерывание можно рассматривать как асинхронный вызов определенных программ (процедур), при котором активный в данный момент процесс (программа) приостанавливается, состояние процессора запоминается и вызывается специальная процедура обработки прерывания. По окончании ее выполнения состояние процессора восстанавливается, и управление возвращается прерванному процессу.

В языке ПЛ/М имеются специальные средства, которые совместно с механизмом прерываний позволяют вести обработку внешних прерываний. Прерывание представляет собой внешнее асинхронное обращение к специальным, разрабатываемым программистом процедурам. В любую программу можно включить не более восьми различных процедур обработки прерываний, соответствующих восьми типам внешних прерываний. Средствами языка можно разрешить или запретить (замаскировать) все прерывания. В последнем случае наличие прерываний не вызывает в программе никаких действий по их обработке.

Запрет и разрешение прерываний выполняются с помощью операторов

```
DISABLE;
ENABLE;
```

Оператор DISABLE переводит процессор в состояние, при котором замаскированы все прерывания. Оператор ENABLE выводит процессор из этого состояния, разрешая обработку прерываний по мере их появления. Процессор всегда начинает работу с замаскированными прерываниями, поэтому в программе, содержащей процедуры обработки прерываний, необходимо разрешить прерывания оператором ENABLE до того, как произойдет обращение к любой из процедур обработки прерываний.

Для обработки прерываний в программу на языке ПЛ/М необходимо включить соответствующие процедуры. Они не имеют параметров и типа и описываются специальным атрибутом INTERRUPT в виде

```
INTERRUPT n
```

где n — число в диапазоне от 0 до 7, соответствующее одному из восьми возможных типов внешних прерываний. Процедура обра-

ботки прерываний должна быть описана в самом внешнем блоке программы.

Пример:

```
DECLARE NMAX LITERALLY '100'
DECLARE MAS(NMAX) BYTE, N BYTE;
DECLARE OVERFLOW LABEL;
/* процедура обработки прерывания PRER */
PRER: PROCEDURE INTERRUPT 3;
    DECLARE Z BYTE;
    N=N+1;
    IF N > NMAX THEN GO TO OVERFLOW;
    IF (Z:=INPUT(5))='A' THEN RETURN;
    MAS(N)=Z;
END PRER;
/* основная программа */
ENABLE;
N=0;
.....
OVERFLOW:DISABLE; /* массив MAS запомнен */
.....
EOF
```

Во втором операторе IF используется «внутренний» оператор присваивания, в котором вместо знака « $=$ » пишут знак « $:=$ », чтобы отличать присваивание от сравнения на равенство.

Здесь процедура обработки прерываний выполняется только в том случае, если возникает прерывание типа 3 и прерывания не замаскированы.

В начале выполнения основной программы оператором ENABLE разрешается обработка всех прерываний, в том числе и прерываний типа 3. В процессе выполнения основной программы при каждом возникновении прерываний типа 3 управление передается процедуре PRER, одновременно маскируются все прерывания и запоминается состояние процессора. В процедуре PRER счетчик N увеличивает свое значение на 1. Далее проверяется степень заполнения массива MAS. Если массив заполнен целиком ($N \geq 100$), то управление передается на метку OVERFLOW и запомненное состояние процессора не восстанавливается.

Если массив MAS еще не заполнен, то анализируется значение величины, прочитанной из порта 5. Если эта величина есть буква A, то управление немедленно передается прерванному процессу. При этом автоматически восстанавливается состояние процессора и снимается запрет на обработку прерываний. Аналогичные действия происходят при нормальном (через END) завершении процедуры обработки прерываний PRER после пересылки принятой величины на следующее место массива MAS.

Таким образом механизм прерываний блокируется при появлении прерываний и автоматически разблокируется при выходе из процедуры обработки прерываний (через RETURN или через END). Запрет на обработку прерываний можно снять в самой процедуре обработки прерываний с помощью оператора ENABLE. Однако следует с осторожностью относиться к разблокированию прерываний в этой процедуре. Запрещается дважды одновременно обращаться к одной и той же процедуре обработки прерываний, по-

сколько она имеет только одну область данных. Наиболее рациональный метод состоит в блокировании прерываний на весь период обработки прерывания, что выполняется автоматически.

Процедуры обработки прерываний могут содержать обычные внутренние процедуры. После вызова эти процедуры передают управление в точку вызова той процедуры обработки прерываний, в которой они объявлены. Состояние процессора в момент прерываний сохраняется только в том случае, если переход на верхний уровень в процедуре осуществляется с помощью оператора RETURN.

Процедуры обработки прерываний могут вызываться обычным образом прямо из внешнего блока программы или из другой процедуры. Однако следует помнить о том, что всегда нужно снимать блокировку прерываний на выходе из процедуры обработки, даже если эта процедура была вызвана не внешним прерыванием.



ГЛАВА 3

ПРИМЕНЕНИЕ ЯЗЫКА ПЛ/М ДЛЯ ПРОГРАММИРОВАНИЯ МИКРОПРОЦЕССОРНЫХ СИСТЕМ

3.1. УПРАВЛЯЮЩИЕ ПРОГРАММЫ МИКРОПРОЦЕССОРНЫХ СИСТЕМ

Управляющие программы — мониторы обеспечивают эффективное использование микропроцессорных систем. Без этих программ не может работать даже простейшая микропроцессорная система, так как программа-монитор осуществляет запуск системы и управляет ее работой. В микропроцессорных системах управляющие программы располагаются в ЗУ, занимая от нескольких сотен байт до 2 Кбайт памяти [7].

Язык ПЛ/М позволяет создавать управляющие программы достаточно быстро, за несколько дней, а главное — легко вносить в них изменения.

Ниже рассматривается пример реализации некоторых функций управляющей программы для простейшей микропроцессорной системы, причем модульная структура программы позволяет легко изменять состав функций, а также адаптировать сами функции к различным внешним устройствам. На практике используются более сложные управляющие программы, чем рассматриваемая нами, такие, например, как мониторы реального времени для многозадачных режимов [6]. Эти мониторы решают задачи управления терминалами, распределения оперативной памяти, сопряжения с внешними устройствами, поддержания возможности работы с файлами на гибком магнитном диске и, наконец, задачи отладки, к которым можно отнести рассматриваемый ниже пример. Программы-отладчики выполняют следующие функции:

ввод информации с клавиатуры по любому адресу;
 чтение или перезапись содержимого любой ячейки;
 чтение содержимого всех внутренних регистров МП
 и изменение содержащейся в них информации;
 выполнение программ с остановами в любой точке;
 программирование постоянных ЗУ;
 запись и считывание информации на магнитную ленту, тестирование узлов системы, управление работой трансляторов и т. д.

Из перечисленных функций видно, что программа-монитор выполняет функции, близкие к функциям простейшей операционной системы.

Перед началом описания программы-монитора (рис. 3.1) определим основные программные блоки и для удобства пользования присвоим им имена. Программа MONITOR включает в свой состав подпрограммы INITIALE (начало), BRANCH (блок выбора операций), подпрограммы функций D,I,M,S,R,G и X, сервис-

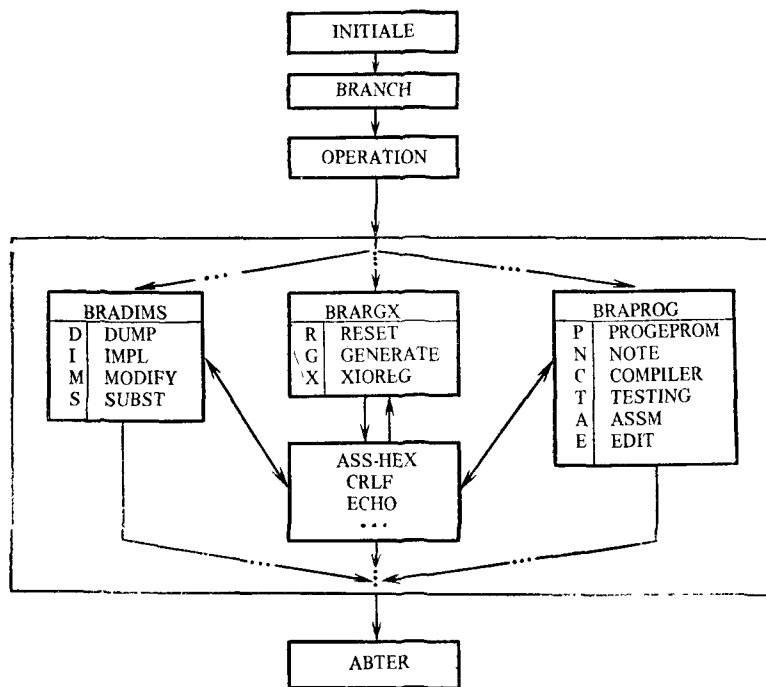


Рис. 3.1. Структурная схема программы-монитора

ные подпрограммы PROGEPPROM, TESTING и т. д. Кроме того, используются программы преобразования и управления устройствами отображения: ASC@HEX, CRLF, ECHO и т. д. Заканчивает программу блок аварийного завершения ABTER.

Рассмотрим функции отдельных блоков. Блок INITIALE выполняет следующие действия:

обрабатывает сигнал RESET, поступающий с клавиатуры;

осуществляет установку в нуль счетчика команд PC;

инициирует интерфейс клавиатуры и дисплея;

сохраняет содержимое всех регистров МП, которое они имели перед входом в программу MONITOR;

выдает на индикатор заголовок программы, осуществляет перевод строки и высвечивает в начале строки указатель-курсор, которым является знак «*».

Блок BRANCH контролирует правильность принятого управляющего сигнала и определяет тип операции, заданной с клавиатуры.

Блок BRADIMS выдает содержимое памяти D(DUMP), осуществляет ввод информации I(IMPL), производит копирование памяти M(MODIFY) и изменяет содержимое памяти S(SUBST).

Другая группа операций (BRARGX) определяет начало или продолжение работы R(RESET), выполнение программы G(GENERATE), просмотр и изменение содержимого регистров X(EXAMENEG'S или XIOREG).

Третья группа операций BRAPROG содержит программу PROGEPPROM, осуществляющую запись информации в перепрограммируемые ПЗУ, например типа КР573РФ1 с ультрафиолетовым стиранием, программы NOTE и COMPILER, выполняющие запись и считывание информации с кассетного магнитофона, программы TESTING, ASSM, EDIT, проверяющие работоспособность микропроцессорных устройств и использование транслятора с языка ассемблера и редактора текста.

Кроме того, в программе MONITOR предусмотрен целый ряд подпрограмм преобразования ASC@HEX и HEX@ASC, вывода информации на дисплей ECHO, CRLF, хранения содержимого внутренних регистров МП SWAP, RET и т. д. Все они пишутся на языке ассемблера с учетом аппаратного состава микропроцессорной системы. (В языке ПЛ/М, ориентированном на микро-ЭВМ СМ-1800, можно обращаться к подпрограммам, написанным на ассемблере.)

Рассмотрим первый блок (обозначен буквой Q) мониторной программы INITIALE, оформленный в виде процедуры прерывания, с параметром, соответствующим уровню прерывания, определяемому конкретной аппаратурной реализацией микропроцессорной системы.

```
INITIALE: PROCEDURE INTERRUPT Q;
          CALL SWAP,
          OUTPUT(0FBH)=11001111B; /* 0CFH */
          OUTPUT(0FBH)=00100111B; /* 27H */
          DECLARE NAME LITERALLY 'MONITOR';
          DECLARE I BYTE;
          DO I=0 TO 6;
            OUTPUT(0F1H)=NAME(I);
          END;
          CALL CRLF;
          DECLARE CURSOR LITERALLY '*' PUBLIC;
          OUTPUT(0F1H)=CURSOR;
          CALL RET;
END INITIALE;
```

Процедура SWAP передает содержимое регистров МП в область памяти, определяемую массивом MEMORY, который представляет собой область свободной памяти, оставшуюся после трансляции исходного модуля. В ту же область помещается согласно абсолютным адресам содержимое регистров F, A, BC, DE, HL, SP и PS, что в дальнейшем позволит обращаться по этим адресам из подпрограмм. Передача содержимого регистров в память и их обратное восстановление (процедура RET) обеспечивают «прозрачность» монитора, т. е. устраняют его влияние на информацию, хранящуюся в микропроцессорной системе до обращения к программе MONITOR.

Операторы вывода иницируют интерфейс клавиатуры, выполненный на БИС BB51, через порт 0FBH, выделенный для связи МП с клавиатурой. Коды 0CFH и 27H, задающие режим работы BB51, являются для форматов MODE INSTRUCTION и COMMAND INSTRUCTION управляющими словами. Они определяют режим работы, скорость передачи, число передаваемых символов и т. д. [1].

Через порт 0F1H на внешнее устройство отображения информации выводится название программы. Его можно легко изменить, заменив в операторе описания значение MONITOR, присваиваемое NAME, на любое другое и изменив число обращений I к порту 0F1H. Через этот же порт выводится курсор. Перевод строки

и переход в ее начало обеспечивается процедурой CRLF, учитывающей конкретное устройство отображения.

Следующий блок программы BRANCH осуществляет прием и распознавание управляющих сигналов и передает управление подпрограммам, реализующим отдельные операции монитора. Если введен неверный символ, управление передается блоку анализа аварийного завершения ABTER (ABNORMAL TERMINATION). Характер ошибки определяется путем введения признаков, передающих в блок ABTER информацию о месте и характере ошибок.

```
BRANCH: DO;
          CALL CRLF;
          CALL OPERATION;
          IF M1=0 THEN CALL ABTER(M1);
        END BRANCH;
```

Выполнение блока BRANCH начинается с перевода строки, что сигнализирует о программном входе в него. Далее управление передается процедуре выбора выполняемой операции OPERATION. После этого анализируется состояние признака M1, и если его значение равно нулю, то управление переходит к блоку ABTER.

Рассмотрим процедуру OPERATION.

```
OPERATION: PROCEDURE INTERRUPT Q;
          DECLARE (M1, ALPH ⊙ BET) BYTE EXTERNAL;
          ALPH ⊙ BET=INPUT(0FAH);
          L1: IF ALPH ⊙ BET <= 39H THEN
            DO;
              DECLARE QUERY LITERALLY '?';
              OUTPUT(0F1H)=QUERY;
              OUTPUT(0FAH)=1;
              GOTO L1;
            END;
          CALL CRLF; CALL ECHO;
          DECLARE (
            A, C, D, E, G, I, M, N, P, R, S, T, X)
            BYTE DATA (
              41H, 43H, 44H, 45H, 47H, 49H, 4DH, 4EH, 50H, 52H, 53H, 54H, 58H);
          DO CASE ALPH ⊙ BET;
            ; ; ; ; /* ВСЕГО 65 СИМВОЛОВ; */ ; ; ; ;
            DO; CALL ASSM; M1=1; END; ;
            DO; CALL COMPILER; M1=2; END;
            DO; CALL DAMP; M1=3; END;
            DO; CALL EDIT; M1=4; END;
            .....
            DO; CALL XIOREG; M1=13; END;
          END;
          RETURN M1;
        END OPERATION;
```

Процедура OPERATION обеспечивает через порт OFAH прием входной информации с клавиатуры, причем это должен быть один из разрешенных символов A,C,D,...,X. Оператор с меткой L1 контролирует правильность ввода. В том случае, когда нажата клавиша цифры (код от 30H до 39H), на индикаторе высвечивается знак «?» (QUERY), подсказывающий оператору о том, что нажата не та клавиша. Порт OFAH служит для ввода сигнала многократного опроса клавиатуры. Оператор GO TO организует цикл опроса. При выполнении условия >39H осуществляется перевод строки.

После приема буквы ее необходимо идентифицировать. Оператор DECLARE размещает массив однобайтных слов, причем каждой букве соответствует определенный код. Если необходимо использовать буквы русского алфавита, применяется код КОИ-7 (см. приложение).

Следующий оператор блока DO CASE передает управление подпрограммам, выполняющим операции монитора. Так как в памяти хранятся значения кода ASCII, то для проверки, например, условия ALPHABET=A необходимо, чтобы число пропускаемых строк в операторе DO CASE составило 65. Этот оператор позволяет проверить заданное условие без преобразования, выполняемого процедурой ASCOHEX. Если произойдет переход к подпрограмме ASSM, то после выхода программы из блока DO CASE признак M1 примет значение, равное 1. Этот параметр будет использован в программе блока ABTER.

После определения выполняемой операции управление из блока BRANCH передается в одну из подпрограмм для ее реализации. Приведем перечень операций, которые осуществляются при выборе управляющего сигнала I (IMPL — ввод). После нажатия клавиши 1 необходимо набрать на клавиатуре адрес ЗУ в шестнадцатеричном коде, но без признака H. Этот адрес определяет ячейку памяти, в которую будет вводиться информация. Этот же адрес индицируется на устройстве отображения, после чего осуществляется побайтный ввод кодов. Вводимые кодовые слова разделяются символами пробела, запятой или возвратом каретки.

Если необходимо ввести несколько байтов подряд, то после набора каждого байта необходимо нажимать клавишу I, тогда каждый последующий байт будет за-

поминаться в последовательно расположенных ячейках памяти, начиная с заданного адреса.

Нельзя разделять между собой цифры одного байта. В случае ошибочно введенного символа, отличного от 0—F, из блока ABTER выдается сообщение об ошибке, а ячейка, куда должен быть записан этот байт, не изменяет своего состояния. Выход из режима IMPL осуществляется нажатием клавиши CCC (смена режима).

Ниже представлена программа блока IMPL.

```
IMPL PROCEDURE INTERRUPT Q;
  DECLARE (CIPHER,NBYT) BYTE;
  (M1,G,T) BYTE;
  ADDOON ADDRESS;
  ADDOON=GENOADDR(CIPHER);
  N=0;
  DECLARE NBYT(100) BYTE AT ( ADDOON+N),
  /* NBYT — МАССИВ ДЛЯ ХРАНЕНИЯ ВВЕДЕННЫХ БАЙТОВ */
L2: NBYT(ADD OON+N)=KEYBOARD(CIPHER);
  CALL CCC(T);
  IF T=0 THEN
    DO;
      CALL FOLLOW(G);
      N=N+1;
      GOTO L2;
    END;
  END IMPL;
```

В операторе, объявляющем переменные, величина ADDOON представляется в формате ADDRESS. Она определяется процедурой-функцией GENOADDR с входным параметром CIPHER и выходным ADDOON.

```
GENOADDR PROCEDURE (CIPHER) ADDRESS;
  DECLARE (CIPHER,HBYT,LBYT,M1,G) BYTE,
  ADDOON ADDRESS;
  DO; DOUBLE (HBYT=KEYBOARD(CIPHER));
  IF M1=0 THEN HBYT=SHL(HBYT,8);
  CALL FOLLOW(G);
  LBYT=KEYBOARD(CIPHER);
  ADDOON=HBYT+LBYT;
  END;
  RETURN ADDOON;
END GENOADDR;
```

Встроенная функция DOUBLE позволяет расширить HBYT до двухбайтовой величины, приписывая спереди восемь нулей. Затем после проверки правильности считывания цифры (при этом значение признака M1=0)

переменная CIPHER сдвигается влево на 8 разрядов и становится старшим байтом НВУТ. В результате формируется адрес, который в МП КР580 занимает 16 двоичных разрядов, его максимальное значение 0FFFFH.

Подпрограмма FOLLOW(G) определяет момент поступления следующего байта информации, о чем свидетельствует сигнал, поступающий в МП через порт 0F9H. Выход из этой подпрограммы происходит только после приема с клавиатуры сигнала о введении очередного байта.

```
FOLLOW: PROCEDURE(G) BYTE;
  DECLARE G BYTE, I BYTE;
L3:    G=INPUT(0F9H);
      IF G=0 THEN
        DO I = 1 TO 40;
          CALL TIME(250);
        END;
        GOTO L3;
      END FOLLOW;
```

В процедуре FOLLOW имеется блок односекундной задержки опроса клавиатуры, созданный встроенной процедурой TIME.

Засылка в память принятых в процедуре IMPL байтов осуществляется в выделенную область памяти, определяемую как однобайтный массив с начальным адресом ADD00N с помощью описателя AT. Действие AT заключается в размещении переменной по адресу, заданному в выражении, заключенному в скобки.

После приема очередного байта проверяется признак CCC(T), сообщающий о переходе к другому режиму. Этот признак позволяет в любой момент прекратить ввод информации в память.

В процедуре IMPL используется процедура KEYBOARD, с ее помощью можно опрашивать клавиатуру микропроцессорной системы, идентифицировать принимаемый символ, отображать его на экране и при поступлении ошибочного символа передавать управление процедуре ABTER.

Процедуры опроса клавиатуры могут включать в свой состав блоки, устраняющие ненадежность контактов, что упрощает аппаратную часть микропроцессорной системы и снижает ее стоимость.

```
KEYBOARD: PROCEDURE(A) BYTE;
  DECLARE (A,G,M1) BYTE;
  CALL FOLLOW(G);
  DO; A=INPUT(0FAH);
    IF (((A=30H) AND (A<=39H) OR
      (A=41H) AND (A<46H) THEN
      DO; CALL ASC0HEX(A);
        CALL ECHO;
        M1=OFFH;
        IF ((A=02CH) OR
          (A=020H) OR
          (A=0AH) THEN
          DO;
            CALL ECHO;
            M1=14;
          END;
        ELSE CALL ABTER(M1);
      END;
    END;
  RETURN A;
END KEYBOARD;
```

Процедура-функция KEYBOARD содержит в качестве входного параметра переменную A, которая затем передается оператором RETURN в формате H. Процедура идентифицирует входные символы не только как шестнадцатеричные цифры, но и распознает служебные символы: запятые (код 02CH), пробелы (код 020H) и символ перевода строки ПС (0AH). Эти символы служат для разделения вводимых байтов, и они не должны оказывать влияния на принимаемую информацию.

Операцией монитора S(SUBSTITUTION — исправление) выводится на индикатор код, хранимый по заданному адресу, и при желании изменяет его. Изображение кода на индикаторе сопровождается символом «_», он указывает оператору, что при вводе нового значения кода старое будет утеряно.

Программный блок SUBST использует те же процедуры, что и подпрограмма INITIAL.

```
SUBST: PROCEDURE;
  DECLARE NEWBYT(100) BYTE;
  DECLARE VALUE BYTE,
    INCHAR ADDRESS;
  DECLARE DACH LITERALLY '-';
  VALUE=
  NEWBYT (GEN0 ADDR(INCHAR));
  CALL HEX0 ASC(VALUE)
  CALL ECHO; /* ВЫВОД VALUE */
  CALL ECHO; /* ВЫВОД DACH */
  CALL CCC(T);
END SUBST;
```

Операция D (MEMORY DUMP — выдать содержимое памяти) сопровождается начальным и конечным адресами, разделяемыми пробелом, запятой или символом ПС. Формат информации, выдаваемой на индикатор, может быть различным; в нашем случае остановимся на следующем: в начале строки индицируется первый и второй адрес, затем после двух пробелов выводится содержимое ячеек памяти. Операция не выполняется в том случае, если неверно введены адреса, за первым адресом следует символ, отличный от пробела или запятой, и второй адрес меньше первого.

Рассмотрим подпрограмму DUMP.

```
DUMP: PROCEDURE;
  ADDR1= GEN⊙ADR(INCHAR);
  CALL IMAGE;
  ADDR2= GEN ADR(INCHAR);
  CALL IMAGE;
  IF ADDR2 < ADDR1 THEN
    DO; M1=15, CALL ABTER(M1); END;
  L4:
    INBYT=NBYT(ADDR1);
    CALL IMAGE;
    CALL CCC(T);
    ADDR1=ADDR1+1;
  IF ADDR1<=ADDR2 THEN GOTO L4;
    ELSE CALL CCC(T);
  END;
END DUMP;
```

Переменная INCHAR обозначает входной код, который передается в процедуру-функцию.

Процедура IMAGE объединяет все процедуры, необходимые для получения изображения, в одну, что делает программу компактнее и облегчает ее чтение.

Так как в цикл процедуры вставлена подпрограмма CCC(T), то при нажатии клавиши смены режима CCC выполнение программы будет прекращено.

Пять первых операторов блока DUMP можно объединить в процедуру ADDR⊙COMPAR, у которой входными параметрами являются ADDR1 и ADDR2.

Операция M (MEMORY MODIFY — копирование или изменение места расположения) имеет вид M A1, A2, A3, где под A_n подразумеваются адреса. Содержимое ячеек памяти начиная с A1 будет перенесено в область памяти с адреса A3. Величина переносимого массива определяется величиной (A2—A1). Если второй адрес

меньше, чем первый, то операция выполняться не будет. Если A3 находится в области между A1 и A2 или A3 < A1, т. е. вторая область перекрывает первую, то в этом случае необходимо начать перезапись с конца первого массива. Определим последний адрес нового массива или области ЗУ, куда будет переписываться информация:

$$A4 = A3 + (A2 - A1)$$

Запись информации в обратном порядке будет осуществляться так: число из ячейки с адресом A2 записывается в ячейку с адресом A4, а из ячейки A2—1 — в ячейку A4—1 и т. д. Представим программную реализацию операции.

```
MODIFY: PROCEDURE;
  CALL ADDR⊙COMPAR(ADDR1, ADDR2);
  ADDR3=GEN⊙ADR(CODE);
  STAG⊙NUM=ADDR2-ADDR1;
  IF ADDR3 > ADDR2 THEN
    CALL MOVE (STAG⊙NUM+1, ADDR1, ADDR3);
  ELSE DO;
    DO;
      ADDR4=ADDR3+(ADDR2-ADDR1);
      I=0;
    L10: MASSIF2(ADDR4-1)=MASSIF1(ADDR2-1);
      I=I+1;
    CALL CCC(T);
    IF I < STAG⊙NUM THEN GOTO L10;
  END;
  CALL CCC(T);
  END MODIFY;
/* СЛУЧАЙ ADDR3<ADDR1 НЕ РАССМАТРИВАЕТСЯ */
```

Подпрограмма сравнения адресов ADDR⊙COMPAR проверяет соотношение между первым и вторым адресами оператора. Затем выполняется процедура-функция GEN⊙ADR, она определяет третий адрес ADDR3. Заметим, что вход в подпрограмму осуществляется путем передачи ей параметра CODE, т. е. новой переменной для входного символа, поступающего с клавиатуры.

Вычисляемая величина STAG⊙NUM задает размер массива, перемещаемого в ЗУ. После этого проверяется условие A3 > A2, которое определяет взаимное расположение двух массивов. Если они не перекрываются, то для выполнения операции MODIFY достаточно использовать встроенную процедуру MOVE с тремя параметрами: числом переносимых байтов, адресом первого сдвигаемого байта и адресом памяти, куда должен быть

перенесен этот байт. Если же области перекрываются, то необходимо определить некоторый вспомогательный адрес ADDR4, который будет первым адресом записи и последним адресом получаемого массива. Адреса определяются путем вычитания единицы из величины адреса. После выполнения операции монитора MODIFY в цикл блока встраивается подпрограмма проверки CCC(T), которая прекращает перенос массива при изменении режима работы.

Операция GENERATE (переход и передача управления в ячейку памяти) предназначена для вхождения в программу в указанной точке. Операция выполняется путем засылки адреса в счетчик команд, при этом состояние всех регистров и признаков МП остается тем же, что и до нажатия клавиши G.

Операция R(RESET) возобновляет выполнение команды с адреса той ячейки, где была установлена точка прерывания. Например, при использовании процедуры CCC(T) передача управления происходит с помощью операции R.

Операция X(XIOREG — проверка содержимого регистров) контролирует состояние всех признаков и регистров F, A, BC, DE, HL, SP и PC.

Команды G, R и X тесно взаимосвязаны с внутренними регистрами МП, поэтому они могут быть выполнены с помощью команд МП КР580, а программы, реализующие эти операции и написанные на языке ассемблера, могут быть оформлены в виде процедур.

Если подпрограмму пересылки содержимого внутренних регистров и признаков МП оформить в виде процедуры EXAMENEOREG, то их содержимое будет храниться в памяти. Предположим, что имена переменных для хранения содержимого регистров будут RF, RA, RBC, RDE, RHL, RSP и RPC, признаки имеют значения: F=0, 1, 0, 1, 0, аккумулятор A=5F, а остальные пары регистров BC=1033, DE=76AA, HL=0110, SP=3456, PC=00F1. Тогда подпрограмма должна реализовать следующую последовательность действий. При нажатии клавиши X подпрограмма EXAMENEOREG создает массив, где каждый элемент соответствует содержимому внутренних регистров в порядке, указанном выше. Такой массив создается оператором

```
DECLARE (RF, RA) BYTE,
        (RBC, RDE, RHL, RSP, RPC) ADDRESS;
```

Содержимое регистров выводится непосредственно после начала выполнения операции

```
S Z AC P C A BC DE HL SP PC
0 1 0 1 0 5F 1033 76AA 0110 3456 00F1
```

Первая строка выводимого сообщения может быть записана следующим образом:

```
DECLARE BL@HEADING(*) BYTE INITIAL(
20H, 20H, 53H, 20H, 5AH, 20H, 41H, 43H, 20H, 50H, 20H, 43H,
20H, 20H, 41H, 20H, 20H, 20H, 42H, 43H, 20H, 20H, 20H, 44H,
45H, 20H, 20H, 20H, 48H, 4CH, 20H, 20H, 20H, 53H, 50H, 20H,
20H, 20H, 50H, 43H);
```

Длина строки будет равняться 41 символу, содержимое регистров будет вводиться после возврата каретки и перевода строки. Каждому значению строго соответствует место, определяемое числом пробелов, отсчитываемых от начала строки. Например, информация о содержимом признака Z будет размещаться после 5 пробелов, а о содержимом счетчика адреса PC — после 38. Эти значения хранятся в массиве

```
DECLARE DATUM (11) BYTE INITIAL(
3,5,7,10,12,15,18,23,28,33,38);
```

При внесении исправлений к определенному признаку или регистру можно обращаться следующим образом:

```
LOOP:      CALL SPACE;
           DAT=DATUM(Z)-1;
           IF DAT<>0 GOTO LOOP;
           ELSE DO;
               Z=30H;
               CALL ECHO;
           END;
```

В случае использования малоразрядного индикатора возможен такой вывод информации:

```
S Z AC P C 0 1 0 1 0
A 5F
B C 1033
```

Этот вывод потребует изменений в программе, но они очень просты и могут быть легко внесены.

Следующая группа операций из блока BRAPROG — P.N.C.T.A.E позволяет вызывать подпрограммы obsługi

живания микропроцессорной системы. Все программы ориентированы на определенную аппаратную реализацию. Остановимся на одной из них — операции P (PROGEPROM — программирование ПЗУ), состоящей из следующих подпрограмм:

чтение содержимого «стертого» ПЗУ. Если при чтении обнаружится, что хотя бы одна из ячеек заполнена, то происходит останов программы с передачей управления блоку ABTER;

запись или программирование ПЗУ. Запись осуществляется блоками по 1024 байта, повторяемыми циклически несколько сотен раз. При выполнении этой части программы необходимо предусмотреть возможность изменения длительности импульса записи;

чтение вновь записанного содержимого ПЗУ и его сравнение с информацией, хранящейся в ОЗУ, откуда была произведена перезапись.

Для выполнения первой подпрограммы операции P используется подпрограмма MEMORY DUMP, дополненная операцией сравнения содержимого каждой ячейки памяти с нулем:

```
IF VALUE <> 0 THEN
DO;
    M1=16;
    CALL ABTER(M1);
END;
```

Вторая подпрограмма может быть построена из подпрограммы MODIFY, в которой значение STAG@NUM нужно определить исходя из адресного пространства данной микропроцессорной системы. В частности, при STAG@NUM=07FFH циклическая запись в ПЗУ осуществляется следующим образом:

```
STAG@NUM=07FFH,
CALL MOVE (STAG@NUM,ADDR1,
    ADDR3);
```

В третьей части программы P сравниваются два массива, один из которых хранится в ОЗУ, а второй — в ПЗУ:

```
L15: ADD@ON@RAM=ADD@ON@RAM+1;
    ADD@ON@ROM=ADD@ON@ROM+1;
    CHARAM=MASSIF2(ADD@ON@RAM);
    CHAROM=MASSIF2(ADD@ON@ROM);
    IF CHARAM <> CHAROM THEN
        CALL ABTER;
    IF ADD@ON@RAM < ADD@ON@RAM@MAX THEN
        GOTO L15;
```

В зависимости от типа ПЗУ иногда требуется изменить длительность импульса записи. Это происходит следующим образом. На выход одного из портов, например 0F0H, подается управляющий сигнал из МП. Далее с использованием подпрограммы TIME организуется таймер-счетчик. После достижения таймером определенного значения на другой выходной порт, например 0FFH, подается сигнал с выхода таймера. Эти два сигнала используются для управления триггером — импульсной схемой, на выходе которой образуется импульс записи. Время задержки второго сигнала легко программируется:

```
OUTPUT(0F0H)=1;
    CALL TIME(K);
OUTPUT(0FFH)=1;
```

Здесь параметр K определяет время задержки как величину, кратную 100 мкс. Например, при тактовой частоте МП, равной 2 МГц, TIME(5) соответствует задержке 0,5 мс.

При разработке программы MONITOR используется целый ряд подпрограмм типа ASC@HEX, ECHO, CRLF и т. д., которые составляются на языке ассемблера. Обращение из управляющей программы BRANCH к подпрограммам PROGEPROM, NOTE, COMPILER, TESTING, ASSM и EDIT осуществляется обычным способом: как к отдельным процедурам.

Для составления полной управляющей программы необходимо знать аппаратный состав микропроцессорной системы, типы используемых БИС для организации интерфейса и т. д. Рассмотренные выше блоки программ хотя и не учитывают аппаратный состав микропроцессорных систем, но дают представление о их структуре и использовании определенных операторов языка ПЛ/М.

Все программы модуля MONITOR имеют выход в блок аварийного завершения ABTER, куда передается управление при обнаружении ошибок.

3.2. ПРОГРАММИРОВАНИЕ УПРАВЛЯЮЩИХ МИКРОПРОЦЕССОРНЫХ СИСТЕМ

Управляющие микропроцессорные системы широко применяются для построения АСУ различного назначения, особенно для автоматизированных систем техни-

ческого обслуживания оборудования. Это оборудование, как правило, размещается в конструкциях стоечного типа, причем в каждой стойке имеется панель управления с контрольными точками, на которые подаются контролируемые напряжения. Алгоритм работы такой системы сводится к следующим действиям: по сигналам от ЭВМ коммутатор «выбирает» контрольные точки (КТ), сигналы с этих точек подаются на аналого-цифровой преобразователь (АЦП). Далее коды поступают через интерфейс в ЗУ ЭВМ, где фиксируются измеренные значения. Они сравниваются с требуемыми значениями, и в зависимости от величины и знака отклонения система «принимает» решения, которые по цепи обратной связи вводятся в контролирующую аппаратуру в виде управляющих сигналов. Этот алгоритм лежит в основе практически всех методов технической диагностики и обслуживания.

Управление работой коммутатора и АЦП, а также выбор КТ осуществляются на основании протокола обмена. Протокол представляет собой программу, обеспечивающую многорежимную работу микропроцессорной системы. Задание режимов работы и управление устройствами производятся путем задания управляющих слов в определенных полях протокола. Набор полей протокола обмена можно представить следующим образом. Поле управления работой коммутатора FIELD@EX-CHANGE может содержать три управляющих слова:

- адрес порта, по которому иницируется коммутатор;
- слово управления работой коммутатора;

- адрес порта, по которому микропроцессорной системой принимаются сигналы подтверждения готовности коммутатора к работе и текущий адрес КТ.

Поле управления работой АЦП FIELD@ADC состоит из двух управляющих слов:

- адреса порта, по которому ведется управление работой АЦП;

- адреса порта, через который поступают 10—12-рядные значения от АЦП, причем считается, что преобразователь имеет внутренний регистр для хранения этого кода.

Поле управления коэффициентом усиления FIELD@AMPLIFIER располагает одним управляющим словом.

Поле управления приемом данных в память микропроцессорной системы FIELD@RAM включает в свой

состав два слова: адрес начала массива и размерность массива.

Поле управления текущим значением адреса коммутатора FIELD@PC обозначается одним словом.

Ниже приводится фрагмент программы, определяющей поля протокола обмена.

```

DECLARE PROTOCOL@ENTER    STRUCTURE

( FIELD@EX@ADR             BYTE,
  FIELD@EX@CON             BYTE,
  FIELD@EX@KT              ADDRESS,
  FIELD@ADC@ARD            BYTE,
  FIELD@ADC@DATA           BYTE,
  FIELD@AMPLIF             BYTE,
  FIELD@RAM@ADR            ADDRESS,
  FIELD@RAM@NUM            BYTE,
  FIELD@PC                 BYTE);

DECLARE ARREY (100) ADDRESS;
INITIAL@EX: PROCEDURE (F1,F2,F3) BYTE;
  DECLARE (F1,F2) BYTE, F3 ADDRESS;
END INITIAL@EX;
CALL INITIAL@EX (FIELD@EX@ADR,
  FIELD@EX@CON, FIELD@EX@KT);

```

Элементами структуры являются переменные, которые задают поля протокола. Их размерность соответствует выполняемым функциям.

После задания структуры объявляется 100-элементный массив ARREY(100), отводимый для приема данных с КТ.

Процедура INITIAL@EX указывается формально. Обращение к ней — CALL INITIAL@EX — дает возможность использовать элементы структуры, принимающие различные значения. Изменение содержимого полей FIELD@EX@ADR, FIELD@EX@CON и FIELD@EX@KT, расположение которых в памяти становится точно известным после трансляции, обеспечивает различные режимы работы коммутатора. На практике применяются такие режимы считывания информации:

- с заданной КТ определенное число раз и с постоянным коэффициентом усиления;

- с многих КТ, осуществляемое в порядке возрастания адресов, с постоянными или переменными коэффициентами по каждой точке;

- с КТ по программе, в которой указывается порядок опроса, число измерений и величина коэффициента усиления по каждой точке.

Программная реализация каждого из режимов достаточно проста. Выбор заданной точки осуществляется либо обращением к порту с определенным адресом, отождествляемым с КТ:

```
КТ01 = INPUT(07H);
```

либо порт ввода остается постоянным, а номер КТ определяется содержимым поля FIELD0PC.

Количество измерений по каждой точке устанавливается путем циклического опроса либо с помощью DO WHILE.

В случае, если измерения необходимо производить до тех пор, пока напряжение в КТ не станет равным нулю, можно употреблять следующую конструкцию:

```
DO WHILE (INPUT(01H) AND 0);
```

Увеличение принятого значения в заданное число раз осуществляется сдвигом влево:

```
AMPL0КТ096 = SHL(КТ096,A);
```

Здесь А — число разрядов, на которое производится сдвиг, причем сдвиг на 1, 2, 3 ... разрядов соответствует увеличению значения в 2, 4, 8 ... раз.

Информация, размещенная в массиве ARREA(100), может быть подвергнута дальнейшей обработке. В результате определяется характер изменения данных по каждой точке — возрастают ли эти значения, убывают, остаются неизменными, а также устанавливаются их минимальные и максимальные значения. Кроме того, путем цифровой обработки можно усреднить измеренные значения или уменьшить объем полученных данных без потерь информации. Например, для вычисления среднеквадратичных значений операцию извлечения квадратного корня можно выполнять приближенно следующим образом:

```
DO SIGN=1 TO DS;
IF SIGN*SIGN >= DS THEN...
```

Характер изменения данных в простейшем случае определяется путем вычитания последующего значения, из предыдущего:

```
IF(ARREY(I+1) - ARREY(I)) >= 0 THEN...
```

После обработки принятой информации и ее анализа микропроцессорная система переходит к управлению

работой оборудования. Пример программы управления приводится ниже.

```
DECLARE (START,STOP) BYTE;
DECLARE (IN0DATA,MTIME) BYTE;
DECLARE (P1,P2,P3,S1,S2,S3) BYTE;

IN DATA=INPUT(0AFH);
IF (IN0DATA AND 00000001B)=1
  THEN START=1;
  ELSE STOP=1;
IF (IN DATA AND 00000010B)=0
  THEN S1=0;
  ELSE DO;
      S2=1;
      S3=1;
  END;

P1=(STARTX OR P1) AND (STOP AND P2)
P3=(NOT S2) AND (NOT S3) AND
(MTIME >=100);
```

В приведенной программе используются управляющие сигналы с именами START и STOP, входной сигнал IN0DATA, поступающий из порта 0AFH, таймер MTIME, отсчитывающий заданное время, например 100 единиц, реле P1, P2 и P3, и переключатели (SWITCH) S1, S2, S3.

В результате опроса порта 0AFH принимается следующее решение: если младший разряд равен 1, то START=1, а если 0, то STOP=1. В случае если второй разряд входного сигнала IN0DATA=0, то S1=0, в противном случае S2 и S3 станут равными 1. В следующем операторе осуществляется управление реле P1, которое срабатывает в случае, если START≠P1, а STOP=P2. И, наконец, P3 сработает, если переключатели S2 и S3 находятся в нулевом положении, а в таймере хранится значение не менее 100.

В сложных системах управления, осуществляющих, например, прогнозирование технического состояния аппаратуры, данные анализируются оператором, для чего информация выводится на дисплей. В качестве примера рассмотрим случай, когда требуется оценить коэффициенты регрессии для последовательности измеренных значений в одном из КТ. Программа состоит из трех блоков:

вычисления оценки коэффициентов линейной регрессии $Y=A+BT$

$$A = [(\sum Y_i)(\sum i^2) - (\sum i) \cdot (\sum Y_i)]/D;$$

$$B = [N(\sum Y_i \cdot i) - (\sum i)(\sum Y_i)]/D;$$

$$D = N(\sum i^2) - (\sum i)^2;$$

подготовки информации к выходу;
вывода информации на печать.

В качестве печатающего устройства выбрано DZM-180, поставляемое с ЭВМ СМ-1800. Резидентный транслятор этой ЭВМ позволяет употреблять буквы русского алфавита (см. приложение).

Первые операторы программы резервируют память и описывают подпрограммы, так как в языке ПЛ/М можно использовать только подпрограммы, описываемые в качестве процедур до начала их выполнения.

```
ОЦЕНКА РЕГРЕССИИ DO;
  DECLARE ВЫХОД(200) BYTE;
  DECLARE (I,J,K КОД,P) ADDRESS;
  DECLARE F(50) ADDRESS;
  DECLARE (A,B,S1,S2,S3,S4 D) ADDRESS;
  DECLARE L BYTE;
  DECLARE N1(2) BYTE;

  DECLARE ЗАГОЛОВОК(*) BYTE INITIAL(
    20H, 20H, 20H, 20H, 20H,
    20H, 20H, 20H, 20H, 20H, 'ТАБЛИЦА ЗНАЧЕНИЙ',
    20H, 20H, 20H, 20H, 'T=0.1,2, ...',
    0DH, 0AH, 0AH, '');
  DECLARE Y(*) ADDRESS INITIAL(
    3 3, 3, 3, 3, . . . . . );
  DECLARE ТАБЛИЦА(*) BYTE INITIAL(30H, . . . , 39H, 41H, . . . , 46H);
  DECLARE N BYTE INITIAL(10);
  ПРЕОБР PROCEDURE;
    КОД=ТАБЛИЦА(КОД); RETURN;
  END ПРЕОБР;
  ВЫВОД PROCEDURE(P);
    DECLARE P BYTE;
    DO WHILE (INPUT(023H) AND 0)
      END;
    OUTPUT(022H)=P;
    RETURN;
  END ВЫВОД;
```

Оператор DECLARE резервирует память для массива Y(50), все элементы массива приняты равными трем, т. е. имеется в виду, что при 50 измерениях в одной контрольной точке все значения равнялись, например, 3В. Следует отметить, что при загрузке программы в память микроЭВМ первая группа операторов описаний только резервирует память в соответствии со спецификациями. Следующая группа операторов не только

резервирует память, но и загружает в память скалярные величины. Например, массив ЗАГОЛОВОК представляет собой последовательность символов КОИ-7, которые при выдаче образуют строку заголовка, а массив ТАБЛИЦА заполнен кодами шестнадцатеричных цифр.

Далее объявлены две процедуры ПРЕОБР и ВЫВОД, осуществляющие преобразование символов и подготовку вывода на печать, причем для связи с печатью используются порты 022H и 023H.

Следующий блок вычисляет коэффициенты регрессии $Y=A+BT$:

```
S1=0;
DO L=0 TO N-1;
  S1=S1+Y(L);
END;

S2=0;
DO I=1 TO N;
  S2=S2+I*I;
END;

S3=0;
DO I=1 TO N;
  S3=S3+I;
END;

S4=0;
DO L=0 TO N;
  I=L+1;
  S4=S4+Y(L)*I;
END;

D=N*S2-S3*S3;
A=(S1*S2-S3*S1)/D;
B=(N*S4-S3*S1)/D;
DO I=0 TO N-1;
  F(I)=A+B*(I+1);
END;
```

Здесь A — оценка свободного члена, B — коэффициент при T, N — длина ряда. Входом для данного блока является массив Y(50), а выходом — F(50). При длине ряда $N \geq 30$ происходит переполнение.

В блоке подготовки информации к выводу основной операцией является преобразование результатов в КОИ-7. Выходной массив F(1), с форматом ADDRESS, трансформируется в массив ВЫХОД путем преобразования полубайтов каждого символа в КОИ-7, после чего эти коды и образуют массив ВЫХОД, причем каждый код занимает теперь один байт. Отсюда размерность выходного массива достигает 200 байт.

```

ПОДГОТОВКА DO I=0 TO N-1;
  J=I*4+3;
  КОД=(I (I) AND 000FH);
  CALL ПРГОВР;
  ВЫХОД (J)-КОД;
  КОД=(I (I) AND 00F0H)/16;
  CALL ПРГОВР;
  ВЫХОД (J-1)=КОД;
  КОД=(I (I) AND 0F00H)/256;
  CALL ПРГОВР;
  ВЫХОД (J-2)=КОД;
  КОД=(I (I) AND 0F000H)/4096;
  CALL ПРГОВР;
  ВЫХОД (J-3)=КОД;
END;
КОД=(N AND 01 0H)/16;
CALL ПРГОВР;
N1 (0)=КОД;
КОД=(N AND 0FH);
CALL ПРГОВР;
N1 (1)=КОД;

```

Завершающий блок программы — вывод информации на печать:

```

ПЕЧАТЬ CALL ВЫВОД ('=') ;
        CALL ВЫВОД (N1 (0)) ;
        CALL ВЫВОД (N1 (1)) ;
        CALL ВЫВОД ('H') ;
        CALL ВЫВОД (20H) ;
        CALL ВЫВОД (20H) ;
        CALL ВЫВОД (20H) ;

```

Блок с меткой ПЕЧАТЬ выдает первую часть заголовка N=0AH..., а блок с именем ПЕЧАТЬ_0 выводит остальную часть заголовка, причем возврат каретки и дважды перевод строки (коды 0DH и 0AH, 0AH) объявлены в описании ЗАГОЛОВОК.

```

ПЕЧАТЬ_0 I=0;
DO WHILE ЗАГОЛОВОК (I) <> ' ';
  CALL ВЫВОД (ЗАГОЛОВОК (I));
  I=I+1;
END;
ПЕЧАТЬ_1: DO K=1 TO N/10,
           DO J=1 TO 10;
             I=J*4-3;

```

Блок P1 выводит четыре элемента из выходного массива ВЫХОД(I) и символ H, причем элементы мас-

сива выводятся построчно и в каждой строке — десять четырехразрядных шестнадцатеричных цифр.

```

P1:      CALL ВЫВОД (ВЫХОД (I-1)) ;
          CALL ВЫВОД (ВЫХОД (I)) ;
          CALL ВЫВОД (ВЫХОД (I+1)) ;
          CALL ВЫВОД (ВЫХОД (I+2)) ;
          CALL ВЫВОД ('H');

```

Печатающее устройство отличается ограниченным объемом буфера. Для устранения его переполнения вводится следующая группа операторов:

```

DO L=0 TO OFFH;
END;

```

Вместо нее можно указывать оператор TIME(K), также обеспечивающий временную задержку.

После каждого четырех цифр и символа H выводятся три пробела, для чего трижды используется подпрограмма ВЫВОД (20H). После вывода десяти значений осуществляются перевод строки ВЫВОД (0DH) и операция возврата каретки ВЫВОД (0AH). Так как для печатающего устройства время возврата каретки может быть значительным по сравнению со временем поступления информации в буфер, то необходимо вновь ввести временную задержку:

```

P2:      CALL ВЫВОД (20H);
          CALL ВЫВОД (20H);
          CALL ВЫВОД (20H);
        END;
          CALL ВЫВОД (0DH);
          CALL ВЫВОД (0AH);
        DO I=1 TO OFFFH;
          END;
        END; HALT;
END ОЦЕНКА РЕГРЕССИИ;

```

По окончании вывода оператор HALT останавливает реализацию программы. После ее выполнения на печать выдается следующий результат:

N=0AH	ТАБЛИЦА	ЗНАЧЕНИЙ	T=0,1,2,
000CH	000CH	000CH	000CH
N=14H	ТАБЛИЦА	ЗНАЧЕНИЙ	T=0,1,2,
0002H	0002H	0002H	0002H
0002H	0002H	0002H	0002H
N=1EH	ТАБЛИЦА	ЗНАЧЕНИЙ	T=0,1,2,
000BH	000BH	0010H	0011H 0012H

Рассмотрим две прикладные программы: построение генератора электрических сигналов и разработка цифрового компрессора.

Генератор электрических сигналов. В контрольно-измерительной технике одно из основных мест отводится генератору электрических сигналов различной формы. С развитием микропроцессорных систем подобные генераторы для среднечастотного диапазона стали создаваться программно. Для этого предназначены два метода: программный, заключающийся в вычислении значений для генерируемой функции, и табличный. Первый из них позволяет программным путем получать значения любых функций, имеющих аналитическую форму представления. К недостаткам этого метода можно отнести большое время реализации. Табличный метод предполагает запоминание и хранение ранее полученных расчетным путем значений функции в запоминающем устройстве. Время считывания хранящихся в памяти отсчетов зависит от аппаратного выполнения, оно может быть достаточно малым — десятки наносекунд на одно значение. Однако при этом требуется большой объем памяти.

При программном методе каждое значение представляется в двоичном коде, соответствующем опреде-

Значение угла	Значение функции	Уровень сигнала, В	Значение двоичного кода
0°	0,000000	0,00000	0000000000
11,25°	0,195090	1,95090	0001100100
33,75°	0,555570	5,55570	0100011101
56,25°	0,831470	8,31470	0110100111
78,75°	0,980785	9,80785	0111110110
90,00°	1,000000	10,00000	0111111111
101,25°	0,980785	9,80785	0111110110
123,75°	0,831470	8,31470	0110100111
146,25°	0,555570	5,55570	0100011101
168,75°	0,195090	1,95090	0001100100
180,00°	0,000000	0,00000	0000000000
191,25°	-0,195090	-1,95090	1110011100
213,75°	-0,555570	-5,55570	1011100011
236,25°	-0,831470	-8,31470	1001011001
258,75°	-0,980785	-9,80785	1000001010
270,00°	-1,000000	-10,00000	1111111111
281,25°	-0,980785	-9,80785	1000001010
303,75°	-0,831470	-8,31470	1001011001
326,25°	-0,555570	-5,55570	1011100011
348,75°	-0,195090	-1,95090	1110011100
360,00°	0,000000	0,00000	0000000000

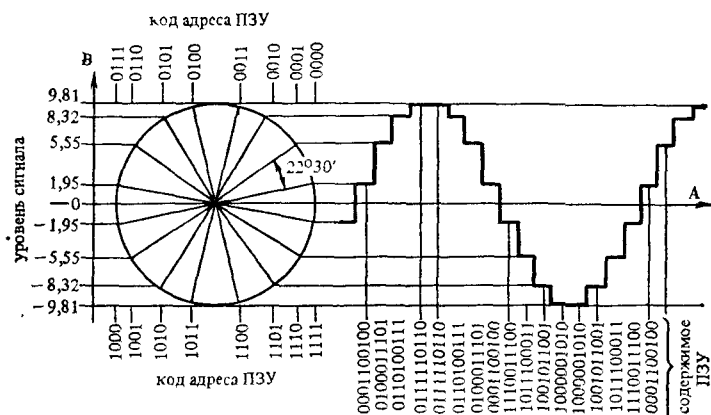


Рис. 3.2. Зависимость фазовых значений и выборок

ленному аргументу вычисляемой функции. Если, например, речь идет о синусоидальном сигнале, то аргументом является значение угла, представляемое в градусах или радианах (рис. 3.2). В табл. 3.3 приведены значения выходного сигнала цифроаналогового преобразователя в диапазоне $-10 \div 10$ В и соответствующий десятиразрядный двоичный код.

Согласно [8] число дискретных отсчетов для реализации генераторов выбирается равным 60—100, они представляются 10—12 двоичными разрядами, так как большинство преобразователей имеет такую же разрядность.

При программных способах реализации генераторов существует большое число методов, позволяющих рассчитывать элементарные функции: разложение в ряд Тейлора и Маклорена, полиномиальная аппроксимация, рациональные приближения, разложение по системам функций Уошла, Хаара, Шaudера, а также методы Волдера и Меджитта.

Используя известное разложение в ряд Маклорена, рассмотрим вычисление $\sin x$:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Подобную сумму удобно вычислять с помощью рекуррентных соотношений вида:

$$U_1 = x,$$

$$U_{k+1} = -\frac{x^2}{2k(2k+1)} U_k \text{ для } k=1, 2, \dots, n-1.$$

На языке ПЛ/М возможно только целочисленное представление аргумента и функции. При формате BYTE аргумент может изменяться в пределах от 0 до 255, что будет соответствовать диапазону от 0° до 146° . Значение функции также будет представлено в целочисленном виде, но может иметь различное число двоичных разрядов — 8, 10 и 12. Это обусловлено тем, что полученные значения будут подаваться на цифроаналоговые преобразователи различных типов.

Фрагмент программы SIN имеет вид:

```

I=1; A=X; SINX=X; EPS=D;
DO WHILE A > EPS;
A=HIGH(A*X)*X/(I+1)/(I+1+1)/K;
IF (I AND 00000001B) > 0 THEN
SINX=SINX-A;
ELSE SINX=SINX+A;
END;

```

Значение синуса по этой программе вычисляется следующим образом: в программу SIN передается параметр X, значение которого присваивается переменным A и SINX. Переменной EPS присваивается значение D, которое можно положить равным I. Если $X > EPS$, то происходит вход в блок DO и начинает выполняться оператор

$$A = \text{HIGH}(A \cdot X) \cdot X / (I+1) / (I+1+1) / K;$$

Так как выражение $\text{HIGH}(A \cdot X)$ в скобках содержит произведение, то для выполнения этой операции отводится транслятором два байта, т. е. как бы описывается тип ADDRESS. Если произведение не превышает 255, то в старшем байте находятся одни нули, и при этом все выражение обращается в 0.

Следующий оператор проверяет значение I, которое может принимать значение 1, 2, 3 и т. д. Если I — величина четная, то к значению SINX прибавляется A, а если нечетное — вычитается. Судить об этом можно по младшему разряду I: если он равен 1, то I нечетное и необходимо вычитать A, если равен 0, то нужно прибавлять A. Для проверки значение I сравнивается с кодом 00000001B с помощью операции конъюнкции.

Для определения значений SIN с большей точностью используются подпрограммы вычисления тригонометрических функций из библиотеки прикладных программ, написанных на ассемблере. Так, например, в работе [7] приводится программа вычисления синуса с удвоенной точностью, занимающая 104 байта.

В некоторых случаях вычисленные значения необходимо представлять не в двоичном коде, а в двоично-десятичном, когда каждая цифра является одной тетрадой. Тогда при использовании, например, двенадцати двоичных разрядов можно представить результат четырьмя десятичными цифрами, но в двоично-десятичном коде. Например, код $(111101011110)_2$, эквивалентный $(3934)_{10}$, должен быть преобразован в код $(0011100100110100)_{2-10}$. Алгоритм такого преобразования состоит в последовательном делении преобразуемой величины на 10^3 , что дает значение старшей цифры, остаток делится на 10^2 и т. д. Подпрограмма преобразования имеет вид:

```

B@ BCD PROCEDURE(X);
DECLARE (X,M) ADDRESS, (A,A1,B,B1,C,C1) BYTE;
A=X/1000; A1=X MOD 1000;
B=A1/100; B1=A1 MOD 100;
C=B1/10; C1=B1 MOD 10;
M=ROL(ROL(A,4)+B,4)+C,4);
/* M=ROL(A,4); M=ROL(M+B,4); M=ROL(M+C,4); */
X=M+C1;
RETURN X;
END B@ BCD;

```

Для пересылки полученных значений синуса в преобразователь используются два порта, например 32 и 64, один — для старшего, а другой — для младшего разряда. Для управления работой преобразователя формируется сигнал ЗАГРУЗИТЬ ЦАП, по которому осуществляется передача информации из портов в приемный регистр преобразователя. Этот сигнал, равный 1, будет засылаться в третий порт, например 16. Сигнал

в регистре порта 16 должен существовать столько времени, сколько требует тип преобразователя. Для этого необходимо организовать задержку между временем записи в порт 16 единицы и ее сбросом, т. е. записью в этот же порт нуля. Если преобразователь имеет большое время установления, то задержка может производиться оператором TIME или последовательным повторением холостой операции.

Рассмотрим построение генератора синусоидальных сигналов с перестраиваемой частотой [8]. Подобный генератор состоит из фазового накопителя, выполненного на сумматоре, и преобразователя фазы в амплитуду сигнала, причем и фаза, и амплитуда представляются в виде кода.

Период колебаний синусоидального сигнала разбивается на ряд равномерных фазовых отсчетов, представляемых 16-разрядным двоичным кодом. При этом два старших разряда определяют квадрант, а остальные — фазовые углы для каждого из четырех квадрантов. Если теперь изменять шаг приращения фазы на величину $K \cdot \Delta$, где K — коэффициент управления частотой, то с каждым тактовым импульсом генератора фазовый угол будет изменяться от 0° до 360° .

Определим изменение частоты сигнала, соответствующей изменению фазы. При значении $K=1$, опорной частоте $F_{оп}=1,311$ МГц и числе разрядов $n=16$ ступенчатое изменение частоты составит 20 Гц:

$$F_{вых} = K \cdot F_{оп} / 2 = 20.$$

Для $K=25$ изменение частоты достигает 500 Гц.

Значение K удобно выбирать кратным 2^n , тогда для низкочастотного диапазона получим значения, приведенные в табл. 3.4 (здесь M — число отсчетов).

Таблица 3.4

К	$F_{вых}$	М	К	$F_{вых}$	М	К	$F_{вых}$	М
16	320	1 024	80	1 600	204	144	2 880	113
32	640	512	96	1 920	170	160	3 200	102
48	960	341	112	2 240	146	176	3 520	93
64	1 280	256	128	2 560	128	192	3 840	85

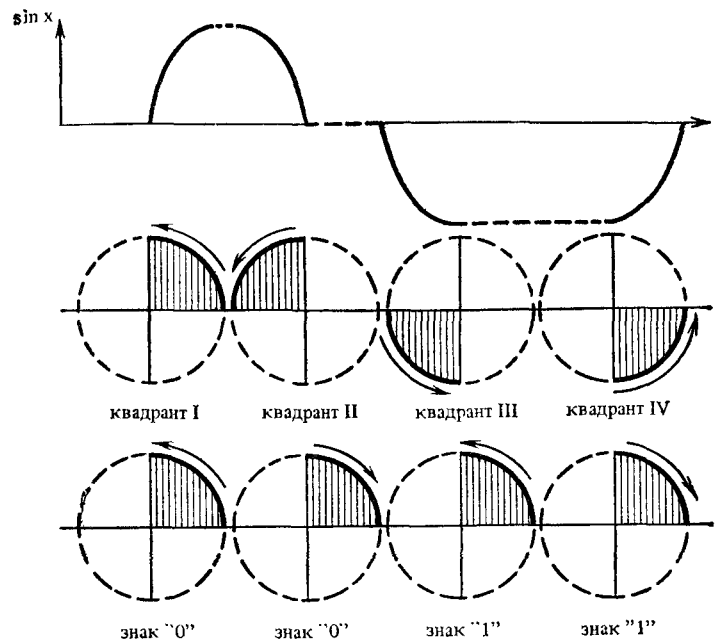


Рис. 3.3. Зависимость получения фазовых значений при формировании синусоидального сигнала в случае использования четырех квадрантов и одного квадранта

Для получения такого ряда частот приращение фазового угла в накопителе составит 0000000000010000, а число отсчетов M в каждом квадранте будет изменяться от 1024 до 85.

При определении отсчетов синусоидального сигнала можно использовать значения только I квадранта (рис. 3.3). Далее значение фазовых отсчетов необходимо преобразовать в значение синусоидального сигнала согласно следующему правилу: адреса ЗУ с хранимыми отсчетами сигнала образуются путем инвертирования фазовых значений, если «просматриваются» II и IV квадранты: признаком этого служит наличие 1 в 15-м разряде (табл. 3.5).

Приведем программную организацию подобного массива во внешнем ЗУ, которое после загрузки может работать независимо от МП и даже асинхронно по отношению к нему. Пусть при записи информации в это

Таблица 3.5

Квадранты	Представление фазовых значений		Двоичное представление адреса ЗУ
	десятичное	двоичное	
I	0	0000000000000000	0000000000000000
	16 383	001111111111	001111111111
II	16 384	0100000000000000	0111111111111111
	32 767	011111111111	0100000000000000
III	32 768	1000000000000000	1000000000000000
	49 151	101111111111	101111111111
IV	49 152	1100000000000000	1111111111111111
	65 535	111111111111	1100000000000000

ЗУ связь МП с ним осуществляется через параллельный интерфейс, выполненный на БИС КР580ВВ55, для управления работой используется порт 17, а для выдачи старшего и младшего байтов — порт 64. Так как в дальнейшем возможны преобразования байтов, то процедуры ввода информации во внешнее ЗУ организуются отдельно для младшего байта IN@1:

```
IN@1 PROCEDURE;
  OUTPUT(17)=00000011B;
  OUTPUT(64)=LOW(OUT@MASS(K));
  OUTPUT(17)=00000001B;
END IN@1;
```

и для старшего байта IN@2:

```
IN@2 PROCEDURE;
  OUTPUT(17)=00000011B;
  OUTPUT(64)=HIGH(OUT@MASS(K));
  OUTPUT(17)=00000001B;
END IN@2;
```

При использовании преобразователя, для работы которого необходимо задавать в старшем разряде знак (0 — знак плюс, а 1 — знак минус), нужно в старший

разряд при формировании отсчетов в III и IV квадрантах записать 1:

```
IN@3 PROCEDURE;
  OUTPUT(17)=00000011B;
  OUTPUT(64)=HIGH(32768+OUT@MASS(K));
  OUTPUT(17)=00000001B;
END IN@3;
```

Если же число разрядов равно 16, то все число нужно сдвинуть вправо на один разряд (при этом произойдет потеря младшего разряда), а затем добавить старший разряд:

$OUTPUT(64) = HIGH(32768 + SHR(OUT@MASS(K), 1));$

Для построения перестраиваемого по частоте генератора необходимо предусмотреть считывание отсчетов с определенным шагом, поэтому в ЗУ нужно хранить большое число отсчетов, например 1024, но получение такого большого числа отсчетов иногда невозможно. Это связано с тем, что при программной реализации нельзя изменять шаг аргумента дробно, и в этом случае используют методы интерполяции, например:

```
DELTA=(SIN(I+1) - SIN(I))/6;
OUT@MASS(K)=SIN(I);
DO I=1 TO 5;
  OUT@MASS(K+1)=OUT@MASS(K)+DELTA;
END;
```

Как следует из табл. 3.5 однозначное соответствие фазовых значений и адресов ЗУ имеет место только в квадранте I. Программа для этого случая имеет вид:

```
IF L< 16384 THEN
  DO;
    K=L;
    CALL IN@1;
    CALL IN@2;
  END;
```

Для квадранта II фазовые значения необходимо инвертировать. Это объясняется тем, что при расчете синуса используются значения из квадранта I, но они «читаются» теперь в обратном порядке (см. рис. 3.3).


```

IF L<32767 THEN
DO;
    L=NOT L; K=L;
    CALL IN@1;
    CALL IN@2;
END;

```

Порядок чтения отсчетов в квадранте III тот же, что и в I, только меняется знак отсчета:

```

IF L<49151 THEN
DO;
    L=L AND 32767;
    K=L;
    CALL IN@1;
    CALL IN@3;
END;

```

В квадранте IV происходят инверсия фазовых значений и изменение знака:

```

IF L<65535 THEN
DO;
    L=(NOT L) AND 32767;
    K=L;
    CALL IN@1;
    CALL IN@3;
END;

```

Разработка цифрового компрессора. На практике это устройство, преобразующее 12-разрядный линейный код в 8-разрядный, строится на элементах жесткой (неперестраиваемой) логики. Программное выполнение подобного устройства позволяет изменять закон компрессирования без переделки устройства. Такие преобразователи необходимы при передаче данных в информационных системах, по линиям связи и т. д. На приемной стороне производится восстановление принятого сообщения. Алгоритмы, по которым осуществляются преобразования, основываются на том, что отбрасываются разряды с меньшими весами, чем у тех, которые остаются. Это приводит к увеличению искажений в передаваемом сообщении, но в некоторых практических случаях эти искажения не столь существенны.

Наиболее простой способ создания подобных устройств — составление таблицы, клетки которой перенумерованы числами, соответствующими различным

значениям 12-разрядного двоичного кода. В каждой графе размещается 8-разрядный двоичный эквивалент. Процедура кодирования весьма проста и состоит в сопоставлении принятого 12-разрядного кода адресу графы, по которому происходит считывание искомого эквивалента. Однако этот способ кодирования требует большого объема памяти (2^{12} байта).

В примере будет использован программный метод, основанный на применении алгоритма преобразования 12-разрядного кода в 8-разрядный.

12-разрядный линейный код

```

S 0 0 0 0 0 0 0 0 a b c
S 0 0 0 0 0 0 0 1 a b c
S 0 0 0 0 0 0 1 a b c *
S 0 0 0 0 0 1 a b c *
S 0 0 0 0 1 a b c *
S 0 0 0 1 a b c *
S 0 0 1 a b c *
S 0 1 a b c *
S 1 a b c *

```

8-разрядный компрессированный код

```

S 0 0 0 0 a b c
S 0 0 0 1 a b c
S 0 0 1 0 a b c
S 0 0 1 1 a b c
S 0 1 0 0 a b c
S 0 1 0 1 a b c
S 0 1 1 0 a b c
S 0 1 1 1 a b c
S 1 0 0 0 a b c

```

Используемые здесь символы имеют следующие значения: S=1 — для положительных сигналов; S=0 — для отрицательных сигналов; символы a, b и c переносятся без изменения; символы, отмеченные звездочкой (*), отбрасываются.

Первый символ S является знаковым, символы между знаковым разрядом и группой abc должны преобразовываться в четыре двоичных разряда следующим образом: считаем, что эти двоичные разряды есть разность, получаемая при вычитании из 8 числа нулей, заключенных между символом знака и первой единицей в 12-разрядном исходном коде. Следующая за нулями единица пропускается во всех кодовых комбинациях, за исключением последней.

Сформулируем алгоритм компрессирования:

1. выделить знаковый разряд;
 2. проанализировать 12-разрядное число и определить один из 9 интервалов;
 3. выделить разряды a, b и c;
 4. образовать 8-разрядное число путем объединения знакового разряда, a, b и c разрядов и одного из 4-разрядных кодов, лежащих в диапазоне от 0000 до 1000.
- Для определения параметров, необходимых для составления программы, рассмотрим табл. 3.6.

таблица 3.6

Номер i порядку	12-разрядный линейный код, размещаемый в 16-разрядной ячейке	Интервал (A)	Маска (I)	Количе- ство # (L)	Значение 6-, 5-, 4- и 3-го разрядов
1	0000S00000000abc	<7	7	0	S0000abc=0
2	0000S00000001abc	<16	7	0	S0001abc=8
3	0000S0000001abc*	<32	15	1	S0010abc=16
4	0000S000001abc**	<64	31	2	S0011abc=24
5	0000S00001abc***	<128	63	3	S0100abc=32
6	0000S0001abc****	<256	127	4	S0101abc=40
7	0000S001abc*****	<512	255	5	S0110abc=48
8	0000S01abc*****	<1024	511	6	S0111abc=56
9	0000S1abc*****	<2048	1023	7	S1000abc=64

Из табл. 3.6 следует, что каждый поступающий на компрессирующее устройство 12-разрядный код при компрессировании размещается в 16-разрядной ячейке. Для определения, к какому интервалу A относится поступивший код, необходимо исключить из него разряды с 11-го по 15-й, в том числе и знаковый разряд S. Эта операция может быть осуществлена при помощи конъюнкции над преобразуемыми кодами и маской вида $(0000\odot 0111\odot 1111\odot 1111)_2$ или $(07FFH)_{16}$. Затем нужно определить интервал A, к которому принадлежит преобразуемый код, что легко сделать, если этот код последовательно сравнивать со всеми интервалами. Наличие единицы в 3-, 4-, 5-, ..., 10-м разрядах и возможность появления любых комбинаций в разрядах «abc» и разрядах, отмеченных знаком *, и определяют интервал. Пусть, например, код после удаления знакового разряда имеет вид:

Линейный код	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	1

Наличие 1 в 6-м разряде при любых наборах в разрядах 5—0 никогда не может превысить 127, так как сумма весов всех предыдущих разрядов с 0-го по 5-й разряд и 1 в 6-м меньше 128, т. е. единицы в следующем старшем разряде после 6-го. Отсюда следует, что проверка выражения $<2^n$ позволяет однозначно определить интервал, к которому относится поступивший 12-разрядный код.

Далее необходимо выделить разряды «abc». Для этого используем маску I, вид которой будет изменяться от интервала к интервалу со сдвигом на один разряд влево, т. е. умножением на два каждого предыдущего значения и вычитанием единицы:

II интервал $0000000000000111-2^3-1=7$;

III интервал $0000000000001111-2^4-1=15$;

IX интервал $0000001111111111-2^{10}-1=1023$.

Следующая операция — удаление разрядов, отмеченных знаком *, может быть осуществлена сдвигами вправо. Число сдвигов L изменяется от 1 до 7, начиная с III интервала. Значения разрядов 6-, 5-, 4- и 3-го определяются следующим образом: 8 минус количество нулей, заключенных между S и первой единицей кода, это образует следующие значения 8, 16, 24, ..., 64. Такую последовательность можно получить из выражения $(L+1) \cdot 8$, что эквивалентно сдвигу на три разряда влево.

Для образования циклической программы необходимо из общего числа интервалов исключить I и II интервалы, которые нельзя охватить в одном цикле, так как в I интервале $A \leq 7$, что не кратно 2^n , а во II интервале отсутствуют разряды, отмеченные знаком *, т. е. нет сдвигов вправо, а параметр сдвига, равный нулю, нельзя использовать в операторе SHR.

Ниже представлена программа компрессирования.

```

CODE⊙COMPRESSION: PROCEDURE(A) BYTE;
  DECLARE (KOD,A,I,J) ADDRESS,(B,L,M) BYTE;
  I=16; J=7; L=1;
  KOD=A; A=A AND 7FFH;
  IF A < J THEN B=A AND J;
  ELSE IF A < I THEN B=(A AND J)+SHL(L,3);
  DO M=1 TO 7;
    IF A < SHL(I,1) THEN
      B=SHR(A AND (I-1),L)+SHL(L+1,3);
    ELSE DO; I=SHL(I,1)
          L=L+1
    END;
  DO .....
  END
END,
B=B HIGH(SHL(KOD AND 800H,4);
RETURN B;
END CODE⊙COMPRESSION;

```

3.4. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СОПРЯЖЕНИЯ МП С ВНЕШНИМИ УСТРОЙСТВАМИ В МИКРОПРОЦЕССОРНЫХ СИСТЕМАХ

Разнообразие областей применения микропроцессорных систем обусловлено возможностью МП осуществлять связь с внешними устройствами как непосредственно, так и с помощью программируемых контроллеров, выполняемых на БИС. Среди таких внешних устройств особое место занимают аналого-цифровые и цифроаналоговые преобразователи, совместная работа которых с МП позволяет применять цифровые методы обработки информации.

Совместная работа АЦП и ЦАП с МП возможна в синхронном и асинхронном режимах. В первом случае к АЦП обращаются как к адресуемой ячейке памяти, причем для образования сигнала запуска АЦП используется команда записи в ЗУ, а результат преобразования поступает в МП по команде чтения из ЗУ. Если время преобразования значительно, то МП переходит в режим ожидания путем выполнения оператора TIME, что снижает производительность микропроцессорной системы.

Более эффективным способом обмена информацией является асинхронный режим, обеспечиваемый либо преобразователем, выставляющим вектор прерывания, либо МП, который после получения сигнала прерывания осуществляет опрос внешних устройств и выявляет преобразователь, требующий обслуживания.

Существует еще один способ обмена информацией — прямой доступ к памяти непосредственно самого преобразователя. В этом случае управление осуществляется специализированным контроллером, что сопряжено с дополнительными аппаратными затратами, но данный метод наиболее высокопроизводительный.

Сопряжение МП с АЦП и ЦАП может устанавливаться как на базе буферных регистров, используемых в качестве портов ввода или вывода, так и с использованием программируемых устройств типа KP580BB55.

Связь между МП и преобразователями устанавливается командами INPUT и OUTPUT, передающими информацию в порты. При этом инициализируются сигналы управления СЧИТЫВАНИЕ, ПРИЕМ, ВЫДАЧА и ЗАПИСЬ. Кроме того, порты необходимо синхронизировать с сигналами, вызываемыми командами INPUT

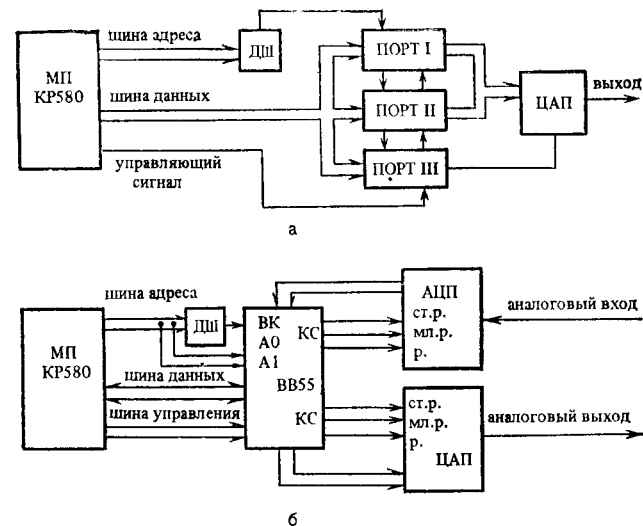


Рис. 3.4. Схема связи: а — МП с ЦАП; б — МП и BB55 с АЦП и ЦАП:

ЦАП — цифроаналоговый преобразователь; ДШ — дешифратор; BB55 — параллельный интерфейс типа KP580BB55; КС — канал управления С; АЦП — аналого-цифровой преобразователь; ст. р. — старший разряд; мл. р. — младший разряд; р — режим

и OUTPUT, чтобы отличить адреса портов от адресов ЗУ, выставляемых на адресной шине.

Схема сопряжения МП и ЦАП представлена на рис. 3.4,а. Для образования интерфейса с ЦАП используются три порта: I — для младшего байта данных, II — для двух или четырех старших разрядов, а порт III — для сигнала управления загрузкой ЦАП, подаваемого на вход ЗАПИСЬ.

Вывод информации из МП обеспечивается в результате выполнения следующей последовательности действий:

выделяется младший байт;
младший байт пересылается в порт I и запоминается;

выделяется старший байт и подготавливаются два или четыре разряда, в зависимости от типа ЦАП, для выдачи в порт II;

два или четыре старших разряда пересылаются в порт II и запоминаются;

в порт III выдается управляющий сигнал ЗАПИСЬ;

в порт III выдается управляющий сигнал ЗАПИСЬ.

Пусть выдаваемые в ЦАП кодовые слова располагаются в массиве ВЫХ_МАССИВ, тогда программа для вывода информации будет иметь вид:

```
DECLARE ВЫХ_МАССИВ (100) ADDRESS;
      (УПР_СИГНАЛ,1) BYTE;
УПР_СИГНАЛ=01...0.1B;
DO I=0 TO 99;
  OUTPUT (16)=LOW(ВЫХ_МАССИВ(I));
  OUTPUT (32)=HIGH(ВЫХ_МАССИВ(I));
  OUTPUT (17)=УПР_СИГНАЛ;
  OUTPUT (17)=NOT УПР_СИГНАЛ;
END;
```

Для согласования работы МП и АЦП используются как аппаратные, так и программные средства, причем последние являются более универсальными, но требуют от разработчиков детального знания аппаратных средств, которые постоянно совершенствуются.

Особенность применения некоторых АЦП состоит в том, что в состав БИС входят устройства для организации побайтного обмена информацией с 8-разрядной шиной данных МП. Для ввода информации в МП предназначены три порта ввода, совмещаемые с АЦП: I— для восьми младших разрядов, II— для четырех старших и III— для получения сигнала КОНЕЦ ПРЕОБРАЗОВАНИЯ. Запуск АЦП осуществляется через порт I, куда поступает сигнал начала преобразования ЗАПУСК.

Порядок организации обмена между МП и АЦП следующий:

формируется сигнал начала преобразования— ЗАПУСК, для чего в порт I вначале засылается 1, а затем 0;

опознается сигнал КОНЕЦ ПРЕОБРАЗОВАНИЯ;

принимаются данные с АЦП, вначале младшие разряды, а затем старшие;

пересылаются данные в массив, отведенный в памяти.

Программа обмена данными МП с АЦП имеет следующий вид, причем в качестве порта I используется порт 17.

```
DECLARE (I,J,K) BYTE,
      ВХ_МАСС(100) ADDRESS,
      (УПР_СИГН,КОНЕЦ_ПР,МЛ_БТ,СТ_БТ) BYTE;
УПР_СИГН=1;
OUTPUT (17)=УПР_СИГН;
OUTPUT (17)=NOT УПР_СИГН-1;
DO I=0 TO 99;
  КОНЕЦ_ПР=INPUT (128);
  J=0;
  DO WHILE J < K;
    IF КОНЕЦ_ПР=1 THEN
      DO: МЛ_БТ=INPUT (16);
        СТ_БТ=INPUT (32);
        ВХ_МАСС(I)=SHL(СТ_БТ,8)+МЛ_БТ;
      END;
    ELSE J=J+1;
  END;
END;
```

В небольших микропроцессорных системах можно организовать ввод-вывод, адресуемый как обращение к памяти. В этом случае управляющие сигналы МП и операторы INPUT и OUTPUT не используются, а для адресации устройств ввода-вывода применяют адреса памяти. Для инициализации внешних устройств можно использовать линии адресной шины, подключаемые к входу ВК внешних устройств. Такой способ адресации удобен для систем, в состав которых входят программируемые интерфейсные устройства. В этом случае вначале программно задаются режимы их работы, а ввод и вывод информации в МП осуществляются из ячеек памяти.

Рассмотрим построение микропроцессорной системы, в состав которой входит программируемое устройство ввода-вывода типа КР580ВВ55 (рис. 3.4,б). Для программного управления подобным устройством предназначены управляющие слова, у которых старший разряд равен 1 либо 0. В первом случае задается один из трех режимов работы устройства, а во втором — устанавливаются отдельные разряды канала С.

Необходимо вывести 100 двухбайтных элементов массива ВЫХ_МАССИВ в ЦАП через канал А, а для управления ЦАП задействовать разряды С₀, С₁ и С₂ канала С. Затем требуется ввести 200 элементов во ВХ_МАССИВ через канал В, а управление осуществлять через С₅, С₆ и С₇.

Программа обмена может быть представлена следующим образом:

```

        OUTPUT(BB55+3)=1000000010B;
        /* УСТАНОВКА РЕЖИМА РАБОТЫ BB55 */
DO I = 0 TO 99;
    OUTPUT(16)=LOW(BYX_MACC(I));
    OUTPUT(17)=00000011B;
    OUTPUT(16)=HIGH(BYX_MACC(I));
    OUTPUT(17)=00000101B;
    OUTPUT(19)=00000001B;
END;
DO J=0 TO 99;
    OUTPUT(20)=00001001B;
    OUTPUT(17)=00001011B;
    BXOMASS(J)=INPUT(16);
    OUTPUT(17)=00001101B;
END;

```

Часто внешние устройства, готовые к обмену информацией с МП, запрашивают возможность прервать работу МП для организации обмена. Для этих целей у МП КР580 предусмотрен вывод ЗАПРОС на ПЕРЫВАНИЕ (INT), на который внешнее устройство подает сигналы запроса. МП завершает выполнение текущей команды и выдает сигнал РАЗРЕШЕНИЕ ПЕРЫВАНИЯ (INTE). В языке ПЛ/М имеются два оператора DISABLE и ENABLE, которые могут запретить или разрешить прерывание, т. е. изменить состояние на выходе INTE. После начала прерывания необходимо запомнить содержимое внутренних регистров и PSW. Для этого внешние аппаратные средства должны выдать на шину данных МП адрес подпрограммы, осуществляющей обработку прерываний, т. е. запоминание состояния МП до начала прерывания. Этот адрес выдается после подачи сигнала управления ПОДТВЕРЖДЕНИЕ ПЕРЫВАНИЯ (INTA).

В МП КР580 можно включать восемь подпрограмм прерываний, имеющих определенные адреса в адресном пространстве МП: 0Н, 8Н, 10Н, ..., 38Н.

Программы обслуживания прерываний в ПЛ/М оформляются как процедуры общего вида без параметров: INTERRUPT *n*, где *n* имеет значения от 0 до 7, соответствующие адресам 0Н, 8Н, 10Н и т. д.

Работа МП всегда начинается с состояния, запрещающего прерывание, а предложение ENABLE разрешает выполнение прерываний по запросу, который поступает от внешнего устройства. Таким образом, в программе необходимо предусмотреть возможность организации прерываний от внешнего устройства. В случае, если прерывание разрешено и идет обработка прерыва-

ния, вновь пришедший запрос от другого внешнего устройства должен быть запрещен оператором DISABLE.

Операторы ENABLE и DISABLE употребляются в том случае, если при выполнении программы необходимо, например, принять массив информации от АЦП: перед началом работы нужно запретить прерывание, а после окончания — разрешить вновь.

Если параметр *n* описателя INTERRUPT совпадает с кодом, выдаваемым на шине данных внешним устройством по сигналу INTA, то инструкцией INTERRUPT будут запрещены прерывание и активизация данной процедуры до выполнения оператора RETURN или END, после чего прерывание вновь разрешается.

Команды вызова процедур прерывания формируются внешними устройствами автоматически или человеком-оператором. При аварийных ситуациях вызываемая процедура включает внешние устройства сигнализации и оповещения, изменяет режим работы системы.

Ниже представлена процедура прерывания для микропроцессорной системы, в которой внешнее устройство может выставить на шину данных команду RST7 в том случае, если отсутствует питающее напряжение в одном из блоков устройства.

```

DAMAGE: PROCEDURE INTERRUPT 7;
    CALL INDICATION;
    CONDITION=CONDITION OR 00110011B;
    CALL SIGNAL;
    GOVERNOR=GOVERNOR+11000000B;
    TIME @ A=TIME @ A+00000011B;
    OUTPUT(0FBH)=12;-
    OUTPUT(3H)=11100111B;
END DAMAGE;

```

Процедура прерывания формирует сигнал оповещения (в виде звуковых или световых сигналов), изменяет некоторые параметры системы, после чего возвращает управление программе.

Приведем еще один пример процедуры прерывания. RST2 вызывает подпрограмму обработки прерывания, которая проверяет некоторые логические условия параметров системы и изменяет выходные сигналы в зависимости от реальных условий.

```

BREAK: PROCEDURE INTERRUPT 2;
DECLARE (ABC,IN⊙1,OUT⊙1,OUT2,) BYTE;
IF IN⊙1=00111111B THEN
    OUTPUT(0F0H)=100;
ELSE ABC=IN⊙1;
    IF (ABC=10000000B) OR
       (ABC=01000000B) OR
       (ABC=00100000B) OR
       (ABC=00010000B) OR
       (ABC=00001000B) OR
       (ABC=00000100B) OR
       (ABC=00000010B) OR
       (ABC=00000001B) THEN
        DO;
            OUT⊙1=11000001B;
            OUT⊙2=00111100B;
        END;
        IF ABC=11111111B THEN
            OUTPUT(0FH)=0FFH;
        IF ABC=00000000B THEN
            DO;
                OUT⊙1=11110000B;
                OUT⊙2=00001111B;
                OUTPUT(0FH)=0F0H;
            END;
        IF ABC=11100111B THEN
            HALT;
END BREAK;

```



ГЛАВА 4

ПОДГОТОВКА ПРОГРАММ ДЛЯ МИКРОПРОЦЕССОРНЫХ СИСТЕМ

Программное обеспечение любой отладочной вычислительной системы обычно включает в себя программы-трансляторы с языков программирования, программы-редакторы, загрузчики и т. д. Для обращения к этим программам используется язык управления заданиями, состоящий из управляющих операторов. С их помощью можно задать все параметры трансляции и определить состав выводимых на печать сообщений о ходе и результатах трансляции.

Обязательным этапом подготовки программ к выполнению является редактирование связей. Операторы, управляющие работой редактора, включают нужные блоки в загрузочный модуль, вызывают библиотеки, содержащие модули, определенные внешними ссылками, и указывают символическое имя входа в загрузочный модуль.

При трансляции программ пользователя на печать или дисплей выдаются различные диагностические сообщения в виде буквенных сочетаний или префиксов, соответствующих различным обрабатываемым программам операционной системы, участвующим в трансляции. После буквенного сочетания или префикса стоит номер, по которому в справочном руководстве отыскивается сообщение о возможных ситуациях, приводящих к ошибкам.

При появлении ошибок во время загрузки программы выдаются сообщения о серьезности ошибок и об именах подпрограмм, не обнаруженных в библиотеках пользователя или среди стандартных подпрограмм.

Успешная загрузка еще не гарантирует выполнения программы, т. е. она может завершиться аварийно. В случае аварийного завершения программы обработки ошибок выдают диагностические сообщения. Иногда

выполняемые задания прерываются из-за сбоев в питании или неисправностей оборудования, в этих случаях также выдаются сообщения об ошибках.

4.1. РЕЗИДЕНТНЫЙ ТРАНСЛЯТОР С ЯЗЫКА ПЛ/М

В состав программного обеспечения микроЭВМ входит резидентный транслятор. Практически все описываемые в литературе трансляторы [9—11] имеют много общего. Отличие состоит только в числе и написании управляющих операторов, а также в форме представления листинга.

Как правило, транслятор поставляется заводом-изготовителем на дискете, где кроме самой программы приводится описание транслятора. Перед началом работы транслятор необходимо загрузить в ЭВМ с системного диска операционной системы, затем ввести исходный текст программы пользователя в файл, расположенный на диске, с помощью программы-редактора. После этого возможна трансляция, осуществляемая в несколько этапов с помощью подпрограмм редактора связей. В итоге создается загрузочная программа.

При нормальном завершении задания протокол задания размещается на том же диске, где и исходная программа, и имеет то же имя, но расширенное несколькими символами типа LISTING.

Для работы транслятора требуется несколько рабочих файлов, перегружаемых в ЗУ с диска, которые после трансляции стираются.

Трансляцией управляют операторы PRINT, OBJECT, SYMBOLS, XREF, PAGING, DATE, TITLE. Протокол трансляции состоит из страниц, каждая из которых имеет заголовок, номер версии используемого транслятора, имя оттранслированного модуля, распечатку исходного текста с номерами операторов и сообщениями об ошибках и т. д.

Построение трансляторов, а также практическая работа с ними достаточно подробно описываются в заводских инструкциях, прилагаемых заводами-изготовителями к поставляемым изделиям. Определенный интерес представляет возможный вариант распределения памяти МП, осуществляемый транслятором. Все адресное пространство делится на четыре части:

программный сегмент, в котором размещаются машинный код, сформированный из исходной программы, и ячейки памяти, определяемые оператором DATA;

сегмент для переменных данных, за исключением базированных, а также данных, присвоенных оператором DATA и выделенных для процедур ввода;

сегмент стековой памяти, куда входят адреса возврата, параметры процедур, ячейки временного хранения, не объявленные явно, адреса локальных переменных в процедурах ввода и слова состояния PSW;

оставшаяся область памяти, куда можно обратиться, используя оператор языка ПЛ/М MEMORY.

При обработке процедур могут быть приняты следующие соглашения: объявленные формальные переменные обрабатываются как локальные переменные. Один параметр BYTE помещается в регистр С, а параметр ADDRESS — в регистр В (старший разряд) и в регистр С (младший разряд). Если имеется второй параметр ADDRESS, то первый размещается, как было указано выше, а второй — в регистры D и E, соответственно в регистр D — старший разряд, а в регистр E — младший разряд. Если задано более двух параметров, то остальные размещаются в области стековой памяти.

Регистры общего назначения используются для приема результатов при выполнении процедур. Если возвращаемый параметр имеет тип BYTE, его значение размещается в регистре А, а если параметр имеет тип ADDRESS, то в регистре H располагается старший байт, а в регистре L — младший.

При обработке прерываний значение счетчика команд хранится в стековой памяти, управление программой переходит по команде JMP, формируемой транслятором. Содержимое регистров может заноситься в стековую память в следующем порядке:

IIL→DE→BC→A→байт признака F

Адресное пространство в реальных системах может быть разделено самим транслятором, т. е. по умолчанию, и пользователем. Абсолютные значения адресов областей машинных кодов, данных и свободной области памяти, определяемых как по умолчанию, так и самим пользователем, зависят от конкретной системы.

4.2. КРОСС-ТРАНСЛЯТОР ДЛЯ ЯЗЫКА ПЛ/М

Кросс-системы позволяют вести отладку программ на универсальных ЭВМ в условиях вычислительного центра как в пакетном режиме, так и в режиме разделения времени. Кроссовый режим отладки может быть организован на машинах серии ЕС ЭВМ или типа СМ-4, а также на вычислительных системах типа АСВТ [5, 12]. При этом используется весь набор сервисных программ универсальных ЭВМ с их богатыми возможностями редактирования. Диалоговый режим работы делает похожим этап отладки на работу с резидентными средствами, но обеспечивает дополнительные возможности, трудно реализуемые в резидентных системах. Основное достоинство кроссового режима состоит в том, что язык ПЛ/М можно изучать без аппаратных средств.

Кросс-транслятор с языка ПЛ/М для МП типа КР580, содержащий около 8000 операторов языка Фортран IV, требует 256 Кбайт оперативной памяти инструментальной ЭВМ. Этот транслятор позволяет задавать параметры трансляции, определять устройства ввода исходного текста и вывода протоколов трансляции.

Входной язык кросс-трансляторов ориентирован на различные версии языка ПЛ/М. Транслятор накладывает ограничения на размеры и число модулей исходного текста, а в некоторых случаях не допускает раздельную трансляцию и объединение отдельных модулей, в связи с чем программист должен объединять модули на уровне исходного текста.

Ограничения накладываются и на загрузчик, который не может перемещать коды программ из одной области ЗУ в другую.

Трансляция программ осуществляется за два прохода или за два шага. При первом проходе вводится исходный текст с перфокарт или из файла, расположенного на диске, затем начинается грамматический разбор текста, выявляются синтаксические ошибки и формируются промежуточные файлы, необходимые для передачи шагу 2. При втором проходе считываются промежуточные файлы и формируется загрузочный модуль программы пользователя.

Для каждого прохода существуют свои наборы параметров трансляции, манипулируя которыми можно

легко задавать конкретные файлы ввода-вывода и порядок взаимодействия с ними.

Существующий набор начальных параметров позволяет избегать аварийного завершения, значения параметров располагаются по умолчанию, поэтому выдаваемый листинг отражает все этапы трансляции.

Загрузочный модуль программы, полученный после трансляции, может загружаться в ОЗУ или ППЗУ микропроцессорной системы, а также в файл, используемый программой моделирования работы МП.

Транслятор размещает загрузочный модуль так же, как и в некоторых резидентных средствах отладки: программный сегмент или область команд, стековый сегмент или область переменных, свободный сегмент.

Распределение памяти представлено на рис. 4.1. Программный сегмент содержит коды команд исходной программы, а также константы, объявленные в декларации с атрибутом DATA. Начальный адрес командной области может определяться либо числовой меткой первого оператора, либо параметром второго прохода $\odot$ HEADER. Если начало программного сегмента не задано, коды программы загружаются с нулевого адреса.

В стековом сегменте хранится стек микропроцессорной системы, а в области переменных размещаются переменные, объявленные в операторах деклараций с ат-

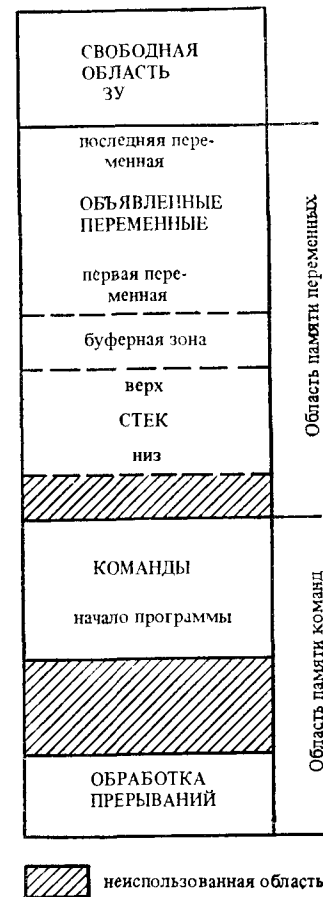


Рис. 4.1. Распределение адресного пространства МП

рибутаи BYTE и ADDRESS, в порядке их появления и разбора в исходном тексте.

Память для стека и переменных выделяется следующим образом:

транслятор определяет емкость памяти для переменных и максимальную для данного исходного текста глубину стека;

последний код сегмента переменных размещается так, чтобы стек не наложился на программный сегмент и емкость памяти между ними не превышала одной страницы, т. е. 256 байт.

Параметры первого прохода трансляции представлены в табл. 4.1.

Таблица 4.1

Параметр	Принимаемые значения	Значение по умолчанию	Комментарий
1	2	3	4
P1A	0 или 1	0	Сообщения синтаксического разбора
P1B	0 или 1	1	Распечатка стека
P1C		0	Начало нумерации строк
P1D	120	120	Величина буфера для вывода
P1E	0 или 1	0	Конец файла, прерывание
P1G	0 или 1	0	Генерация промежуточного кода
P1I*	от 1 до 7	1	Номер устройства ввода шага 1
P1J	от 1 до 7	6	Номер устройства вывода шага 1
P1K	72	72	Промежуточный массив данных
P1L*		1	Левая граница
P1M	0 или 1	1	Передача таблицы символов в шаг 2
P1O*	от 1 до 7	1	Номер устройства вывода для распечатки
P1P*	0 или 1	1	P=0, не печатать исходный текст P=1, печатать исходный текст
P1R*	от 0 до 80 0, 1 или 2	72 0	Правая граница Печать таблицы символов после шага 1
P1T	0, 1 или 2	1	T=0, пакетный режим T=1, диалоговый режим T=2, режим технического обслуживания
P1U	от 1 до 7	7	Номер устройства для таблицы символов
P1V	72	72	Управляющий параметр для массива данных
P1W	от 0 до 120	94	Длина строки вывода
P1Y	1	1	Номер страницы при выводе

Примечание. Параметры, отмеченные знаком *, используются при нормальной работе транслятора.

Таблица 4.2

Параметр	Принимаемое значение	Значение по умолчанию	Комментарий
P2F*	0 или 1	1	Печать мнемоники
P2G*	0 или 1	1	Вывод перекрестных ссылок
P2H*		0	Начальный адрес программы
P2M*	0 или 1	1	Вывести таблицу символов
P2O*	от 1 до 7	1	Номер устройства вывода
P2Q*	0 или 1	1	Выдача объектного модуля в шестнадцатеричном коде
P2V*	0 или 10	0	Печать содержимого памяти с нулевой страницы

Обязательные параметры второго прохода приведены в табл. 4.2.

При стандартном наборе параметров протокол первого прохода содержит сообщение о номере версии транслятора, команды для первого прохода и исходный текст программы. Под строкой, содержащей ошибку, печатается сообщение об ошибках:

pppp ERROR m NEAR S,

где pppp — номер строки;
ERROR — указание на ошибку;
m — признак рода ошибки;
NEAR S — ближайший от ошибки символ.

Завершается протокол сообщения о числе обнаруженных транслятором ошибок.

При втором проходе выдается сообщение о номере прохода, выводятся таблица соответствия номера строки программы и кода загрузочного модуля, максимальная глубина стека, таблица размещения в ОЗУ переменных, меток и процедур программы, мнемоническое представление загрузочного модуля и т. д. Вновь, как при шаге 1, выводятся сообщения об ошибках. В случае неправильного формирования пакета выдается один из ошибочных кодов завершения задания.

Общее количество сообщений о синтаксических ошибках, выявляемых при первом проходе, равно 50. При втором проходе выдается более 50 сообщений об ошибках, связанных с формированием загрузочного модуля.

Характер ошибок, допускаемых обучающимися языку ПЛ/М, ничем не отличается от ошибок, возникающих при изучении других языков высокого уровня. При

работе в пакетном режиме значительную долю ошибок составляют ошибки перфорации. Их количество можно сократить предварительной проверкой перфокарт. Трудноопределимые ошибки связаны с неверно отперфорированными картами заданий.

4.2.1. Описание задания на запуск кросс-транслятора*

Компиляция программы, написанной на языке ПЛ/М, состоит из двух шагов. Для выполнения первого шага предназначена программа с именем PL#E1, а для выполнения второго — с именем PL#E2.

Задание на трансляцию программы с языка ПЛ/М может быть составлено в следующем виде (см. с. 145), если пользователь не применяет каталогизированные процедуры, хранящиеся в библиотеке SUS1.PROCLIB.

В этом задании первый шаг компиляции назван PLM1, а второй — PLM2. Первый проход компилятора требует 150 Кбайт ОЗУ, второй — 180 Кбайт. Задание составлено для трансляции исходной программы, представленной на перфокартах, с получением загрузочного модуля на перфоленте.

Для работы компилятора, написанного на Фортране IV, употребляются операторы языка управления заданиями JOB, EXEC, DD с именем STEPLIB, которое определяет библиотеку загрузочных модулей.

В нашем случае оба модуля PL#E1 и PL#E2 кросс-компилятора с языка ПЛ/М находятся в библиотеке загрузочных модулей MICRO.LOAD, размещенной на томе прямого доступа DWC014. Все промежуточные рабочие наборы в нашем случае также расположены на томе прямого доступа с именами оператора DD:

FT10F001 — определяет выходной последовательный набор данных; содержит текст исходной программы, скомпонованной транслятором;

FT11F001 — устанавливает выходной последовательный набор данных транслятора; содержит коды объектной программы, может также включать в себя таблицу символов;

FT12F001 — задает выходной последовательный на-

* При написании этого раздела был использован материал Ю. И. Снонского.

```
//S1 JOB (00000,11111), 'ИВАНОВ'
//PLM1 EXEC PGM=PL #E1, REGION=150K
//STEPLIB DD DSN=MICRO.LOAD, DISP=SHR,
//          UNIT=SUSDA, VOL=SER=DWC 014
//FT05F001 DD DDNAME=SYSIN1
//FT06F001 DD SYSOUT=A
//FT10F001 UNIT=SYSDA,
//          DCB=(RECFM=FB, LRECL=80, BLKSIZE=400)
//          VOL=SLR=DWC 014, DSN=&&PL1,
//          SPASE=(TRK,(5,1), DISP=(PASS)
//FT11F001 DD UNIT=SYSDA,
//          DCB=(RECFM=F, BLKSIZE=121),
//          VOL=SER=DWC014, DSN=OBJ,
//          SPASE=(TRK,(2,1), DISP=(KEEP)
//FT12F001 DD UNIT=SYSDA,
//          DCB=(RECFM=F, BLKSIZE=121),
//          DISP=(MOD, PASS),
//          SPASE=(121,(300,200))
//PLM1.SYSIN1 DD *
//          ПУСТАЯ ПЕРФОКАРТА
//          ТЕКСТ ПРОГРАММЫ ПЛ/М НА ПК
EOF
//FT05F002 DD *
//          УПРАВЛЯЮЩИЕ ПАРАМЕТРЫ ПЕРВОГО ШАГА
//*
//PLM2 EXEC PGM=PL #E2, REGION=180K
//STEPLIB DD DSN=MICRO.LOAD,
//          DISP=SHR, UNIT=5061,
//          VOL=SER=DWC014
//FT05F001 DD DDNAME=SYSIN4
//FT05F002 DD DDNAME=SYSIN5
//FT05F003 DD DDNAME=SYSIN6
//FT06F001 DD SYSOUT=A
//FT10F001 DD DSN=*.PLM1.FT10F001
//          DISP=(OLD, DELETE)
//FT11F001 DD DSN=OBJ, DISP=(OLD, KEEP),
//          UNIT=5061, VOL=SER=DWC014
//FT12F001 DD DSN=*.PLM1.FT12F001,
//          DISP=(OLD, DELETE)
//FT13F001 DD DSN=*.PLM1.FT13F001,
//          DISP=(OLD, DELETE)
//PLM2.PTOUT UNIT=009
//PLM2.SYSIN4 DD *
//          УПРАВЛЯЮЩИЕ ПАРАМЕТРЫ ВТОРОГО ШАГА
//          ПУСТАЯ ПЕРФОКАРТА
//PLM2.SYSIN5 DD DUMMY
//PLM2.SYSIN6 DD DUMMY
//*
```

бор данных, альтернативный для набора FT11F001, в случае нехватки памяти;

FT06F001 — указывает выходной набор данных, предназначенный для печати результатов выполнения программы;

FT05F001 — на первом шаге компиляции содержит ссылку на оператор DD с именем PLM1.SYSIN1. После

этого оператора идет пустая перфокарта и текст программы на языке ПЛ/М, представленной на перфокартах. Этот оператор означает начало входного набора данных транслятора;

FT05F002 — используется только на первом шаге компиляции. В нем содержится список операторов, необходимых для управления выполнением первого шага, если ввод исходной программы осуществляется с перфокарт.

Входной набор данных для первого шага компиляции. Входным набором для первого шага компиляции является текст программы, написанный на языке ПЛ/М и размещенный на перфокартах, а также набор параметров, управляющих промежуточными выходными потоками первого шага (распечатка программы, запись программы в библиотеку и т. д.). Этот параметр обеспечивает выдачу листинга с текстом программы и сообщением об ошибках на печатающее устройство. Текст программы включается во входной поток после управляющего оператора DD с именем PLM1.SYSIN1 и должен начинаться с пустой перфокарты.

Управляющие операторы кросс-компилятора с языка ПЛ/М первого шага включаются во входной поток после управляющего оператора с именем FT05F002.

Выходные наборы первого шага. Выходной листинг программы, содержащий текст программы и сообщения об ошибках, можно получить на первом шаге компиляции, задав во входном наборе первого шага управляющий оператор @PRINT=1. Используя другие управляющие операторы первого шага компиляции, можно получить ряд таблиц, содержащих результаты этого шага компиляции.

Наиболее часто применяются следующие операторы:

@PRINT=1 — печать исходного текста;

@INPUT=1 — номер устройства вывода;

@GENERATE=1 — генерация промежуточного кода.

Допускаются сокращенные имена этих операторов — соответственно P, I и G.

Кроме листинга программы, выдаваемого на печать на первом шаге, получается ряд выходных наборов данных, которые затем передаются на второй шаг как входные.

Входные и выходные наборы второго шага. Входными наборами второго шага компиляции служат резуль-

таты синтаксического анализа, проведенного на первом шаге. В трех управляющих операторах DD находятся входные наборы второго шага, являющиеся результатами выполнения первого:

оператор с именем FT10F001 определяет входной последовательный набор данных, включающий текст исходной программы;

оператор с номером FT11F001 содержит коды объектной программы и таблицу символов;

оператор с именем FT13F001 включает в себя промежуточную таблицу символов.

На втором шаге оператор DD с именем FT05F001 содержит ссылку на оператор DD с именем PLM2.SYSIN4, после которого идут управляющие операторы для выполнения второго шага компиляции.

Действие операторов DD с именами FT05F002 и FT05F003 поддается с помощью параметра DUMMY.

Набор управляющих операторов ввода-вывода второго шага поступает на вход кросс-компилятора после управляющего оператора DD с именем PLM2.SYSIN4.

В листинге второго шага трансляции выдаются следующие таблицы:

таблица адресов выполняемых операторов. Она включает в себя номер оператора и после знака равенства адрес в шестнадцатеричном коде, с которого этот оператор располагается в памяти МП;

таблица имен используемых переменных и их адреса в памяти МП;

текст программы на языке ассемблера для МП КР580.

Таблицы создаются с помощью следующих операторов:

@INPUT=1 — ввод данных из файла;

@GENERATE=1 — выдача содержимого памяти;

@FINISH=1 — печать таблицы ссылок.

Загрузочный модуль, полученный в результате трансляции, может быть выдан на перфоленгу, которая через считывающее устройство вводится в программатор.

Для получения итоговой перфоленги в выходном потоке второго шага в операторе DD с именем PLM2.PTOUT указывается параметр UNIT=009.

Техническое обеспечение. Кросс-транслятор устанавливается на ЕС ЭВМ с емкостью памяти не менее 512 Кбайт, имеющую стандартный набор устройств

прямого доступа к памяти типа 5050 или 5061, магнитные ленты типа 5010, устройство ввода с перфокарт типа ЕС-6012, устройство вывода на перфоленду типа ЕС-7903.

Кросс-компилятор работает с операционной системой ОС ЕС ЭВМ версии 4.1 и выше.

При адаптации кросс-компилятора на ВЦ используются системные библиотеки и библиотека загрузочных модулей с именем MICROLOAD. Применяются следующие машинные носители:

магнитная лента — для хранения копий с текстами загрузочных модулей программ PL#E1 и PL#E2;

магнитные диски — для хранения системных библиотек загрузочных модулей с программами PL#E1 и PL#E2, рабочих наборов транслятора и операционной системы;

перфокарты — для подготовки на них заданий на трансляцию и для перфорации на них операторов программ, написанных на языке ПЛ/М;

перфоленда для вывода на нее загрузочного модуля, используемого для записи в ППЗУ и ПЗУ.

При заполнении бланков текст программы должен записываться в позициях со 2-й по 72-ю. Использование позиций с 73-й по 80-ю недопустимо. В одной строке может быть несколько операторов.

Инструкция оператору при решении задачи:

установить пакет магнитных дисков с библиотекой, содержащей загрузочные модули PL#E1 и PL#E2;

включить устройство перфокарточного ввода;

нажать кнопку «Конец картотеки»; если емкость памяти не более 512 Кбайт, нажать кнопку «Пуск». В случае необходимости включить устройство вывода на перфоленду;

для надежности работы системы снять копию библиотеки, содержащей загрузочные модули PL#E1 и PL#E2 на магнитной ленте. На магнитной ленте хранить небольшую библиотеку, включающую процедуры, необходимые для работы с кросс-компилятором, в частности процедуру MCPLM.

4.2.2. Технологический процесс подготовки программ

Этот процесс можно разбить на два этапа. Первый — работа на ВЦ, получение листингов и исправление ошибок. При этом можно использовать моделирующую

программу и различные системные библиотеки. На различных этапах работы программный модуль может быть помещен в библиотеку исходных или загрузочных модулей. После отладки с помощью кросс-системы загрузочный модуль выводится на перфоленду для передачи на второй этап отладки, он может быть сохранен в библиотеке загрузочных модулей, если возникнет необходимость внести изменения в программу, или передан в архивную библиотеку на машинном носителе (перфокарты, магнитные ленты, диски).

Второй этап — запись стандартным программатором загрузочного модуля в ППЗУ. После этого ППЗУ размещается в микропроцессорной системе при помощи специальных гнезд-розеток, получивших название сокет (от английского socket — гнездо). В этой системе проходит проверка работы программы с внешними устройствами. После отладки и устранения ошибок полученный загрузочный модуль вновь выводится на перфоленду, далее возможно массовое тиражирование ПЗУ.

Перечислим основные этапы технологического процесса подготовки программного обеспечения для пакетного режима работы на ВЦ:

1) составление алгоритма, разработка программы на языке ПЛ/М и составление управляющих заданий для вызова процедуры трансляции, перенесение программы на стандартные бланки;

2) перфорация и получение пакета перфокарт;

3) ввод в ЭВМ пакета перфокарт и получение после трансляции и исправления ошибок загрузочного модуля;

4) представление загрузочного модуля в виде листинга, перфоленды для программатора или записи в библиотеку загрузочных модулей.

Запуск кросс-компилятора языка ПЛ/М для трансляции осуществляется с помощью специальной процедуры MCPLM. Эта процедура, записанная в системную библиотеку, состоит из двух блоков, имеющих соответствующие имена PLM1 и PLM2. Начальные операторы листинга процедуры представлены на с. 150.

Управление процедурой MCPLM производится с помощью параметров ID, DS, OB, OV, V, VL, R, LIB, NP.

ID — задает распределение памяти на устройство прямого доступа и указывает расположение всех наборов данных, используемых транслятором на первом

```
//MCPLM PROC ID=' ,SPACE=(TRK,(2,2),RLSE), DISP=(MOD,PASS)',
// DS='A,DISP=(,PASS), SPACE=(TRK,(2,2))', PROG=,
// OB='OBJPLM, DISP=(,PASS), SPACE=(TRK,(2,1))',
// PROG='PRT=SYSDA, DSL=,OUT='SYSOUT=A',
// ,PT='DUMMY',
// OV=DWC014, V=DWC014, VL=DWC014, R=180, NP=2
//PLM1 EXEC PGM=PL=E1, REGION=150K, DPRTY=(11,11)
//STEPLIB DD DSN=MICRO.LOAD, DISP=SHR,
// ПОВТОРЕНИЕ ЗАДАНИЯ НА ТРАНСЛЯЦИЮ
```

шаге и передаваемых на второй шаг. Пример использования параметра ID:

```
ID=' ,SPACE=(TRK,(2,2),RLSE),
DISP=(MOD,PASS)'
```

DS — распределяет память под рабочие наборы данных, описанных в операторе DD с именем FT10F001. Пример использования параметра DS:

```
DS='&&A,DISP=(,PASS),SPACE=(TRK,(2,2))'
```

OB — распределяет память под рабочие наборы данных, представленных в операторе с именем FT11F001. Пример использования параметра OB:

```
OB='&&OBJPLM,DISP=(,PASS),SPACE=(TRK(2(1))'
```

Два знака && показывают, что набор данных временный. Этот набор передается на второй шаг трансляции.

OV — задает распределение памяти на устройстве DWC014.

V — определяет имя пакета магнитных дисков, на котором расположен набор, описанный параметром DS. Пример использования параметра V:

```
V=DWC014.
```

VL — задает имя магнитного диска, на котором расположена библиотека загрузочных модулей, где хранятся программы PL#E1 и PL#E2, предназначенные для выполнения первого и второго шага трансляции.

LIB — задает имя библиотеки загрузочных модулей, содержащей программы PL#E1 и PL#E2. Этот параметр связан с параметром VL, задающим имя магнитного диска. Пример использования параметра LIB:

```
VL=DWC014,LIB='CRM.LOAD'
```

R — задает емкость оперативной памяти, используемой кросс-компилятором для трансляции с языка ПЛ/М как на первом, так и на втором шаге. Обычно

значение этого параметра равно 180 Кбайт. Пример использования параметра R:

```
R=180K.
```

NP — задает номер стандартного списка управляющих операторов, употребляемых кросс-транслятором. Этот параметр введен для облегчения применения кросс-компилятора лицами, не освоившими глубоко язык управляющих операторов. Список управляющих операторов располагается в системной библиотеке с именем SYS1.PROCLIB. Пример использования параметра NP:

NP=2 (т. е. второй набор операторов).

Запуск программ осуществляется с помощью вышеописанной процедуры следующим образом:

```
//имя задания JOB параметры
//имя пункта EXEC MCPLM параметры процедуры
//PLM1. SYSIN DD *
// ПУСТАЯ ПЕРФОКАРТА
// ТЕКСТ ПРОГРАММЫ НА ПЛ/М
// EOF
//
```

Это задание — простейший вариант запуска программ при трансляции с помощью процедуры MCPLM, когда список управляющих параметров для первого и второго шага подготовлен заранее (используется параметр процедуры NP). Если приводится нестандартное задание параметров и нестандартный оператор DD, задание на запуск процедуры MCPLM может выглядеть таким образом:

```
//имя задания JOB параметры
//имя пункта EXEC MCPLM параметры процедуры
//PLM1. SYSIN1 DD *
// ПУСТАЯ ПЕРФОКАРТА
// ТЕКСТ ПРОГРАММЫ НА ЯЗЫКЕ ПЛ/М
//PLM1. SYSIN2 DD *
// УПРАВЛЯЮЩИЕ ОПЕРАТОРЫ ДЛЯ ПЕРВОГО ШАГА
//PLM2. PTOUT DD UNIT=009
//PLM2. SYSIN4 DD *
// УПРАВЛЯЮЩИЕ ОПЕРАТОРЫ ДЛЯ ВТОРОГО ШАГА
```

В этом примере текст программы на языке ПЛ/М вводится с перфокарт, а загрузочный модуль выводится на перфоленту; операторы DD с именами SYSIN2 и SYSIN4 позволяют ввести необходимые управляющие параметры для первого и второго шага трансляции.

Повышение требований к микропроцессорной технике и совершенствование технологии изготовления БИС обусловили создание 16-разрядного МП типа КМ1810ВМ86, совместимого с МП КР580 программно. Основные достоинства нового МП — увеличение адресного пространства до 1 Мбайта, возможность обработки различных типов данных, разнообразие способов адресации, развитая структура прерываний, возможность работы в многопроцессорных системах.

МП КМ1810 состоит из следующих блоков: двунаправленного буфера адреса/данных, устройства сопряжения с каналом, устройства обработки информации, устройства управления и синхронизации.

Устройство сопряжения с каналом позволяет формировать 20-разрядный адрес памяти, выбирать команды из памяти, формировать очереди команд и запоминать результаты выполнения команд в памяти. Устройство содержит шесть 8-разрядных регистров очереди команд, четыре 16-разрядных сегментных регистра, а также регистры адреса, обмена и сумматор адреса.

Устройство обработки информации связано с устройством сопряжения посредством 8-разрядной магистрали команд и 16-разрядной шины данных. В состав устройства обработки информации входят: 16-разрядное АЛУ, восемь 16-разрядных регистров общего назначения и регистр признаков состояния МП, а также микропрограммное устройство управления.

Устройство управления и синхронизации позволяет изменять конфигурацию путем системной «перестройки» МП, осуществляемой подачей управляющего сигнала на специальный вывод — выбор режима. Существует два режима работы МП: в вычислительных системах с минимальной и максимальной конфигурацией. Второй режим позволяет строить многопроцессорные вычислительные системы.

Система команд МП КМ1810 содержит 135 базовых команд, представляемых 1—6 байтами. В первом байте команды размещается код операции, а в остальных байтах — адресная информация. В командах используются следующие способы адресации: регистровая, непосредственная, абсолютная, косвенная, простая и индексированная адресация по базе и относительная. Коман-

ды МП можно разделить на следующие группы: команды пересылки данных, арифметические, логические и сдвиговые команды, управления и обработки строк. Команды могут оперировать как с байтами, так и с двухбайтными словами, выполняя операции умножения и деления, а также команды над десятичными числами.

Разветвленная система команд обеспечивает эффективную работу с языками высокого уровня, в том числе с расширенным языком ПЛ/М. Расширение языка осуществлено за счет введения новых типов данных и нового типа представления адресов POINTER с оператором обращения @, значительного увеличения встроенных процедур. Несмотря на эти изменения в языке, программы, составленные для 8-разрядных МП, могут быть использованы и для МП КМ1810 с учетом некоторых ограничений.

Расширенный язык ПЛ/М позволяет иметь несколько типов переменных:

BYTE — целые однобайтные числа без знака;
WORD — целые двухбайтные числа без знака;
INTEGER — знаковые целые числа;
REAL — знаковые числа в форме с плавающей запятой.

Диапазон представления переменных типа INTEGER и REAL зависит от состава вычислительной системы и от параметров используемого компилятора.

В зависимости от представления данных, объявленных в декларациях программы, имеются три типа арифметических операций: беззнаковые, знаковые и с плавающей запятой.

Для работы с адресами — получения, сравнения, изменения — используется указатель адресов типа POINTER, позволяющий вычислять 20-разрядные адреса.

В расширенном языке ПЛ/М имеется большое количество встроенных процедур, среди них больше всего процедур преобразования формата данных, учитывающих знак. В язык введено много процедур, осуществляющих обработку строк, их пересылку по заданному адресу, сравнение двух строк, преобразование строк с использованием таблицы объемом в 256 значений, поиск одинаковых элементов в строке, производимый как по восходящему, так и по нисходящему принципу, и установкой каждого элемента строки в определенное значение.

Обращение к портам МП выполняется с помощью системы ввода-вывода информации. При адресации портов используется собственное адресное пространство, которое дает возможность обращаться к большому числу портов, передавая или получая одновременно один или два байта. Для этих целей в языке предназначены процедуры вывода OUTPUT и OUTWORD и процедуры ввода INPUT и INWORD. Так же как и в МП КР580, в МП КМ1810 обращение к портам осуществляется через аккумулятор.

В МП КМ1810 реализована двухуровневая приоритетная система прерывания по вектору. Прерывания обслуживаются непосредственно после выполнения текущей команды, но при обработке строк работа МП может прерываться. Прерываниями управляют встроенные процедуры, аналогичные используемым в языке ПЛ/М для МП КР580.

Предусмотрены специальные прерывания:
в случае ошибки при делении;
для пошаговой работы в режиме отладки;
по контрольной точке;
по переполнению.

Существует процедура управления работой ЗУ при организации многопроцессорной вычислительной системы, которая может быть использована и для целей синхронизации.

Еще одним отличием расширенного языка ПЛ/М является работа итерационного блока DO. Индексная переменная для этого блока может принимать значения как типа BYTE, WORD, так и типа INTEGER, причем работа блока в случае применения первых двух типов переменных ничем не отличается от работы в случае использования обычного языка. Если индексная переменная имеет тип INTEGER, то блок работает в следующей последовательности:

индексной переменной присваивается значение «начало», а значения «окончание» и «шаг» вычисляются так, как будто они имеют тип INTEGER;

если значение шага отрицательно, а значение индексной переменной меньше, чем значение «окончание», выполнение блока заканчивается и управление передается оператору, следующему за оператором END;

если значение шага неотрицательно, а значение индексной переменной больше, чем значение «окончание»,

то выполнение блока заканчивается и управление передается оператору, следующему за оператором END;

если работа итерационного блока не заканчивается на одном из двух предыдущих этапов, то исполняются внутренние операторы блока DO, и после выполнения последнего исполняемого оператора переменная увеличивается на величину шага, и работа блока DO продолжается.

При обращении к элементам массивов необязательно использовать только целочисленные константы BYTE или WORD, могут употребляться и INTEGER. При задании базированных переменных рекомендуется применять тип POINTER, а не WORD.

ЛИТЕРАТУРА

1. Каган Б. М., Сташин В. В. Микропроцессоры в цифровых системах. — М.: Энергия, 1979. — 193 с.
2. PL/M-80 Programming Manual. — Intel Corp. USA, 1977. — 106 p.
3. Mc Crachen D. D. A Guide to PL/M Programming for Microcomputer Application-SI, Addison-Wisley, 1978. — 262 p.
4. Мыскин А. В., Торгашев В. А. Инструкция по программированию PL/M-80. — Л.: ЛНИВЦ АН СССР, 1980. — 63 с.
5. Справочник по цифровой вычислительной технике. Программное обеспечение/Под ред. Б. Н. Малиновского. — Киев: Техника, 1981. — 207 с.
6. Фаулджер Р. Программирование встроенных микропроцессоров. — М.: Мир, 1985. — 275 с.
7. Duncan F. G. Microprocessor programming and Software Development. — London: Prentice-Hall, International, 1979. — 320 p.
8. Гнатек Ю. Р. Справочник по цифроаналоговым и аналого-цифровым преобразователям. — М.: Радио и связь, 1982. — 552 с.
9. Микропроцессоры: системы программирования и отладки/Под ред. В. А. Мясникова, М. Б. Игнатъева. — М.: Энергоатомиздат, 1985. — 272 с.
10. Корниенко Г. И., Барсук Я. И., Пих З. А. и др. О средствах создания математического обеспечения микропроцессорных систем. — Кибернетика, 1979, № 1, с. 41—46.
11. Резидентный транслятор с языка PL/M для системы Intellec. — Электроника, 1977, № 2, т. 50. — 88 с.
12. Слободянюк А. И., Погорелый С. Д., Вайсбанд С. Г. Кроссовая система отладки программного обеспечения микропроцессора K580ИК80 на малых ЭВМ. — Управляющие системы и машины, 1982, № 2, с. 38—41.

ПРИЛОЖЕНИЕ

КОД КОИ-7 (НАБОР 3)

B ₇	0	0	0	0	1	1	1	1
B ₆	0	0	1	1	0	0	1	1
B ₅	0	1	0	1	0	1	0	1

B ₇	B ₆	B ₅	B ₄	B ₃	B ₂	B ₁
			0	0	0	0
			0	0	0	1
			0	0	1	0
			0	0	1	1
			0	1	0	0
			0	1	0	1
			0	1	1	1
			1	0	0	0
			1	0	0	1
			1	0	1	0
			1	0	1	1
			1	1	0	0
			1	1	0	1
			1	1	1	0
			1	1	1	1

№ 0	1	2	3	4	5	6	7
0 ПУС	AP1		0 @	Р	Ю	П	
1 НЗ	СУ1	!	1 А	Q	А	Я	
2 НТ	СУ2	"	2 В	Р	Б	Р	
3 КТ	СУЗ	#	3 С	С	Ц	С	
4 КП	СТП	Ø	4 D	T	Д	T	
5 КТМ	НЕТ	%	5 E	У	Е	У	
6 ДА	СИН	&	6 F	V	Ф	Ж	
7 ЗВ	КБ	,	7 G	W	Г	В	
8 ВШ	А	(	8 H	X	Х	Ь	
9 ГТ	КН	)	9 I	У	И	Ы	
А ПС	ЗМ	*	: J	Z	Й	З	
В ВТ	АР2	+	; K	[	К	Ш	
С ПФ	РФ	,	< L	\	Л	Э	
D ВК	РГ	-	= M	1	М	Щ	
E Вых	РЗ	.	> N	Λ	Н	Ч	
F ВХ	РЗ	/	? O	-	О	ЗБ	

**УКАЗАТЕЛЬ ОПЕРАТОРОВ,
ВСТРОЕННЫХ ФУНКЦИЙ И ПРОЦЕДУР ЯЗЫКА ПЛ/М**

ADDRESS	29*, 101	MEMORY	39, 98
AND	43, 103	MINUS	31
AT	30, 102	MOD	31, 121
BASED	71	MOVE	105
BY	55	NOT	43, 126
BYTE	29, 101	OR	43, 136
CALL	67, 103	OUTPUT	38, 109
CARRY	87	PARITY	39
CASE	48, 99	PLUS	31
DATA	37, 99	PROCEDU-	
DEC	88	RE	65, 99
DECLARE	29, 101	PUBLIC	78, 98
DISABLE	92	RETURN	66, 99
DO	46, 101	ROL	86, 121
DOUBLE	39, 101	ROR	86
ELSE	45, 103	HALT	18, 117
ENABLE	92	HIGH	39, 125
END	46, 99	SCL	87
EOF	40	SCR	87
EXTERNAL	78, 99	SIGN	39
GO TO	41, 101	SHL	88, 112
(GOTO)		SHR	88, 125
IF	45, 103	STACKPTR	39
INITIAL	36, 107	STRUCTURE	60, 111
INPUT	38, 113	TIME	70, 109
INTERRUPT	92, 135	THEN	45, 103
LABEL	81	TO	53, 117
LAST	59	WHILE	52, 116
LENGTH	59	XOR	43, 113
LITERALLY	85, 103	ZERO	39
LOW	39, 124		

* Первая ссылка относится к описанию языка, а остальные — к практическому примеру.

Предисловие	3
Глава 1. Микропроцессорные системы. Их устройство и программирование	5
1.1. Микропроцессоры	5
1.2. Микропроцессор КР580	8
1.3. Микропроцессорные системы	13
1.4. Программирование микропроцессорных систем	15
1.4.1. Система команд МП КР580	16
1.4.2. Программирование в машинных кодах	18
1.4.3. Языки программирования микропроцессорных систем	20
Глава 2. Описание языка ПЛ/М	24
2.1. Алфавит	24
2.2. Переменные и константы	26
2.3. Ключевые слова	27
2.4. Числовые константы	27
2.5. Строки	28
2.6. Описание переменных	29
2.7. Арифметические выражения	30
2.8. Операторы и комментарии	32
2.9. Линейные программы	33
2.9.1. Оператор присваивания	33
2.9.2. Назначение начальных значений переменным	35
2.9.3. Назначение имен константам	37
2.9.4. Понятие о встроенных функциях	37
2.9.5. Структура простой программы	40
2.10. Разветвляющиеся программы	40
2.10.1. Метки и оператор перехода	40
2.10.2. Операции отношения и логические операции	42
2.10.3. Условный оператор	45
2.10.4. Составной оператор	46
2.10.5. Оператор DO CASE	48
2.11. Циклические программы	50
2.11.1. Оператор DO WHILE	50
2.11.2. Итеративный цикл	53
2.11.3. Вложенные циклы	56
2.11.4. Массивы	56
2.11.5. Встроенные функции для массивов	59
2.11.6. Структуры	60
2.12. Процедуры	64
2.12.1. Назначение процедур	64
2.12.2. Описание процедуры	65
2.12.3. Обращение к процедуре	67
2.13. Базированные переменные и адресные ссылки	70
2.13.1. Базированные переменные	70
2.13.2. Адресные ссылки	72
2.14. Блочная структура программы	74
2.14.1. Объекты и область их существования	74

2.14.2. Описание и область существования меток	81
2.14.3. Особенности использования блоков	84
2.15. Макроопределения	85
2.16. Встроенные функции сдвига кодов	86
2.17. Встроенные функции десятичной арифметики	88
2.18. Средства обработки прерываний	91
Глава 3. Применение языка ПЛ/М для программирования микропроцессорных систем	95
3.1. Управляющие программы микропроцессорных систем	95
3.2. Программирование управляющих микропроцессорных систем	109
3.3. Прикладные программы для микропроцессорных систем	118
3.4. Программная реализация сопряжения МП с внешними устройствами в микропроцессорных системах	130
Глава 4. Подготовка программ для микропроцессорных систем	137
4.1. Резидентный транслятор с языка ПЛ/М	138
4.2. Кросс-транслятор для языка ПЛ/М	140
4.2.1. Описание задания на запуск кросс-транслятора	144
4.2.2. Технологический процесс подготовки программ	148
4.3. Расширение языка ПЛ/М	152
Литература	156
Приложение. Код КОИ-7 (набор 3)	157
Указатель операторов, встроенных функций и процедур языка ПЛ/М	158

Практическое руководство

**Леонид Кузьмич Гребенников,
Леонид Александрович Летник**

ПРОГРАММИРОВАНИЕ МИКРОПРОЦЕССОРНЫХ СИСТЕМ НА ЯЗЫКЕ ПЛ/М

Зав. редакцией **И. Г. Дмитриева**
Редактор **Л. А. Табакова**
Мл. редактор **Т. А. Студеникина**
Худож. редактор **М. К. Гуров**
Техн. редактор **Г. А. Полякова**
Корректоры **Т. М. Колпакова, Н. П. Сперанская**
Обложка художника **М. К. Гурова**

ИБ № 1874

Сдано в набор 19.02.86. Подписано в печать 07.07.86. А13372.
Формат 84×108¹/₃₂. Бум. кн.-журн. Гарнигура «Литературная».
Печать высокая. Усл. п. л. 8,4. Усл. кр.-отг. 8,61. Уч.-изд. л.
7,77. Тираж 40 000 экз. Заказ 1918. Цена 45 коп.

Издательство «Финансы и статистика», 101000, Москва,
ул. Чернышевского, 7.
Областная типография управления издательств, полиграфии
и книжной торговли Ивановского облисполкома, 153628, г. Ива-
ново, ул. Типографская, 6.